

Robotic Motion Planning: Sample-Based Motion Planning

Part 1

Path-Planning in High Dimensions

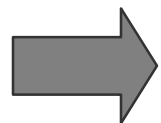
- IDEAL:

Build a **complete** motion planner

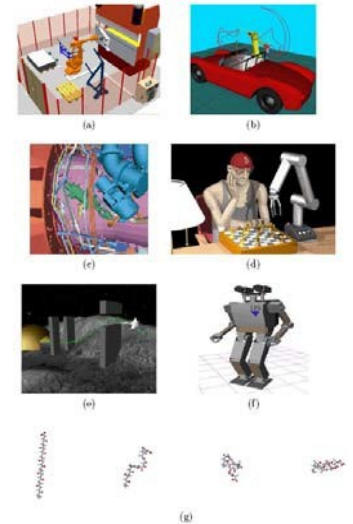
- PROBLEM:

Complexity is exponential in the dimension of the robot's C-space [Canny 86]

Building Configuration Space



Heuristic algorithms trade off completeness for practical efficiency. Weaker performance guarantee.



Ways to Simplify Problem

- Project search to lower-dimensional space
- Limit the number of possibilities
(add constraints, reduce “volume”
of free space)
- Sacrifice optimality, completeness

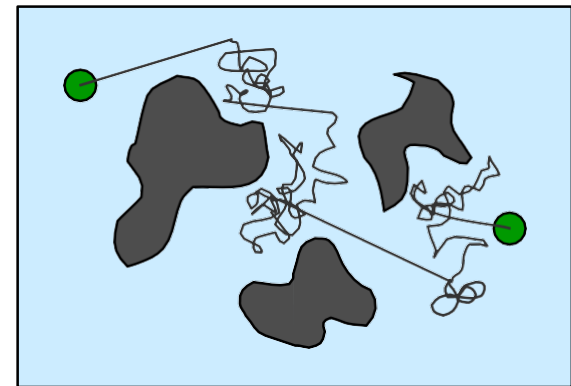
The Rise of Monte Carlo Techniques

- KEY IDEA:
Rather than exhaustively explore ALL possibilities, randomly explore a smaller subset of possibilities while keeping track of progress
- Facilities “probing” deeper in a search tree much earlier than any exhaustive algorithm can
- What’s the catch?
Typically we must sacrifice both *completeness* and *optimality*
Classic tradeoff between solution quality and runtime performance

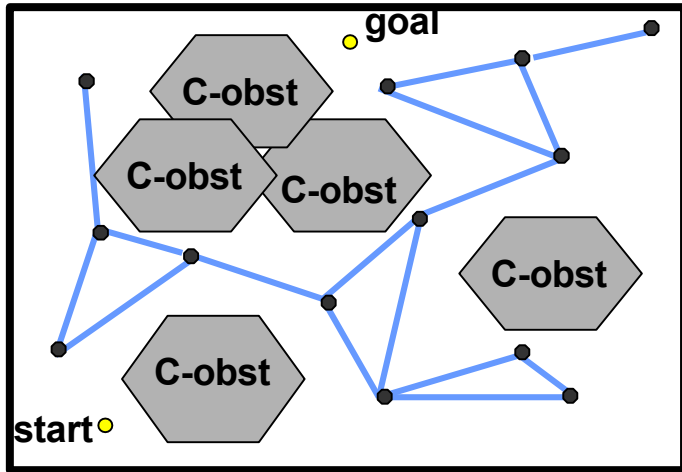
Sampling Based Planning:

Search for collision-free path only by sampling points.

EXAMPLE: Potential-Field



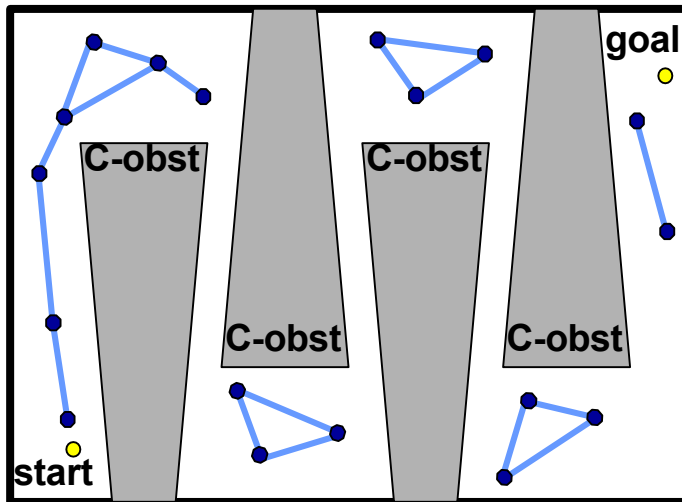
Good news, but bad news too



Sample-based: The Good News

1. *probabilistically complete*
2. Do not construct the C-space
3. apply easily to high-dimensional C-space
4. support fast queries w/ enough preprocessing

Many success stories where PRMs solve previously unsolved problems



Sample-Based: The Bad News

1. don't work as well for some problems:
 - unlikely to sample nodes in narrow passages
 - hard to sample/connect nodes on constraint surfaces
2. No optimality or completeness

Everyone is doing it.

• Probabilistic Roadmap Methods

- Uniform Sampling (original) [Kavraki, Latombe, Overmars, Svestka, 92, 94, 96]
- Obstacle-based PRM (OBPRM) [Amato et al, 98]
- PRM Roadmaps in Dilated Free space [Hsu et al, 98]
- Gaussian Sampling PRMs [Boor/Overmars/van der Steppen 99]
- PRM for Closed Chain Systems [Lavage/Yakey/Kavraki 99, Han/Amato 00]
- PRM for Flexible/Deformable Objects [Kavraki et al 98, Bayazit/Lien/Amato 01]
- Visibility Roadmaps [Laumond et al 99]
- Using Medial Axis [Kavraki et al 99, Lien/Thomas/Wilmarth/Amato/Stiller 99, 03, Lin et al 00]
- Generating Contact Configurations [Xiao et al 99]
- Single Shot [Vallejo/Remmler/Amato 01]
- Bio-Applications: Protein Folding [Song/Thomas/Amato 01,02,03, Apaydin et al 01,02]
- Lazy Evaluation Methods: [Nielsen/Kavraki 00 Bohlin/Kavraki 00, Song/Miller/Amato 01, 03]
- Seth Hutchinson workspace-based approach, 2001

• Related Methods

- Ariadnes Clew Algorithm [Ahuactzin et al, 92]
- RRT (Rapidly Exploring Random Trees) [Lavage/Kuffner 99]

Overview

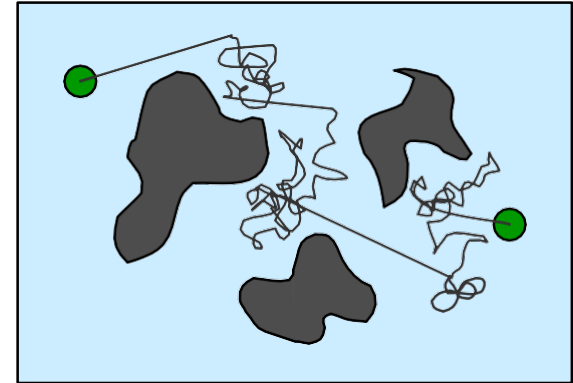
- Probabilistic
- RoadMap Planning (PRM) by Kavraki
 - samples to find free configurations
 - connects the configurations (creates a graph)
 - is designed to be a multi-query planner
- Expansive-Spaces Tree planner (EST) and Rapidly-exploring Random Tree planner (RRT)
 - are appropriate for single query problems
- Probabilistic Roadmap of Tree (PRT) combines both ideas

High-Dimensional Planning as of 1999

Single-Query:

Barraquand, Latombe '89; Mazer, Talbi, Ahuactzin, Bessiere '92; Hsu, Latombe, Motwani '97; Vallejo, Jones, Amato '99;

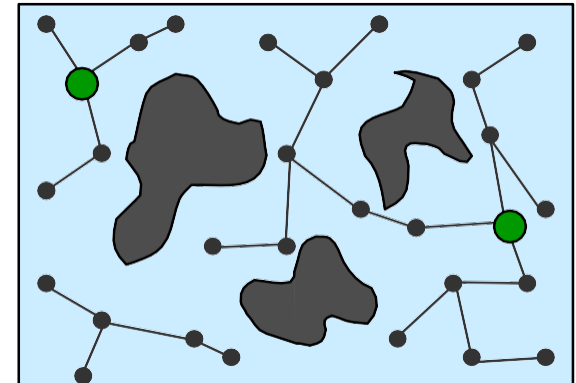
EXAMPLE: Potential-Field



Multiple-Query:

Kavraki, Svestka, Latombe, Overmars '95; Amato, Wu '96; Simeon, Laumond, Nissoux '99; Boor, Overmars, van der Stappen '99;

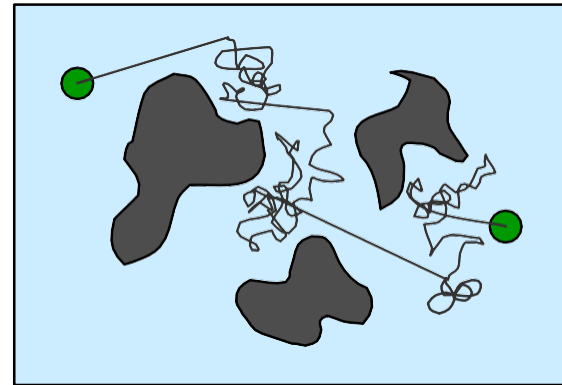
EXAMPLE: PRM



Randomized Potential Functions (Barranquand and Latome)

May take a long time in local minima

EXAMPLE: Potential-Field



Probabilistic Roadmaps

(Kavraki, Latombe and lots more)

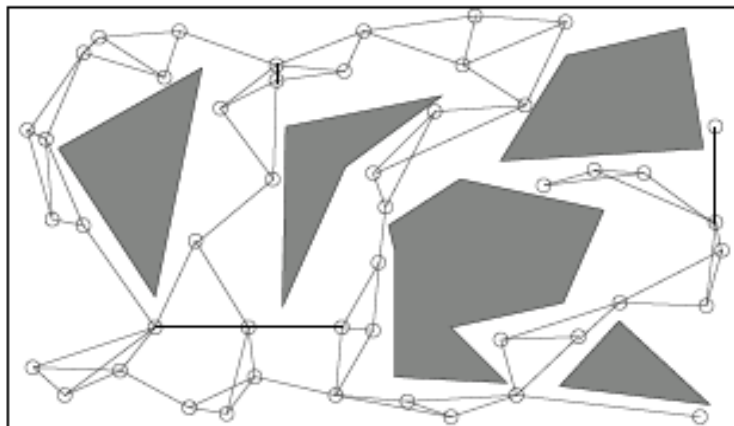
- Learning Phase
 - Construction Step
 - Expansion Step
- Query Phase

The Learning Phase

- Construct a probabilistic roadmap by generating random free configurations of the robot and connecting them using a simple, but very fast motion planner, also known as a *local planner*
- Store as a graph whose nodes are the configurations and whose edges are the paths computed by the *local planner*

Learning Phase (Construction Step)

- Initially, the graph $\mathbf{G} = (\mathbf{V}, \mathbf{E})$ is empty
- Then, repeatedly, a *random free configuration* is generated and added to $\mathbf{V}$
- For every new node c , select a number of nodes from $\mathbf{V}$ and try to connect c to each of them using the *local planner*.
- If a path is found between c and the selected node v , the edge (c,v) is added to $\mathbf{E}$. The path itself is not memorized (usually).



How do we determine a random free configuration?

- We want the nodes of V to be a rather **uniform** sampling of Q_{free}
 - Draw each of its coordinates from the interval of values of the corresponding degrees of freedom. (Use the uniform probability distribution over the interval)
 - Check for collision both with robot itself and with obstacles
 - If collision free, add to V , otherwise discard
 - What about rotations? Sample Euler angles gives samples near poles, what about quaternions?
- This is HUGE TOPIC, which we will get to later

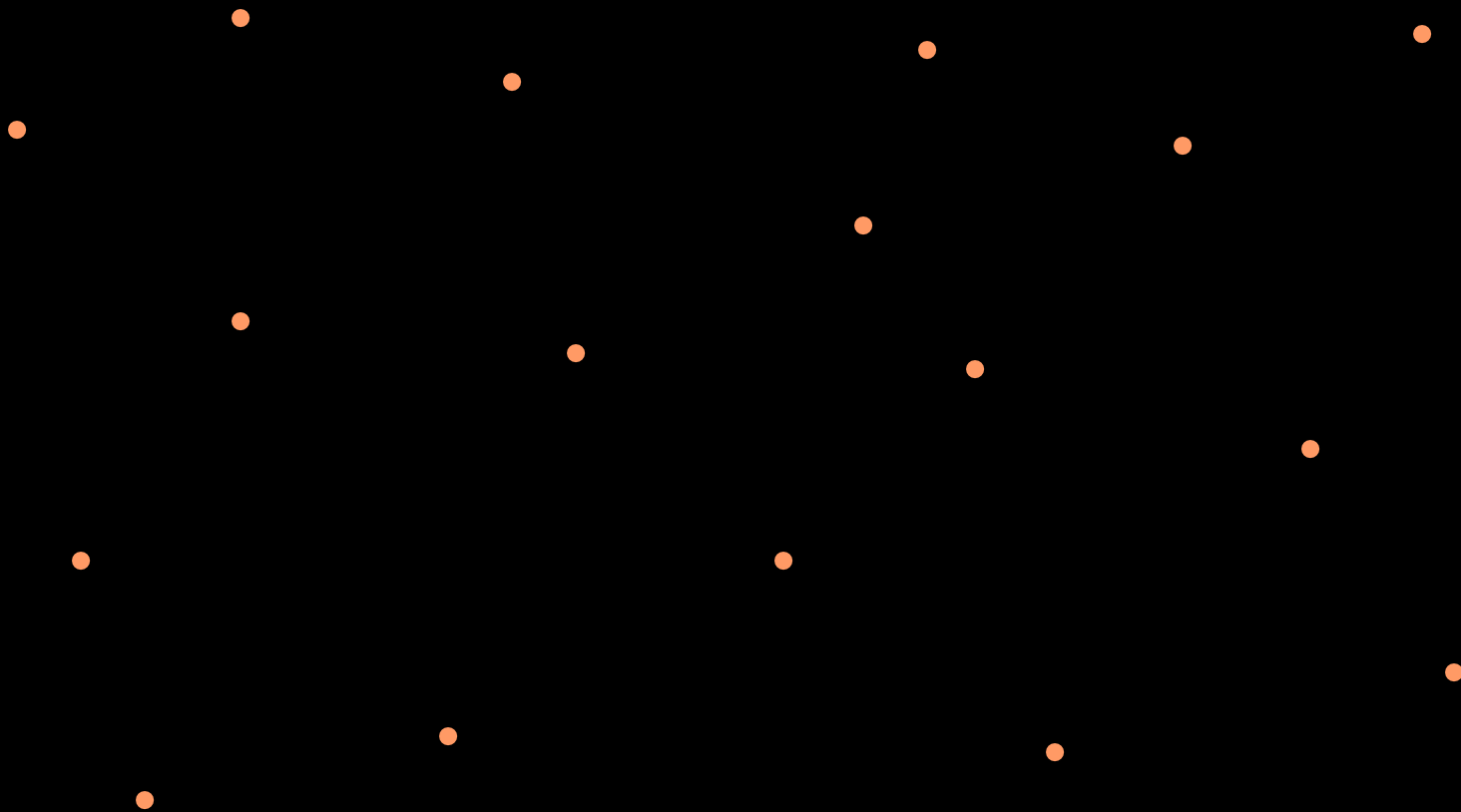
What's the local path planner?

- Can pick different ones
 - Nondeterministic – have to store local paths in roadmap
 - Powerful - slower but could take fewer nodes but takes more time
 - Fast - less powerful, needs more nodes

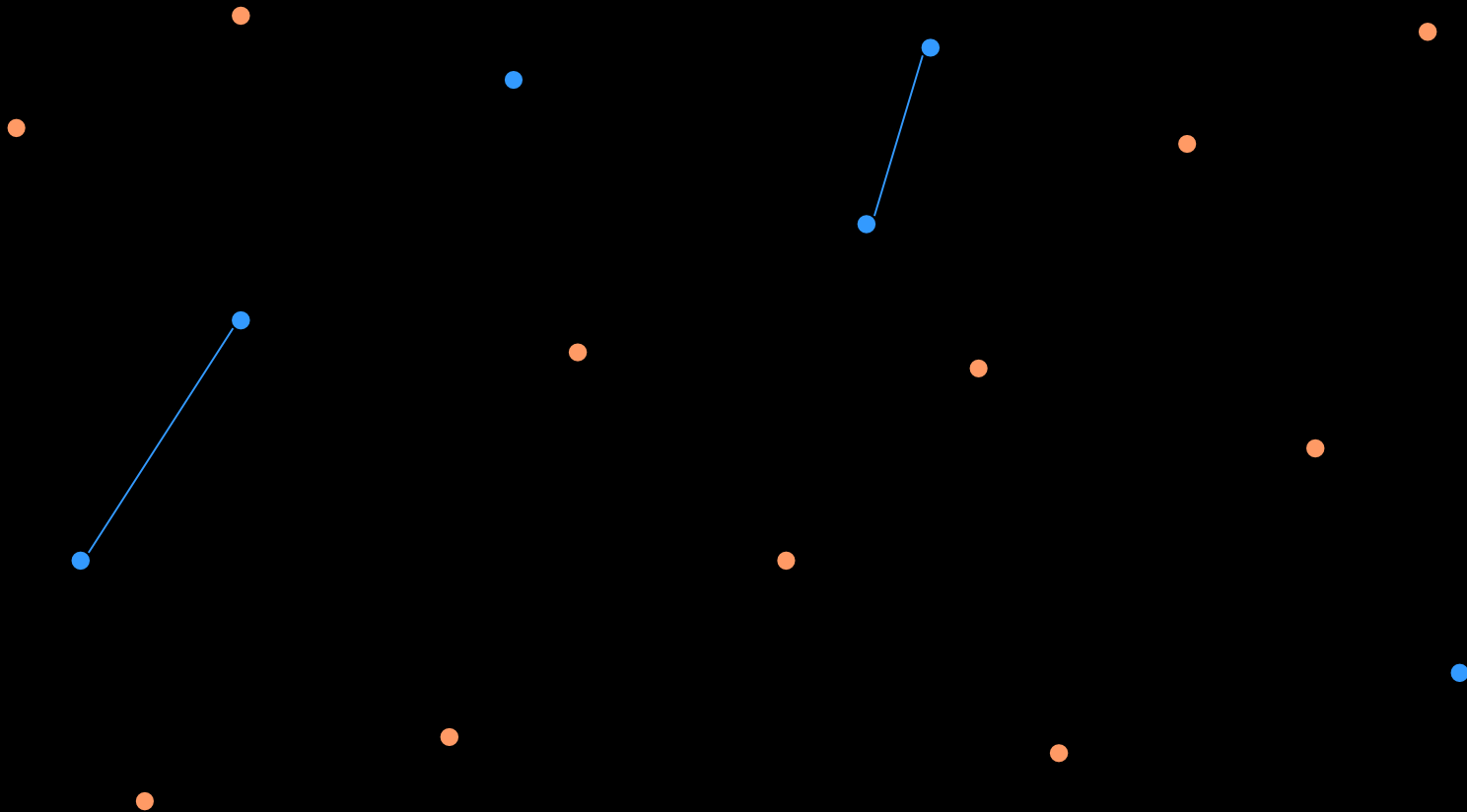
Go with the fast one

- Need to make sure start and goal configurations can connect to graph, which requires a somewhat dense roadmap
- Can reuse local planner at query time to connect start and goal configurations
- Don't need to memorize local paths

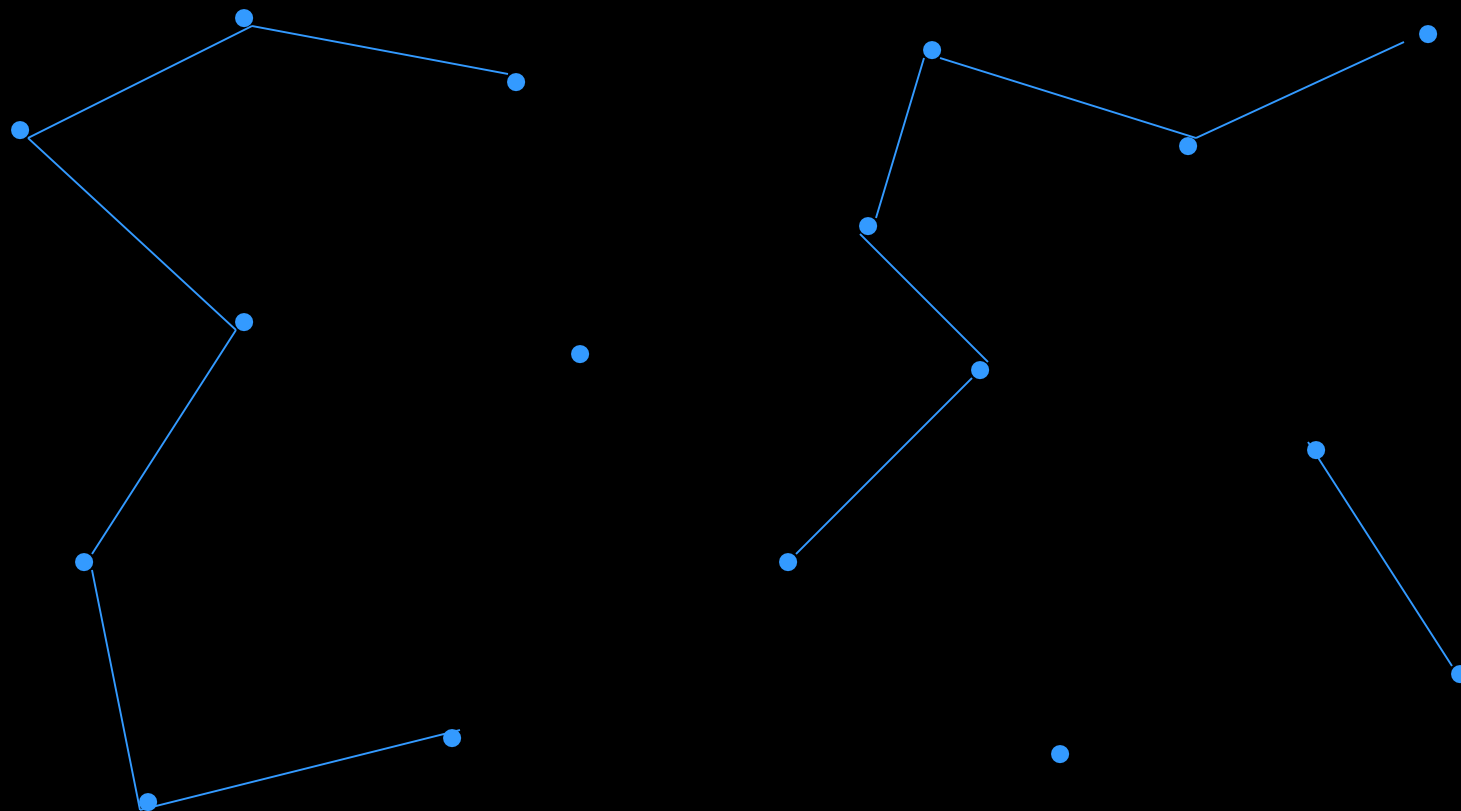
Create random configurations



Update Neighboring Nodes' Edges



End of Construction Step



Basic PRM, reviewed

- Goal: construct a graph $G=(V,E)$ where $e=(q_1,q_2)$ is an edge only if there is a collision-free path from q_1 to q_2
- Notation:
 - let Δ be a (deterministic) local planner that is correct (but may not be complete)
 - $D: Q \times Q \rightarrow [0,\infty]$ --- a distance function on Q

1. While $|V| < n$ do

- sample until a collision-free configuration q is found; add q to V

2. for all $q \in V$

 for all $q' \in N_q$ (k closest neighbors of q)

 if $\Delta(q,q') = \text{True}$

$E = E \cup \{(q,q')\}$

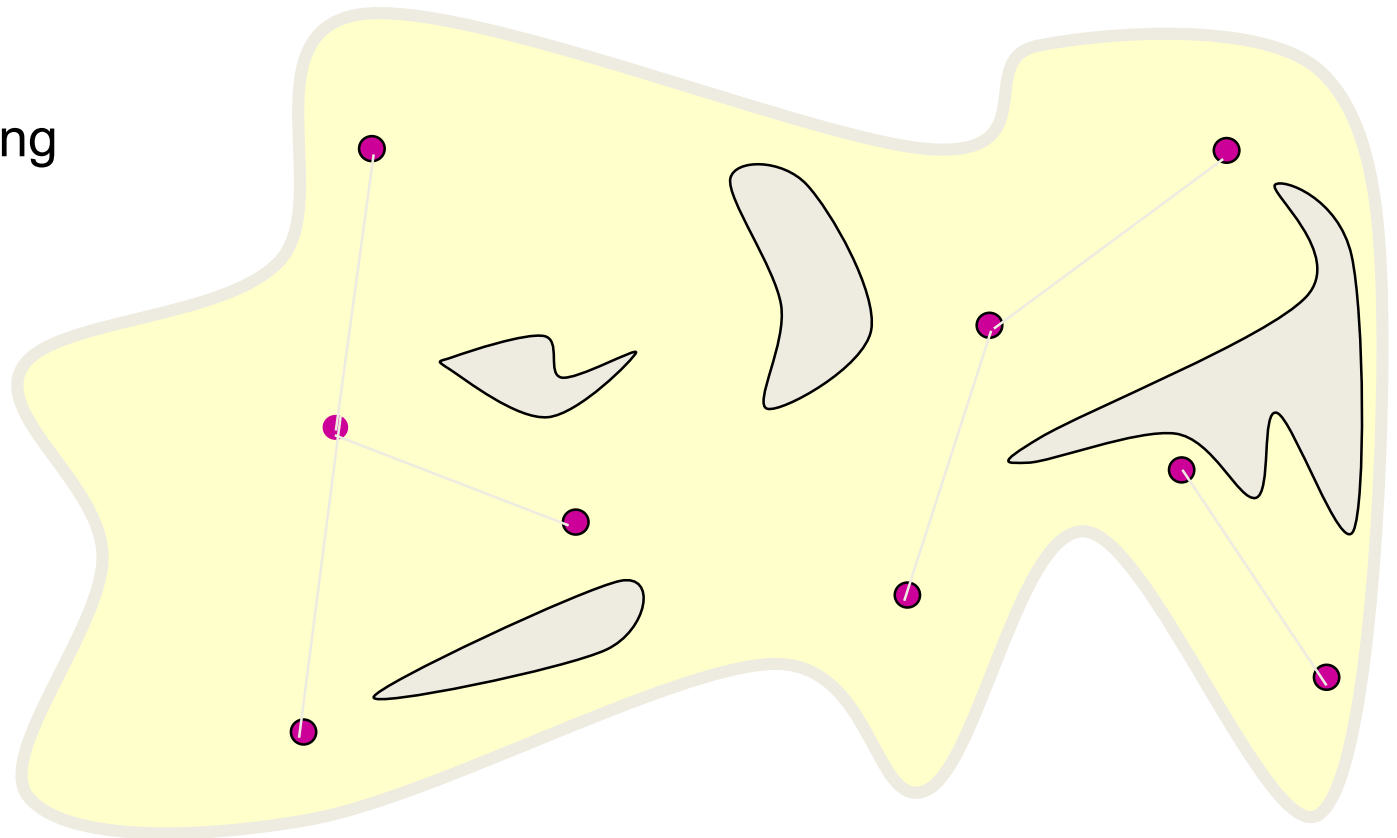
 end

 end

end

Connectedness of PRM

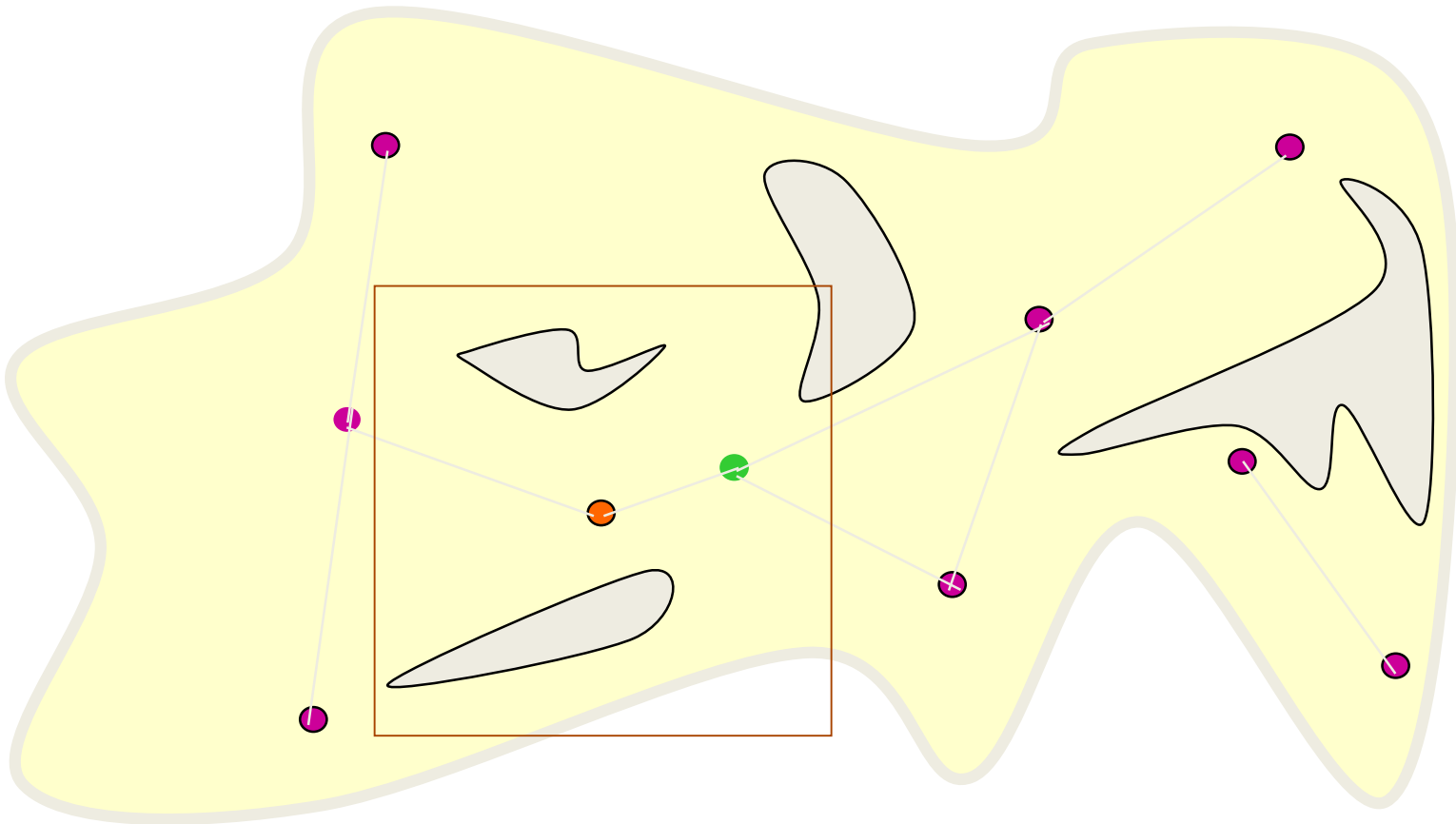
- Problem: Roadmap graph needs not to be connected although freespace is connected
- Idea: one can use intermediate sampling results to identify regions of the free space whose connectivity is more difficult to capture
- Multi stage planning



[Kavraki, 94]

Connectedness of PRM

- Observation: The connectivity of free space is more difficult to capture near its boundary than in wide-open area
- Obstacle sensitive strategies

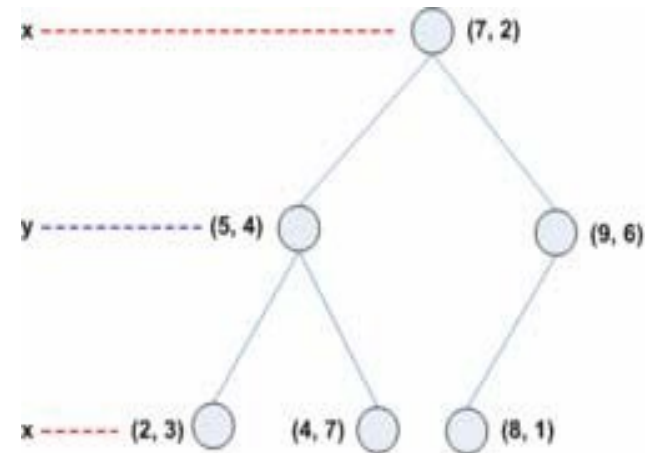
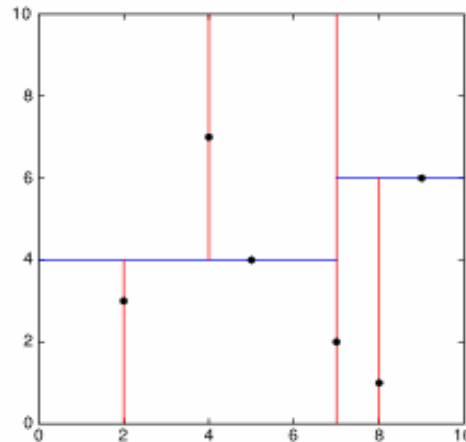


[Kavraki, 94]

Distance Functions

- Really, D should reflect the likelihood that the planner will fail to find a path
 - close points, likely to succeed
 - far away, less likely
- Ideally, this is probably related to the area swept out by the robot
 - very hard to compute exactly
 - usually heuristic distance is used
- Typical approaches
 - Euclidean distance on some embedding of c-space
 - $dist(q', q'') = \|emb(q') - emb(q'')\|$
 - Embedding is often based on control points (recall end of potential field chapter)
 - Alternative is to create a weighted combination of translation and rotational “distances”
 - Workspace volume
 - $dist(q', q'') = w_t \|X' - X''\| + w_r f(R', R'')$

Selecting Closest Neighbors

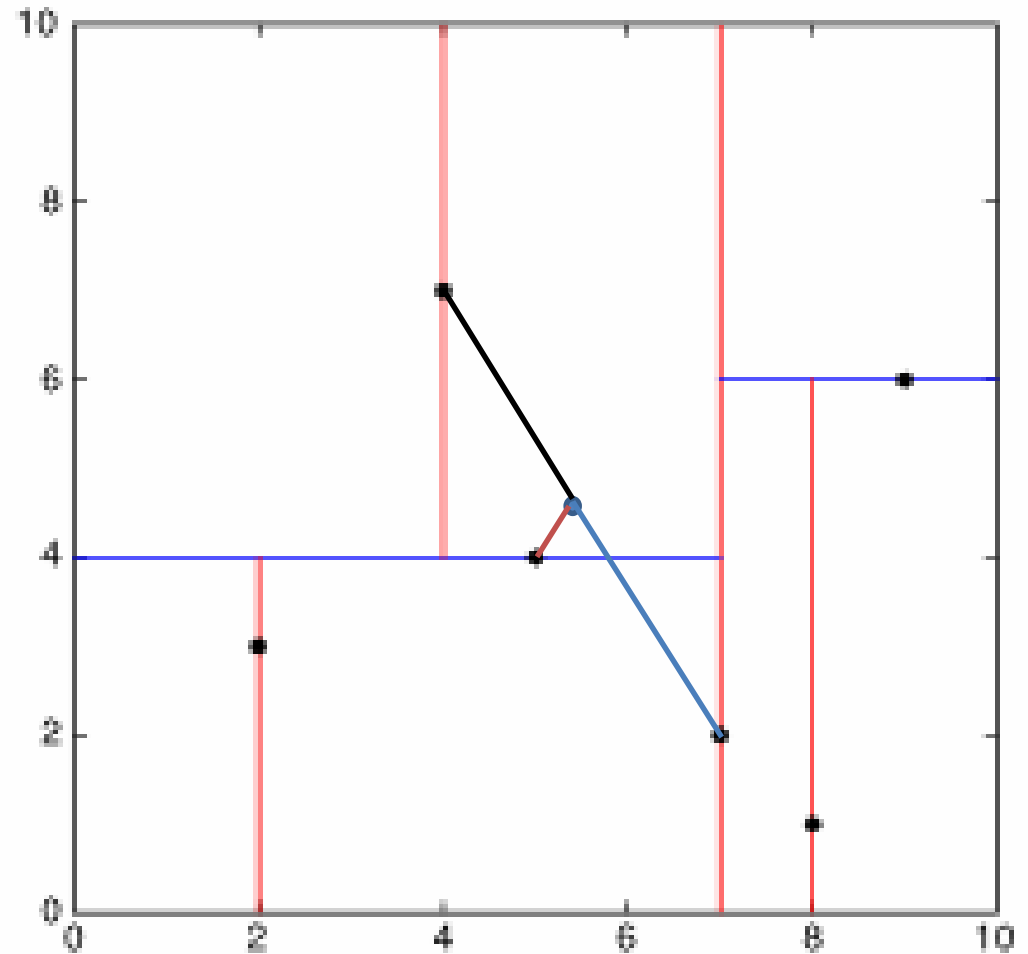
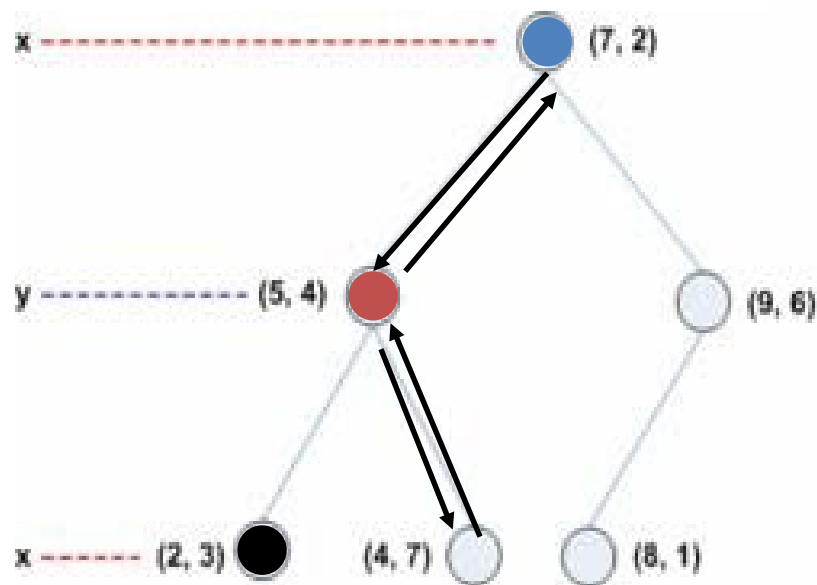


- k-d tree (construction)
 - Given: a set S of n points in k -dimensional space
 - Recursively
 - choose a plane P that splits S about evenly (usually in a coordinate dimension)
 - store P at node
 - apply to children S_l and S_r
 - Requires $O(kn)$ storage, built in $O(kn \log n)$ time
 - Query takes $O(n^{1-1/k} + m)$ time where m is # of neighbours
 - asymptotically linear in n and m with large k

Selecting Closest Neighbors

Nearest Neighbour Search (NN- Search)

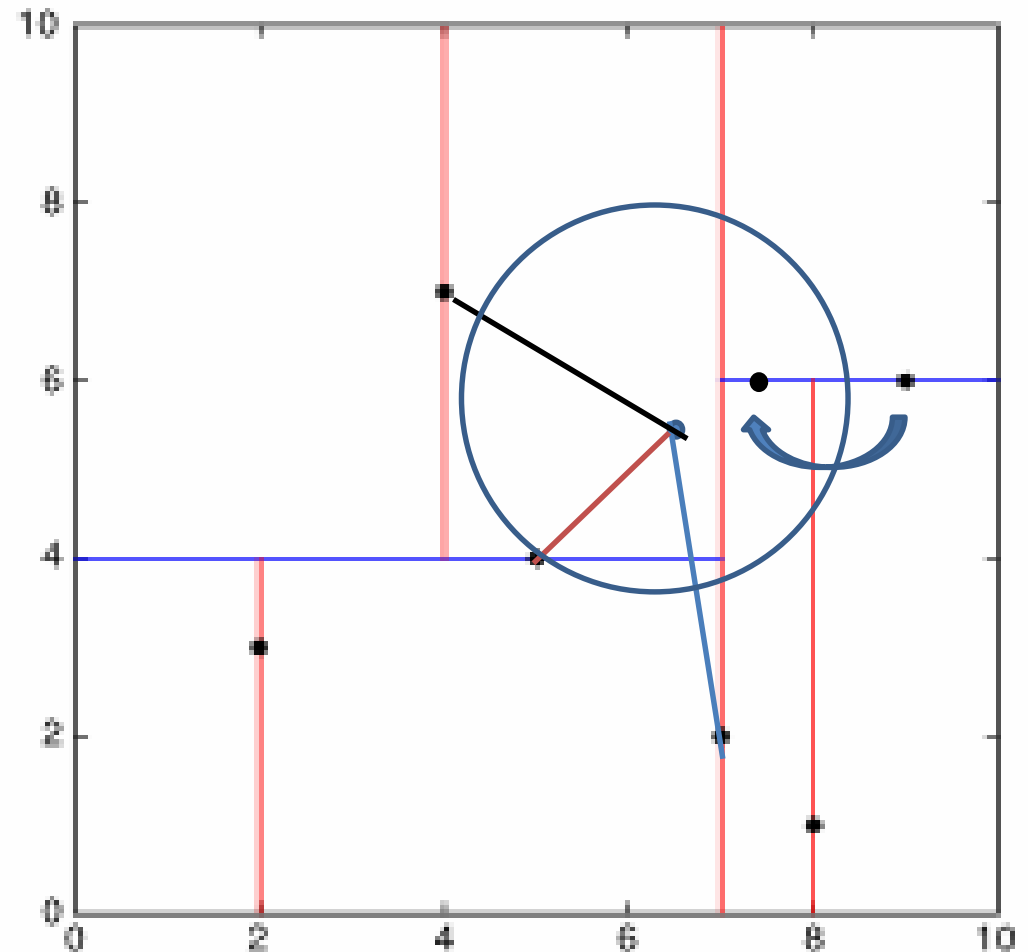
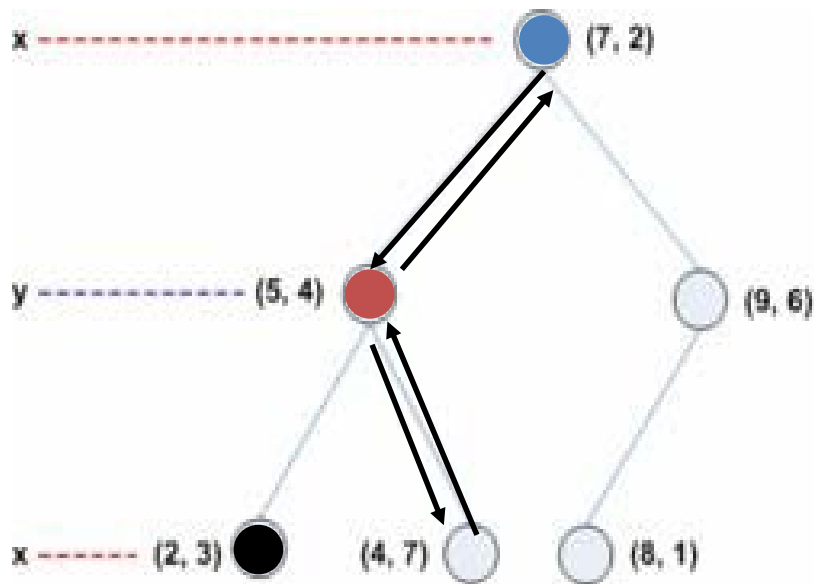
- Once the algorithm reaches a leaf node, it saves that node point as the "current best"
- The algorithm unwinds the recursion of the tree



Nearest Neighbour Search (NN- Search)

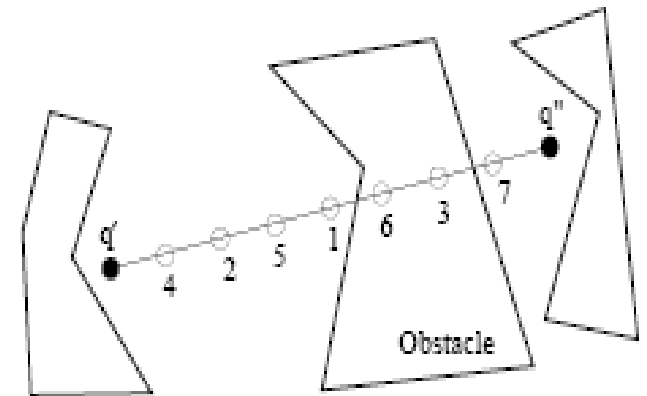
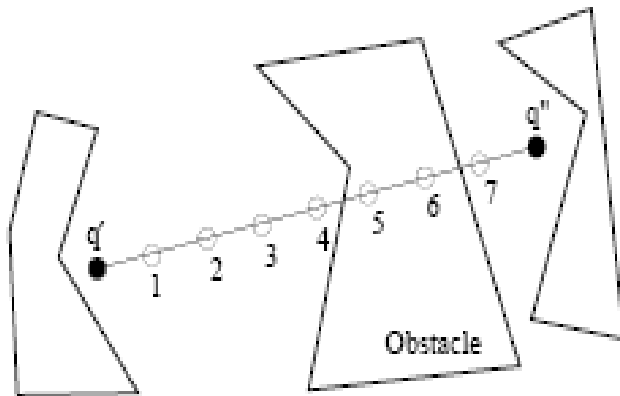
At each node:

- If the current node is closer than the current best, then it becomes the current best
- The algorithm checks whether there could be any points on the other side of the splitting plane that are closer to the search point than the current best.



Local Planner

- Properties
 - deterministic
 - symmetric
- Again, chose the quick and dirty one
- Don't necessarily store paths

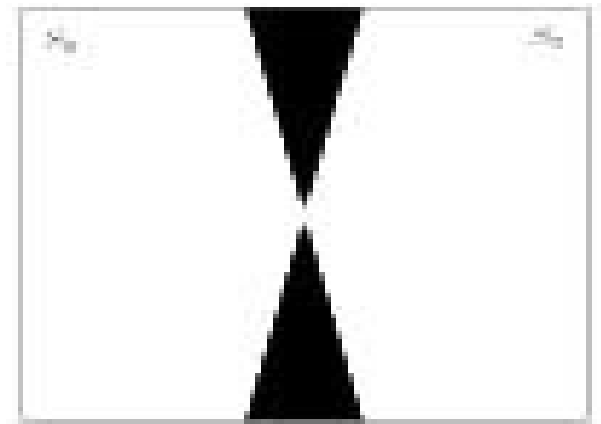


How to chose step size?

Expansion

- Sometimes G consists of several large and small components which do not effectively capture the connectivity of Q_{free}
- The graph can be disconnected at some narrow region
- Assign a positive weight $w(c)$ to each node c in V

$w(c)$ is a heuristic measure of the “difficulty” of the region around c . So $w(c)$ is large when c is considered to be in a difficult region. We normalize w so that all weights together add up to one. The higher the weight, the higher the chances the node will get selected for expansion.



How to choose $w(c)$?

- Can pick different heuristics
 - Count number of nodes of V lying within some predefined distance of c .
 - Check distance D from c to nearest connected component not containing c .
 - Use information collected by the local planner.
(If the planner often fails to connect a node to others, then this indicates the node is in a difficult area).

One Example:

At the end of the construction step, for each node c , compute the failure ratio $r_f(c)$ defined by:

$$r_f(c) = \frac{f(c)}{n(c) + 1}$$

where $n(c)$ is the total number of times the local planner tried to connect c to another node and $f(c)$ is the number of times it failed.

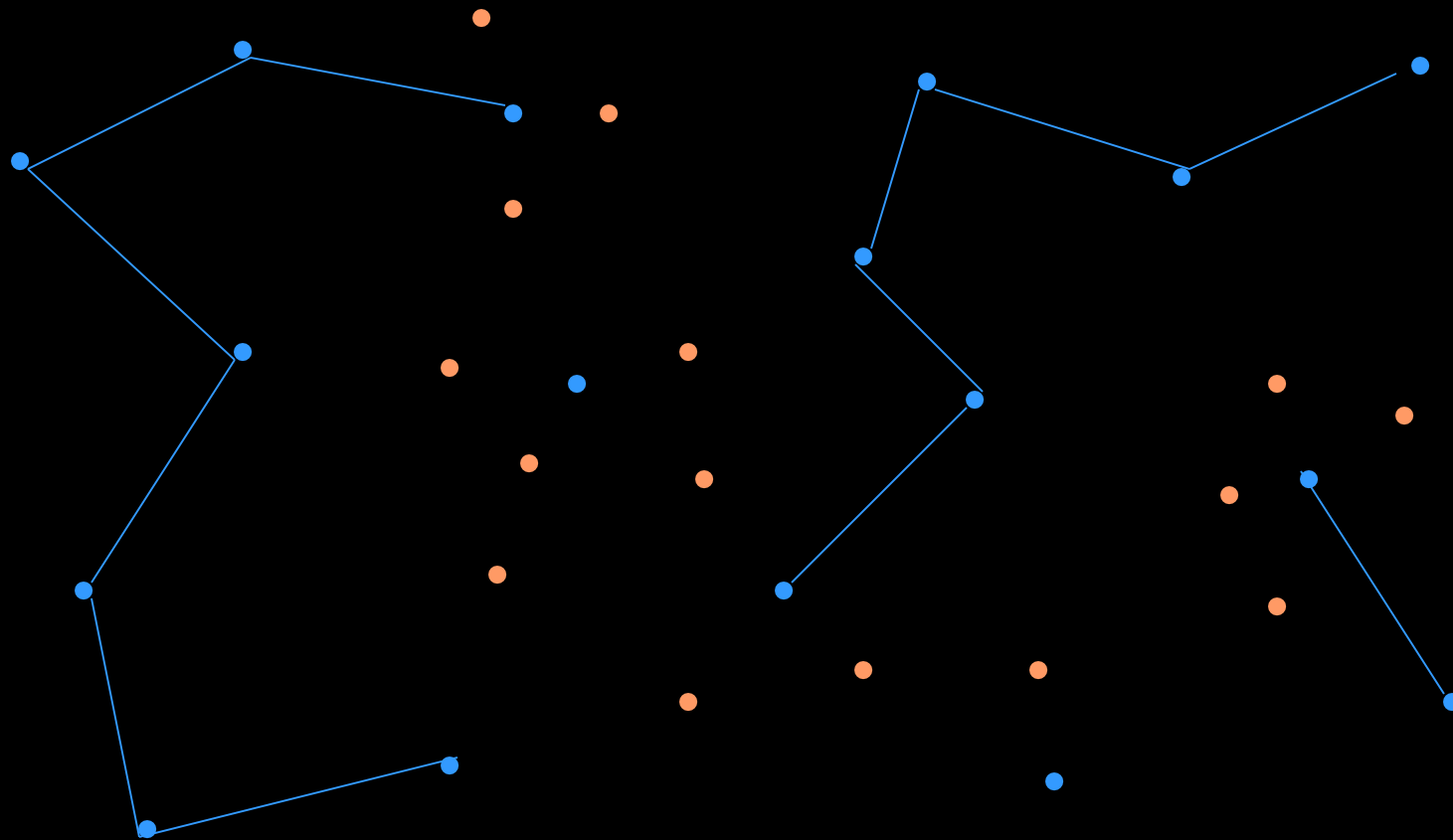
How to choose $w(c)$?

At the beginning of the expansion step, for every node c in V , compute $w(c)$ proportional to the failure ratio.

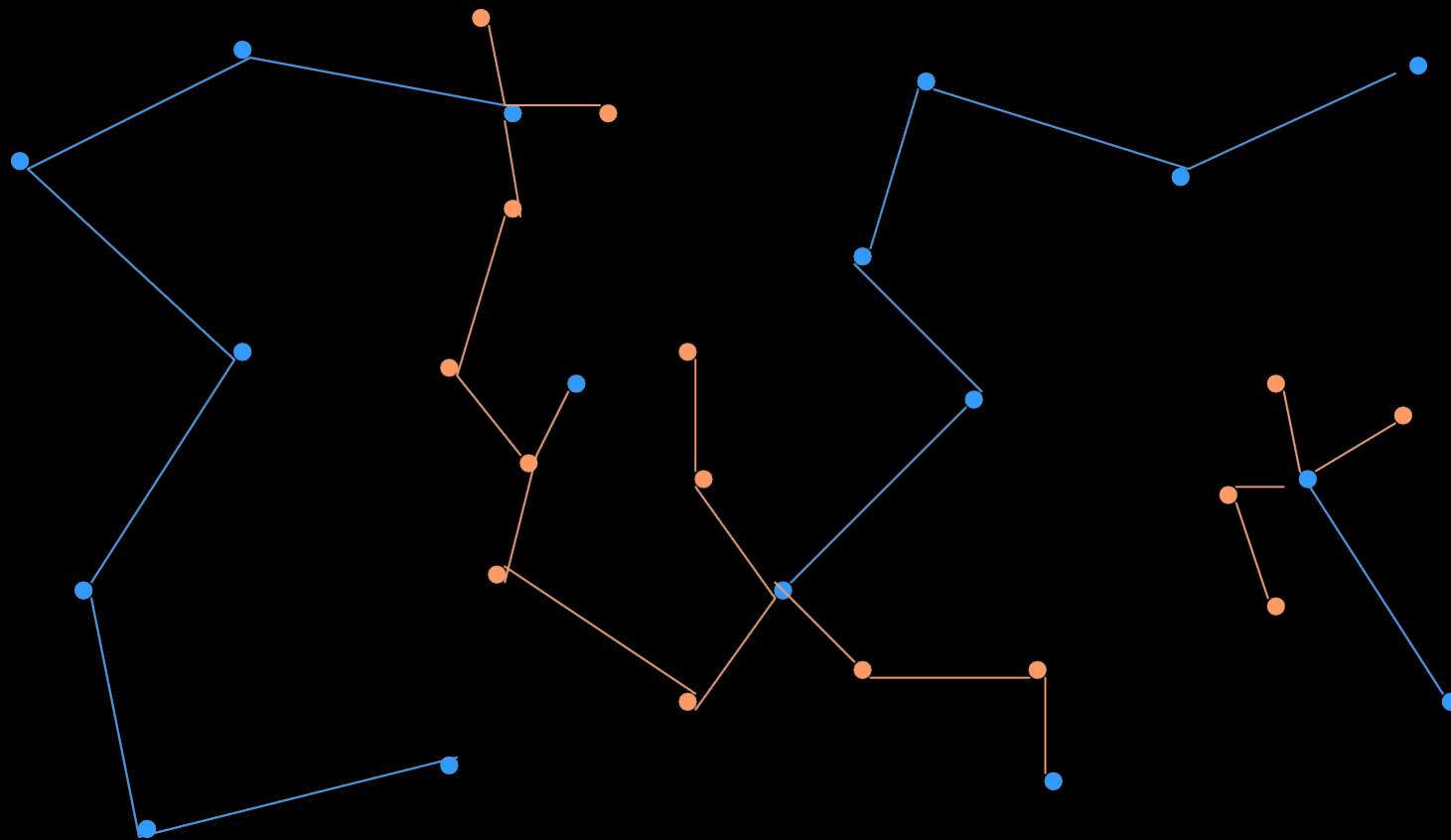
$$w(c) = \frac{r_f(c)}{\sum_{a \in V} r_f(a)}$$

- To expand a node c , we compute a short random-bounce walk starting from c . This means
 - Repeatedly pick at random a direction of motion in C-space and move in this direction until an obstacle is hit.
 - When a collision occurs, choose a new random direction.
 - The final configuration n and the edge (c,n) are inserted into R and the path is memorized.
 - Try to connect n to the other connected components like in the construction step.
 - Weights are only computed once at the beginning and not modified as nodes are added to G .

Expansion Step

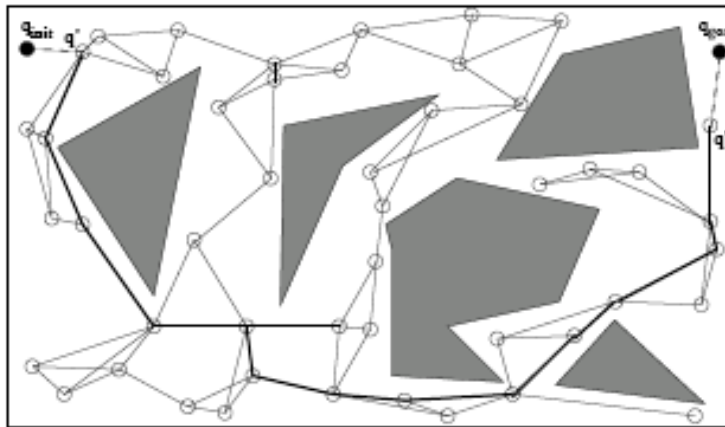


End of Expansion Step



The Query Phase

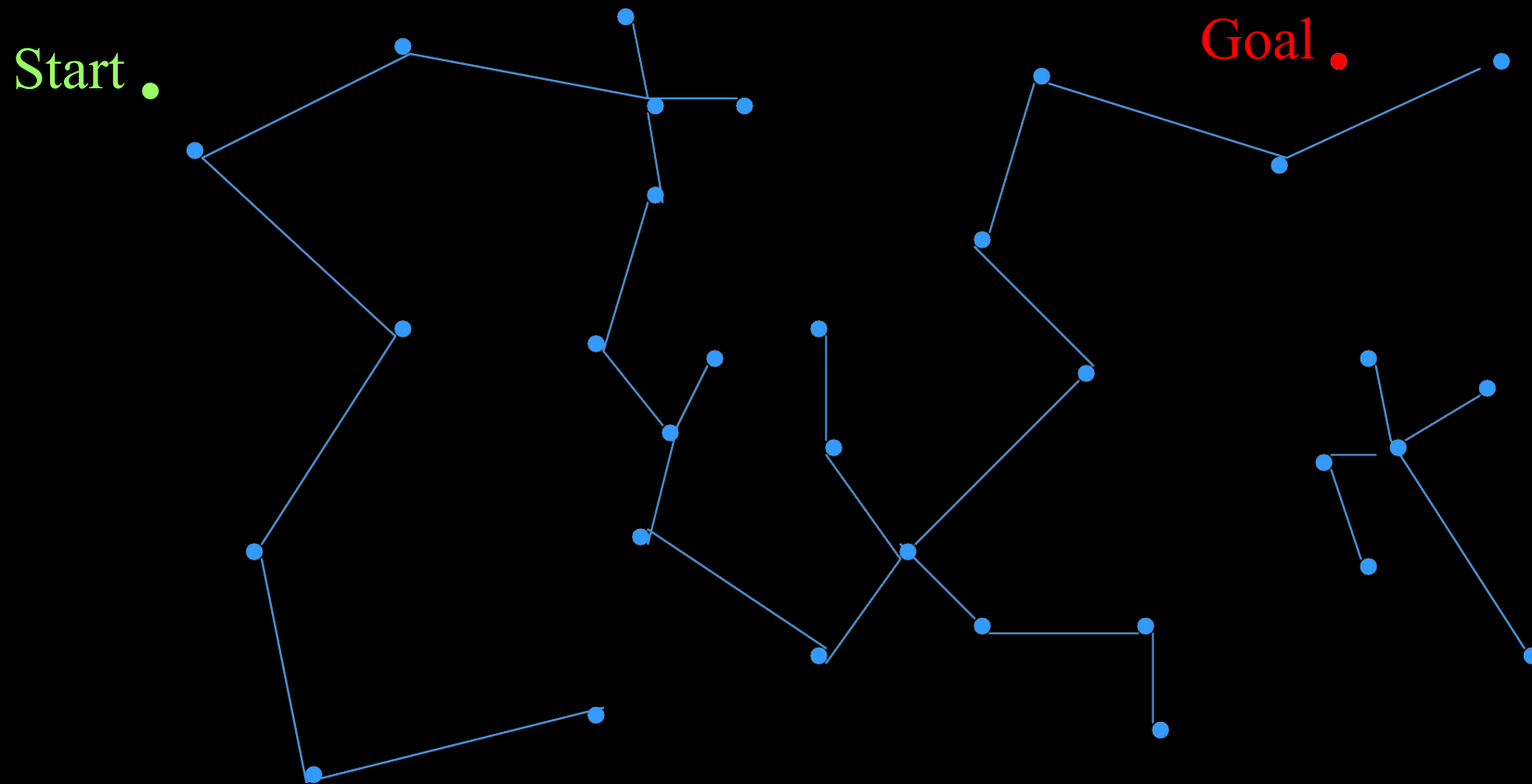
- Find a path from the start and goal configurations to two nodes of the roadmap
- Search the graph to find a sequence of edges connecting those nodes in the roadmap
- Concatenating the successive segments gives a feasible path for the robot



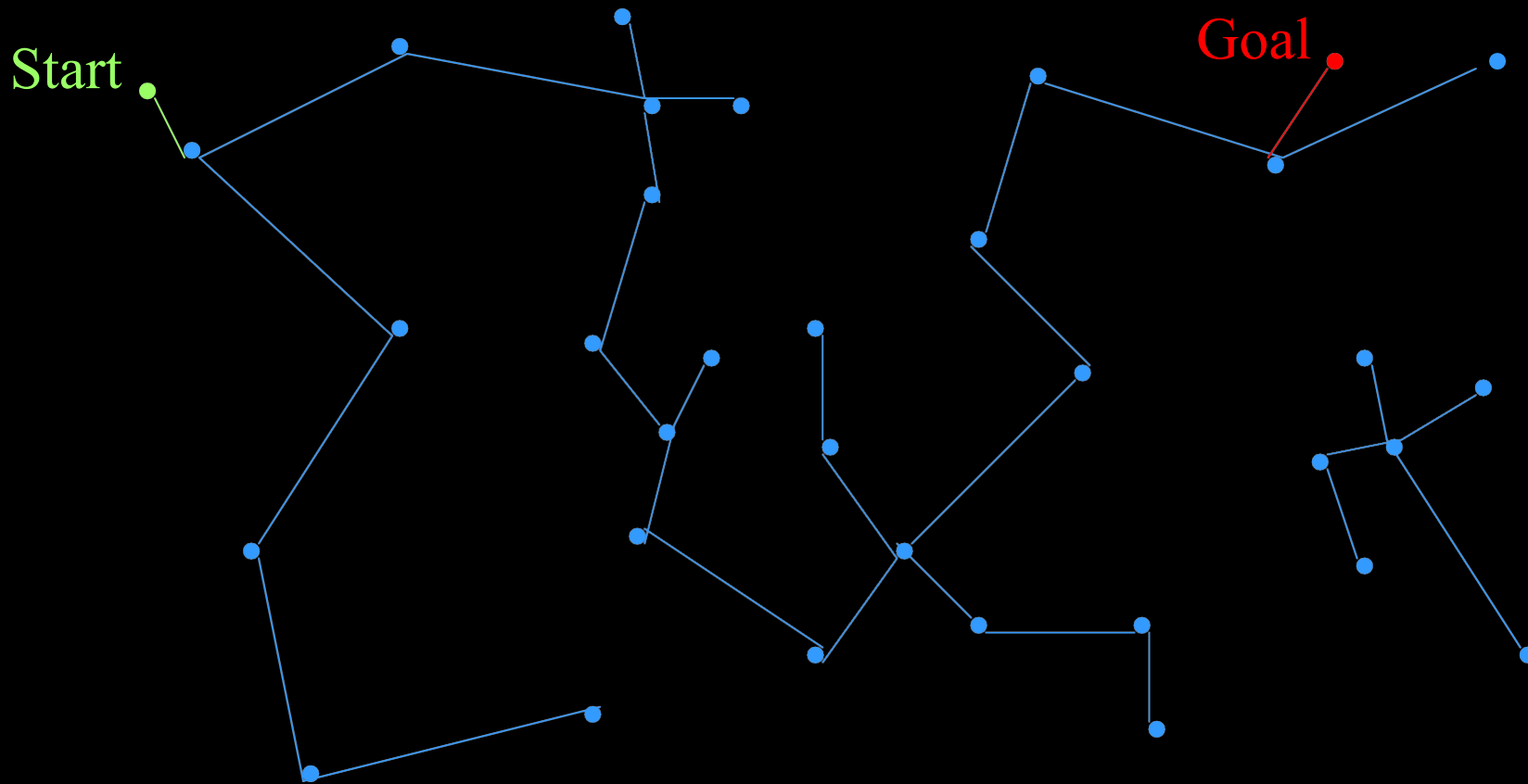
The Query Phase (contd.)

- Let start configuration be s
- Let goal configuration be g
- Try to connect s and g to Roadmap R at two nodes $\hat{s}$ and $\hat{g}$, with feasible paths P_s and P_g . If this fails, the query fails.
- Consider nodes in G in order of increasing distance from s (according to D) and try to connect s to each of them with the local planner, until one succeeds.
- Random-bounce walks
- Compute a path P in R connecting $\hat{s}$ to $\hat{g}$.
- Concatenate P_s , P and reversed P_g to get the final path

Select start and goal



Connect Start and Goal to Roadmap



Find the Path from Start to Goal

Path Shortening / Smoothing

- Don't even think of using a path generated by a sampling-based planner without smoothing it!!!
- “Shortcut” Smoothing

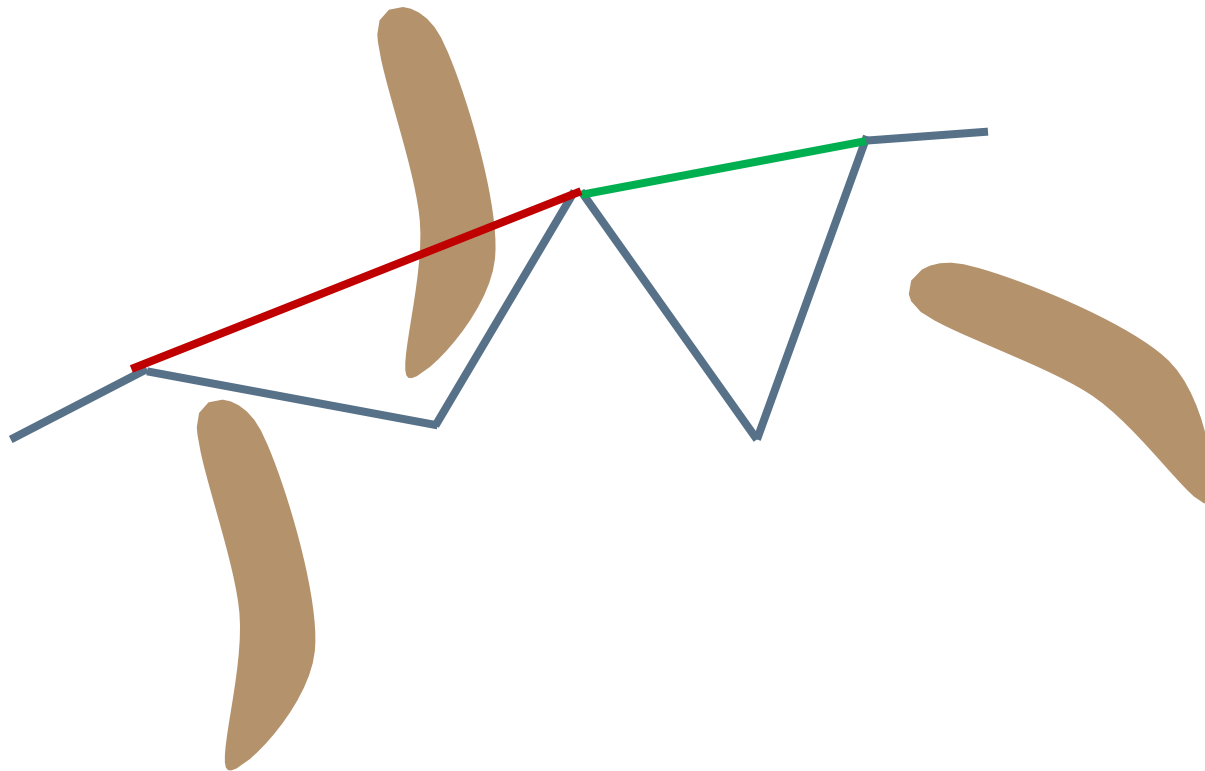
For $i = 0$ to MaxIterations

Pick two points, q_1 and q_2 , on the path randomly

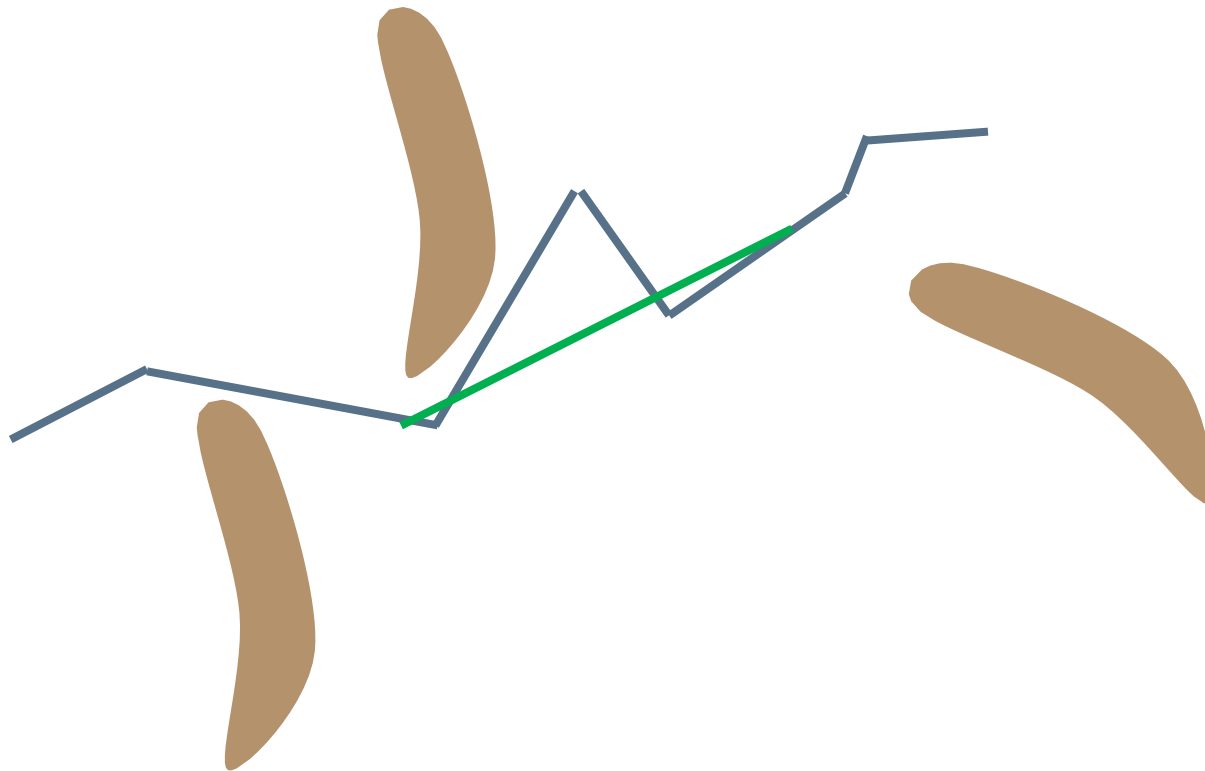
Attempt to connect (q_1, q_2) with a line segment

If successful, replace path between q_1 and q_2 with the line segment

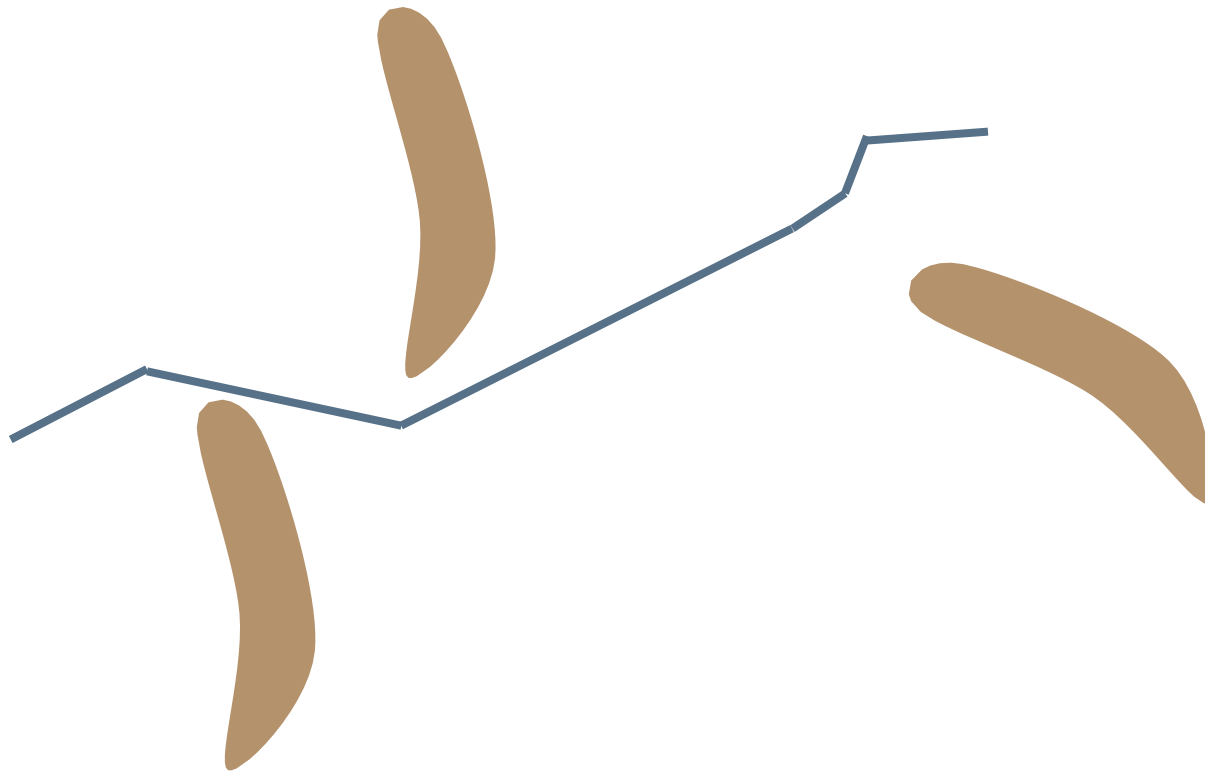
Shortcut Smoothing



Shortcut Smoothing



Shortcut Smoothing

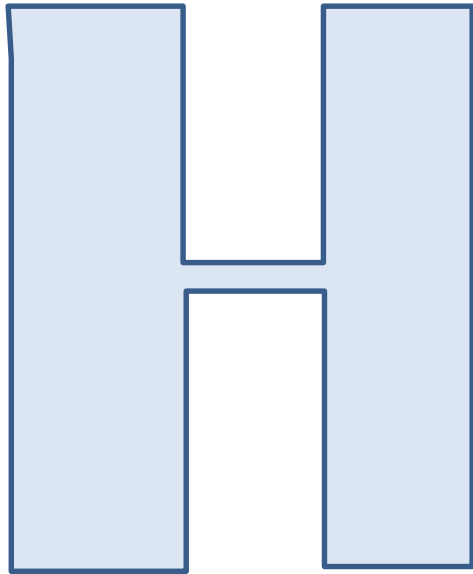


What if we fail?

- Maybe the roadmap was not adequate.
- Could spend more time in the Learning Phase
- Could do another Learning Phase and reuse R constructed in the first Learning Phase.
- In fact, Learning and Query Phases don't have to be executed sequentially.
- Questions:
- Where to sample new milestones? → Sampling strategy
- Which milestones to connect? → Connection strategy
- Goal:
- Minimize roadmap size to correctly answer motion-planning queries

Sampling Strategies

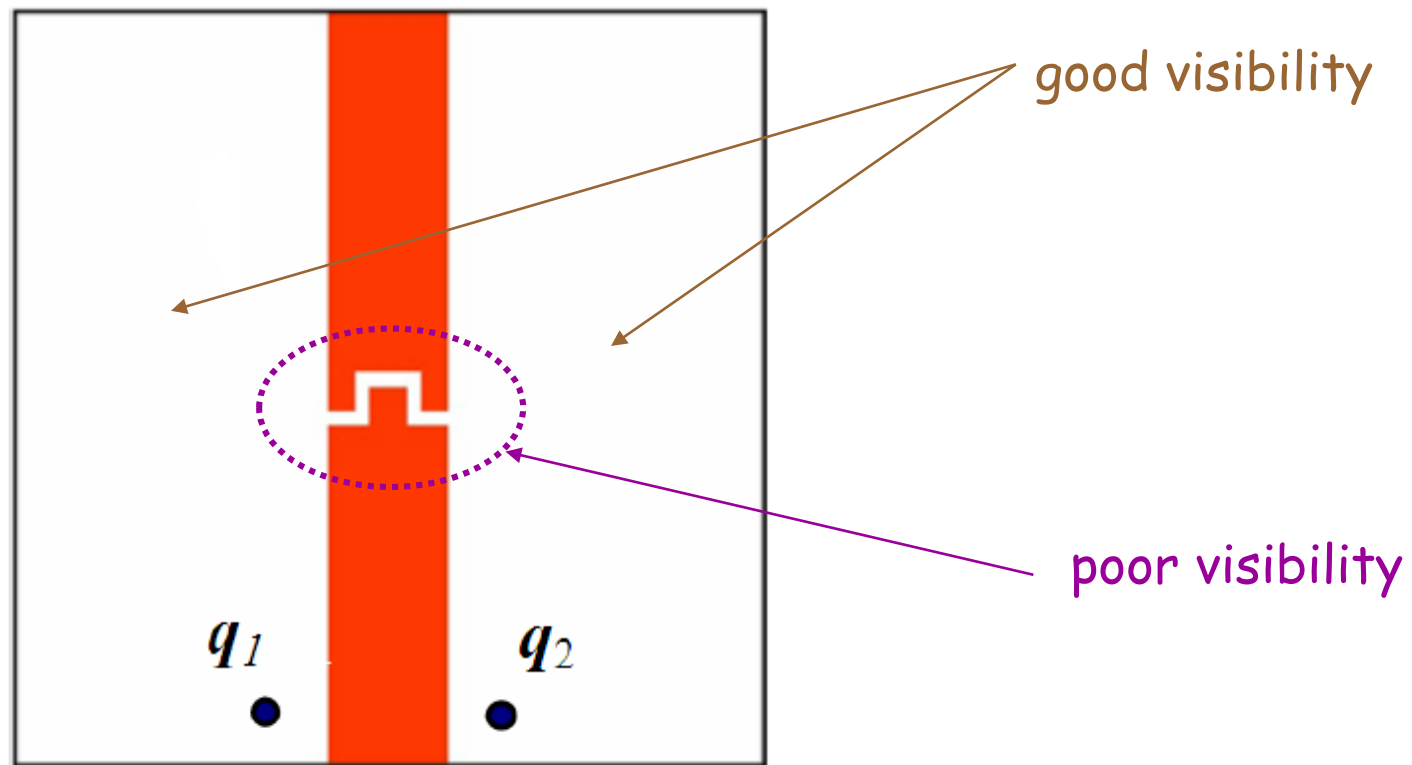
- C-Space is infinite
- Sampling the C-Space gives a finite set of samples
- Important, to choose „good“ sampling strategy, since the performance is dependent on the way the C-space is sampled
- Uniform is good because it is easy to implement but is bad because of difficulty to sample in narrow passages of free space



- Learning Phase
 - Construction Step
 - Uniform sampling
 - Expansion Step
 - Uniform around neighbour (local repair)
- Query Phase

Rationale for Non-Uniform Sampling Strategy

- Visibility is not uniformly favorable in free space



- Regions with poorer visibility should be more densely sampled

How to identify poor visibility regions?

What is the source of information? Robot and workspace geometry

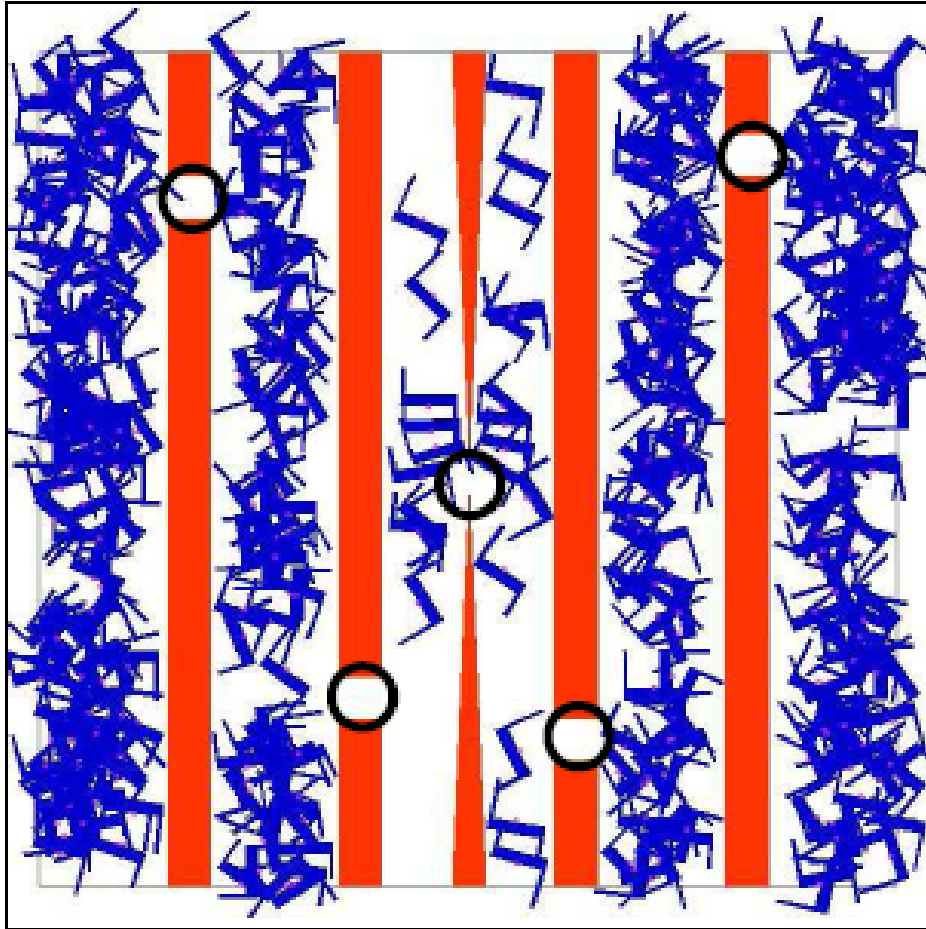
How to exploit it?

- Workspace-guided strategies
 - Configuration guided strategies (OBPRM)
 - Filtering strategies
 - Adaptive strategies
 - Deformation strategies
-
- **Workspace-guided strategies**
Identify narrow passages in the workspace and map them into the configuration space
 - **Filtering strategies**
Sample many configurations, find interesting patterns, and retain only promising configurations
 - **Adaptive strategies**
Adjust the sampling distribution on the fly
 - **Deformation strategies**
Deform the free space, e.g., to widen narrow passages

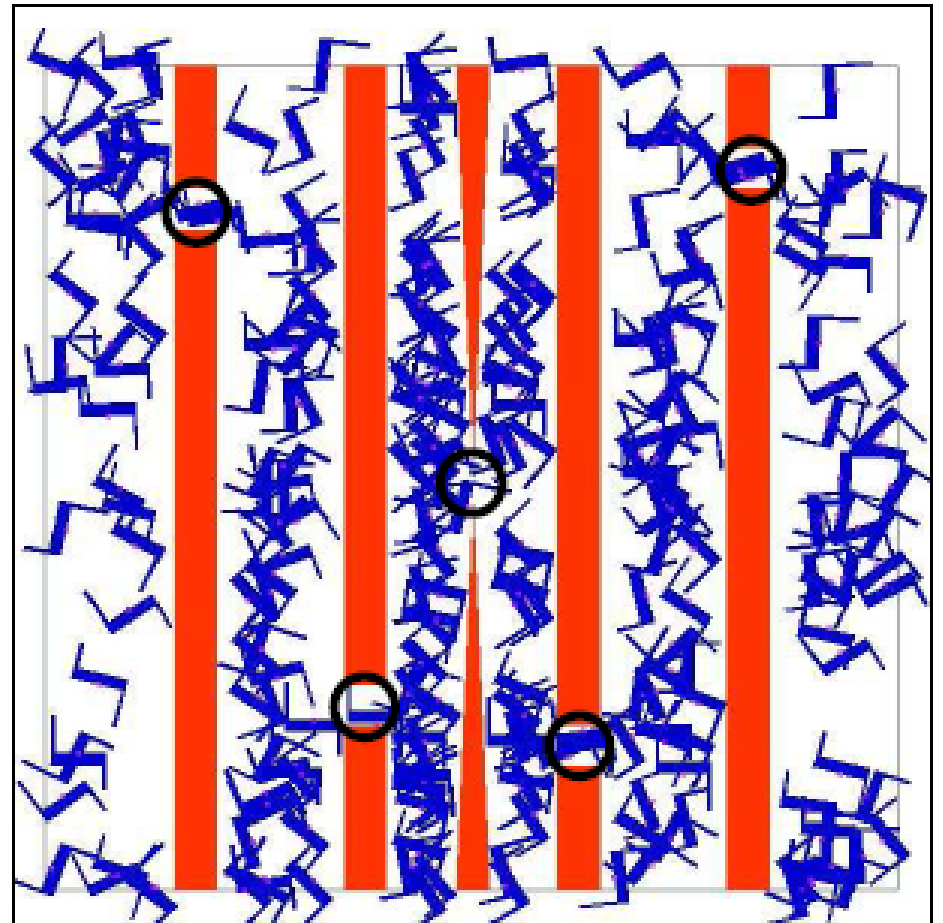
Workspace-Guided Strategies

- **Rationale:**
 - Most narrow passages in configuration space are caused by narrow passages in the workspace
 - **Method:**
 - Detect narrow passages in the workspace (e.g., cell decomposition, medial-axis transform)
 - Sample robot configurations that place selected robot points in workspace's narrow passages
- H. Kurniawati and D. Hsu. Workspace importance sampling for probabilistic roadmap planning. In Proc. IEEE/RSJ Int. Conf. on Intelligent Robots & Systems, pp. 1618–1623, 2004.
- J.P. van den Berg and M. H. Overmars. Using Workspace Information as a Guide to Non-Uniform Sampling in Probabilistic Roadmap Planners. IJRR, 24(12):1055-1071, Dec. 2005.

Workspace-Guided Strategies

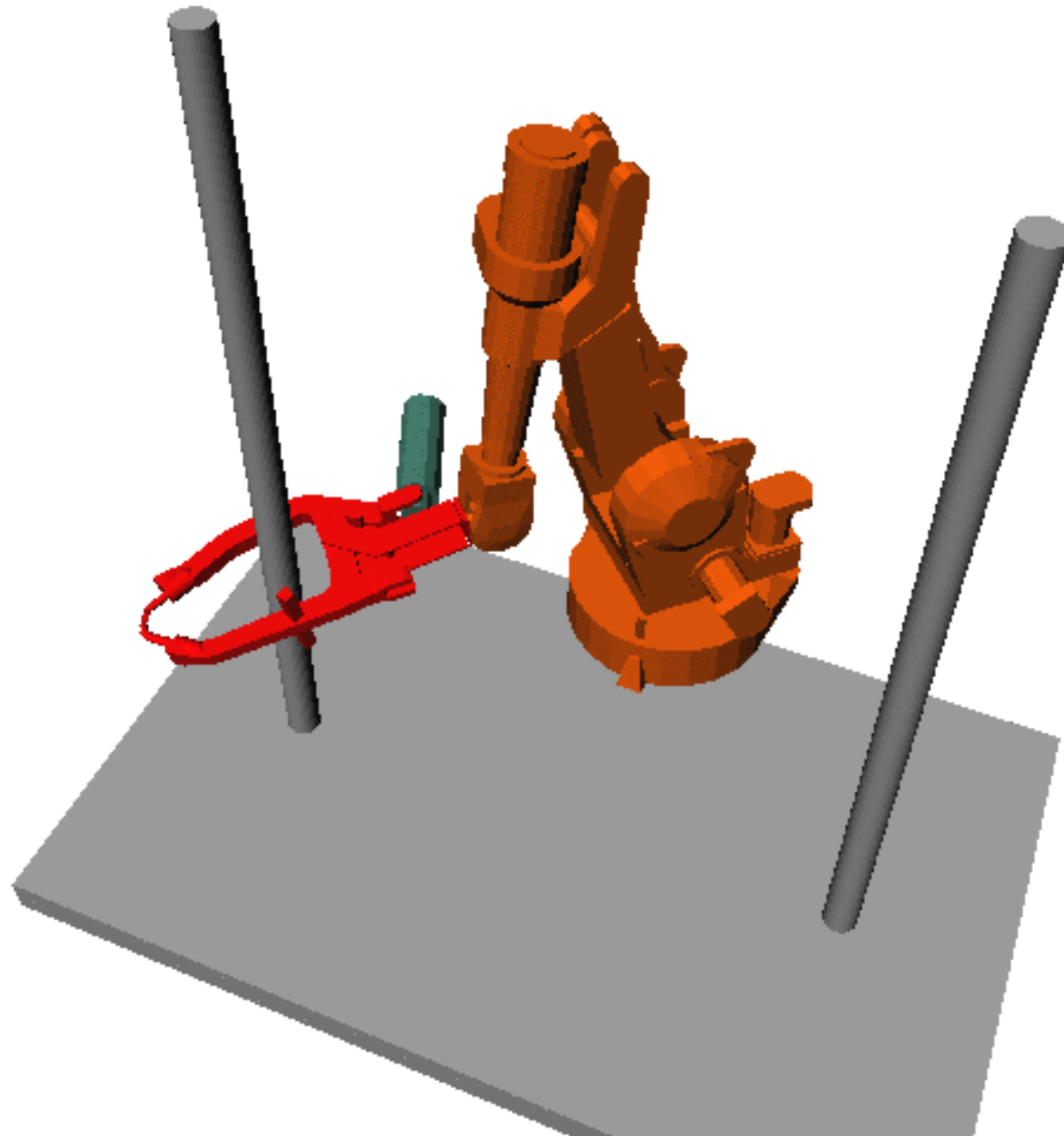


Uniform sampling



Workspace-guided sampling

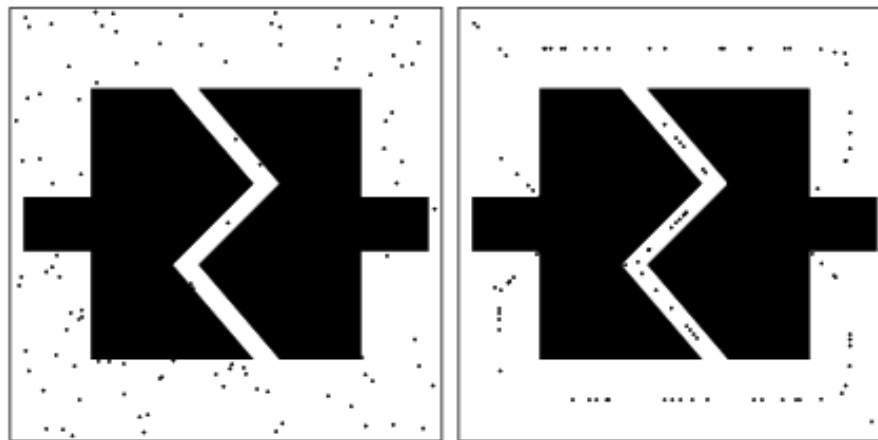
Example



Sampling Strategies – Medial-Axis Bias

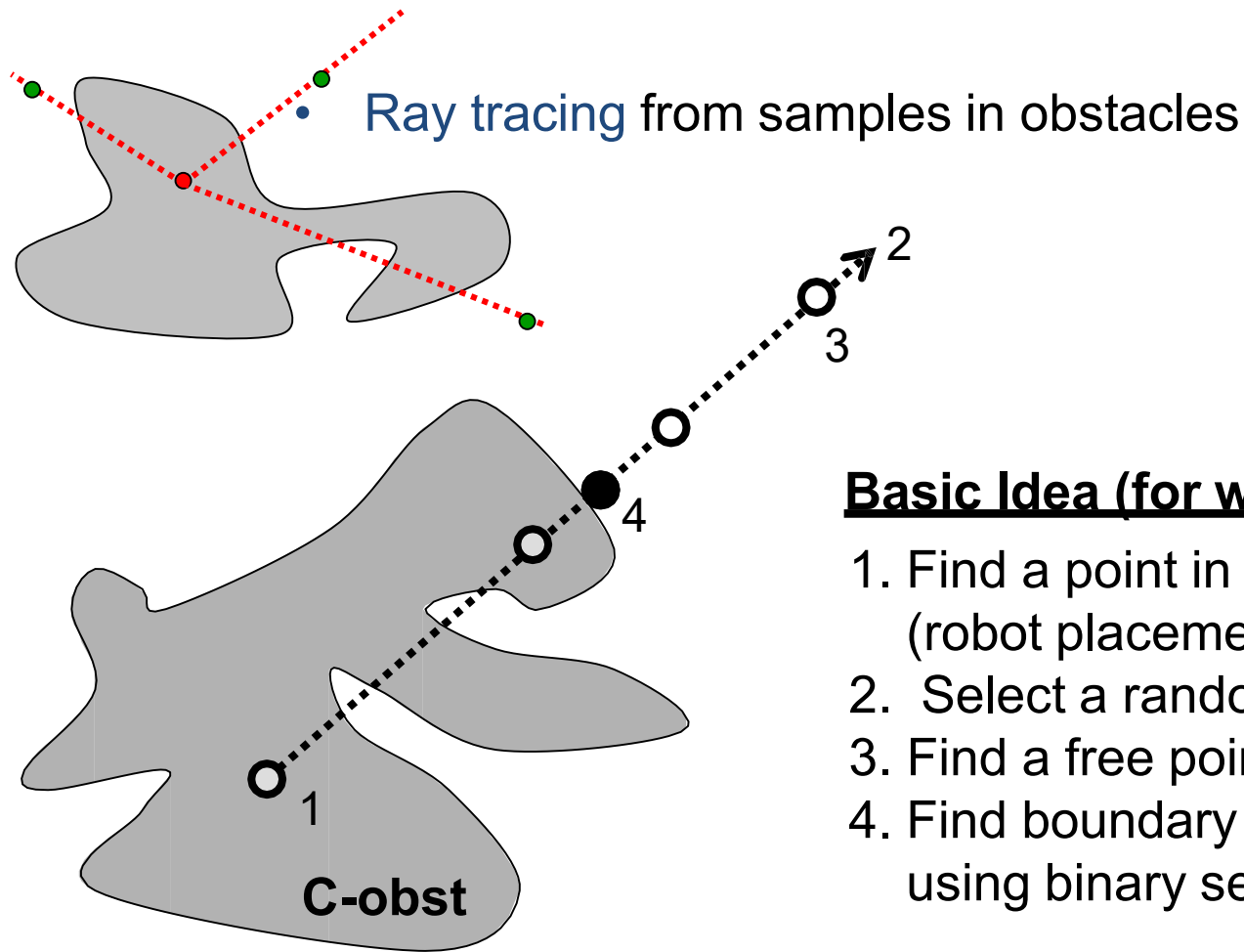
Medial-Axis Bias

- Force the samples to be as far from the boundary as possible
- The centers of all maximal balls trace out a one-dimensional set of points referred to as the medial axis.
- Sampling on the medial axis is difficult, especially because of the representation of C_{free} .
- Distance information from collision checking can be used to start with a sample, and iteratively perturb it to increase its distance from ∂C_{free}
- Sampling on the medial axis of $W \setminus O$
- the medial axis in $W \setminus O$ is easier to compute
- can be used to heuristically guide the placement of good roadmap vertices in C_{free} .



[Amato, Kavraki]

OBPRM: Finding Points on C-obstacles



Basic Idea (for workspace obstacle S)

1. Find a point in S's C-obstacle
(robot placement colliding with S)
2. Select a random direction in C-space
3. Find a free point in that direction
4. Find boundary point between them
using binary search (collision checks)

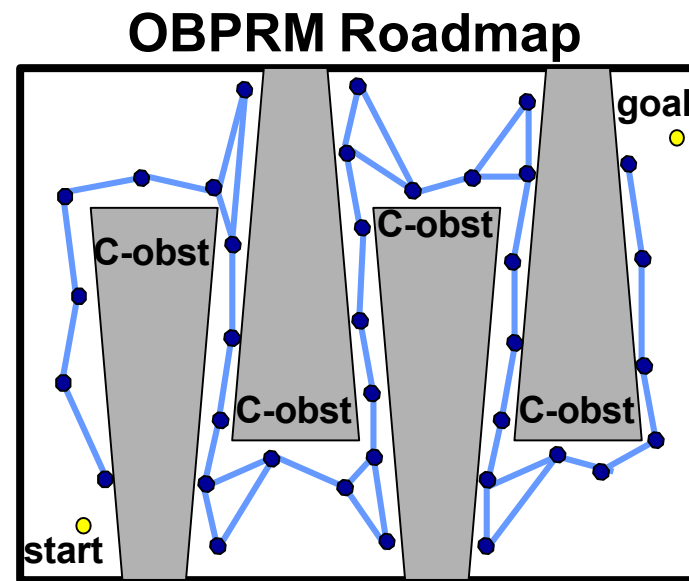
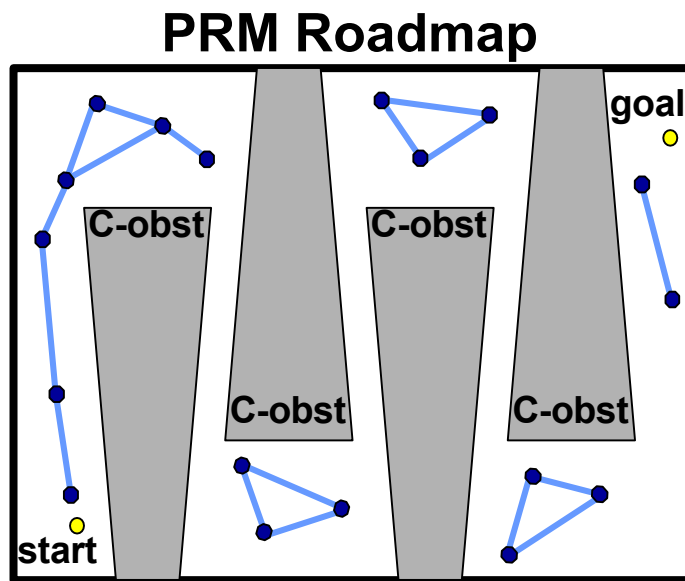
Note: we can use more sophisticated heuristics to try to cover C-obstacle

OBPRM: An Obstacle-Based PRM

[IEEE ICRA'96, IEEE ICRA'98, WAFR'98]

To Navigate Narrow Passages we must sample in them

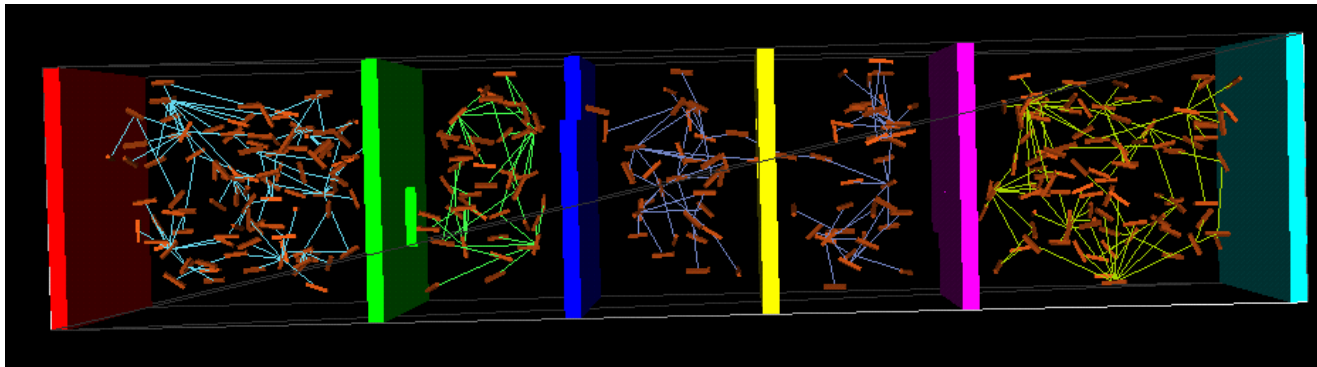
- most PRM nodes are where planning is easy (not needed)



Idea: Can we sample nodes near C-obstacle surfaces?

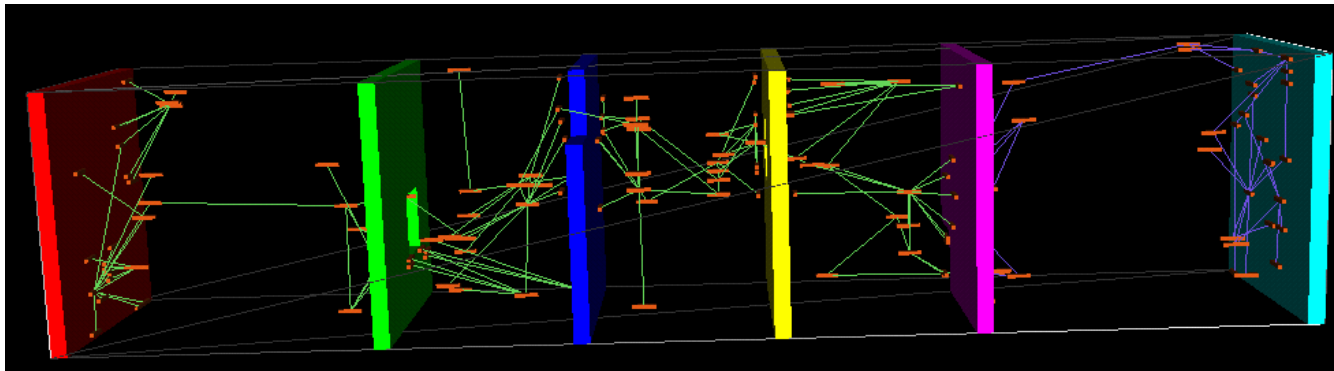
- we cannot explicitly construct the C-obstacles...
- we do have models of the (workspace) obstacles...

PRM vs OBPRM Roadmaps



PRM

- 328 nodes
- 4 major CCs



OBPRM

- 161 nodes
- 2 major CCs

How to identify poor visibility regions?

What is the source of information? Robot and workspace geometry

How to exploit it?

- Workspace-guided strategies
 - Configuration guided strategies (OBPRM)
 - **Filtering strategies**
 - Adaptive strategies
 - Deformation strategies
-
- **Workspace-guided strategies**
Identify narrow passages in the workspace and map them into the configuration space
 - **Filtering strategies**
Sample many configurations, find interesting patterns, and retain only promising configurations
 - **Adaptive strategies**
Adjust the sampling distribution on the fly
 - **Deformation strategies**
Deform the free space, e.g., to widen narrow passages

Filtering Strategies

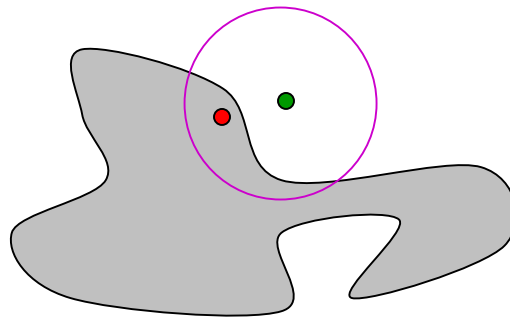
- **Main Idea:**
 - Sample several configurations in the same region of configuration space
 - If a “pattern” is detected, then retain one of the configurations as a roadmap node
 - More sampling work, but better distribution of nodes
 - Less time is wasted in connecting “non-interesting” milestones
- **Methods:**
 - Gaussian sampling
 - Bridge Test
 - Hybrid

- V. Boor, M. H. Overmars, and A. F. van der Stappen.
The Gaussian sampling strategy for probabilistic roadmap planners. In Proc. 1999 IEEE Int. Conf. Robotics and Automation, 1999, pp. 1018–1023.

- Z. Sun, D. Hsu, T. Jiang, H. Kurniawati, and J. Reif .
Narrow passage sampling for probabilistic roadmap planners. IEEE Trans. on Robotics, 21(6):1105–1115, 2005.

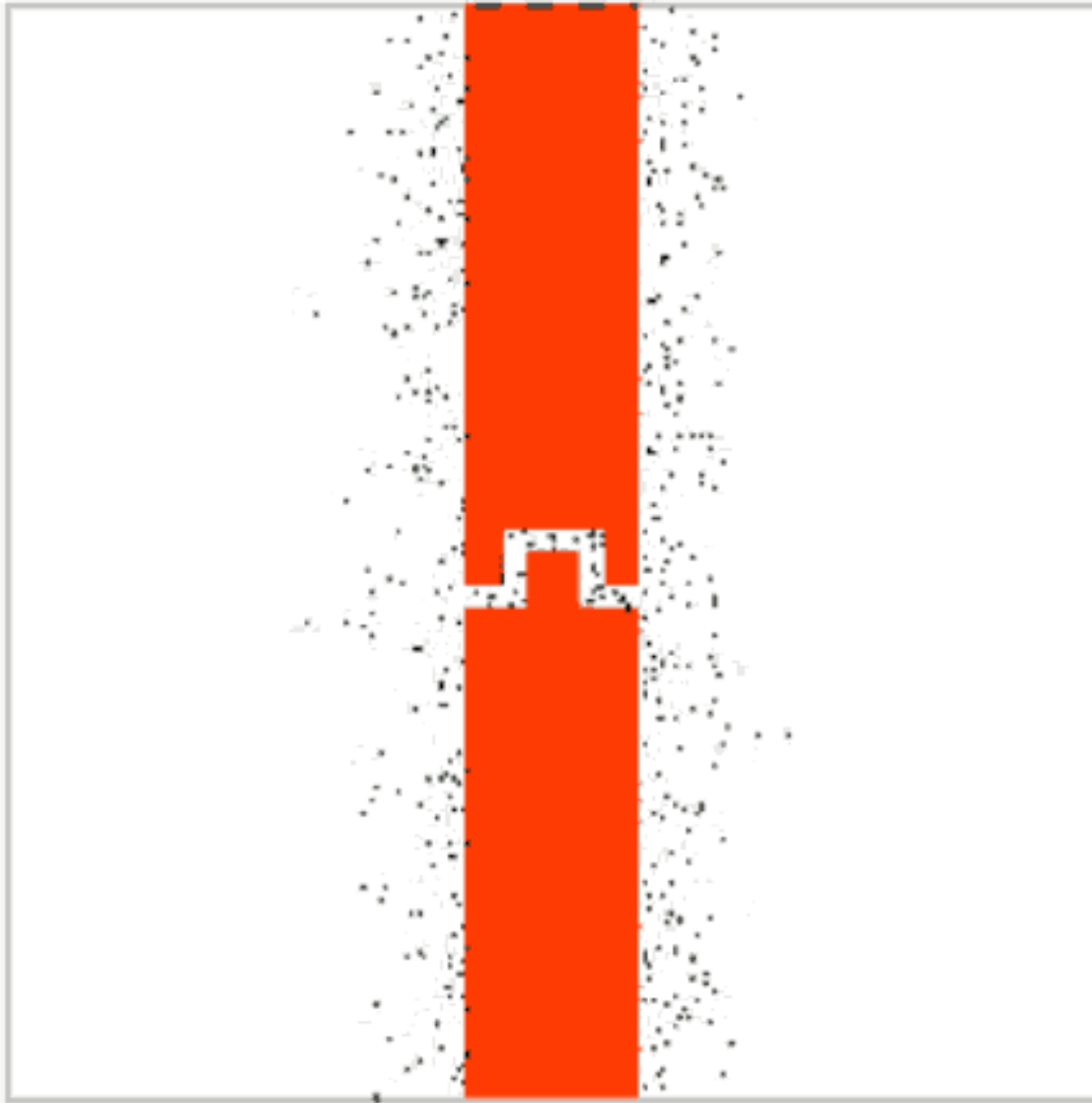
Sampling Strategies Near obstacles

- Gaussian sampling
- Find a q_1
- Pick a q_2 from a Gaussian distribution centered at q_1
- If **both** are in collision or collision-free, discard them, if **one** free, keep it



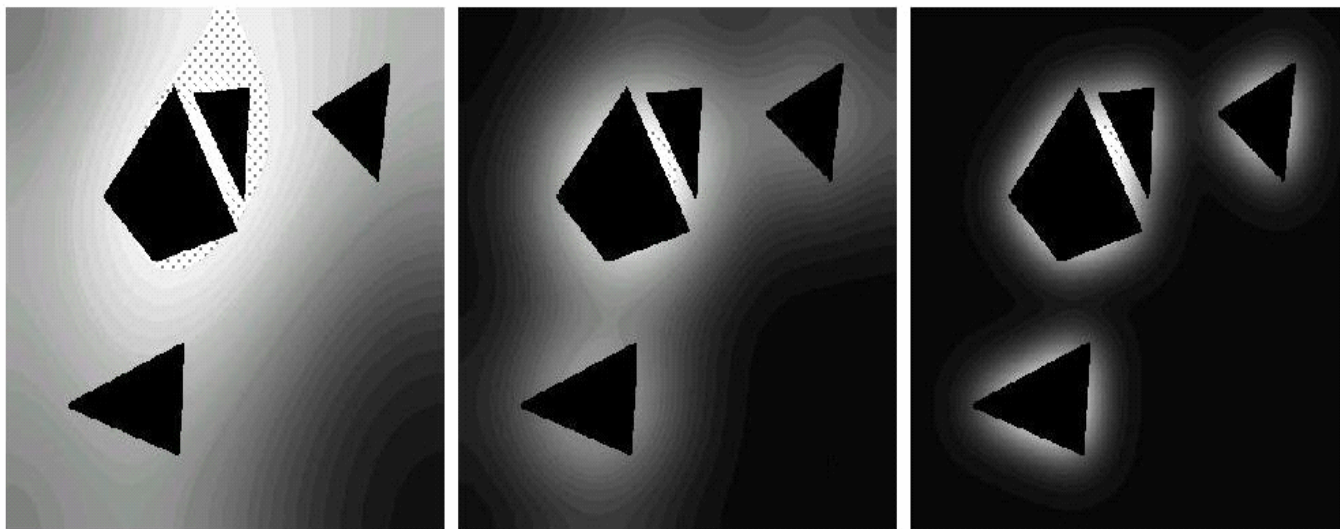
[Boor, Overmars, van der Stappen, 99]

Example of Node Distribution

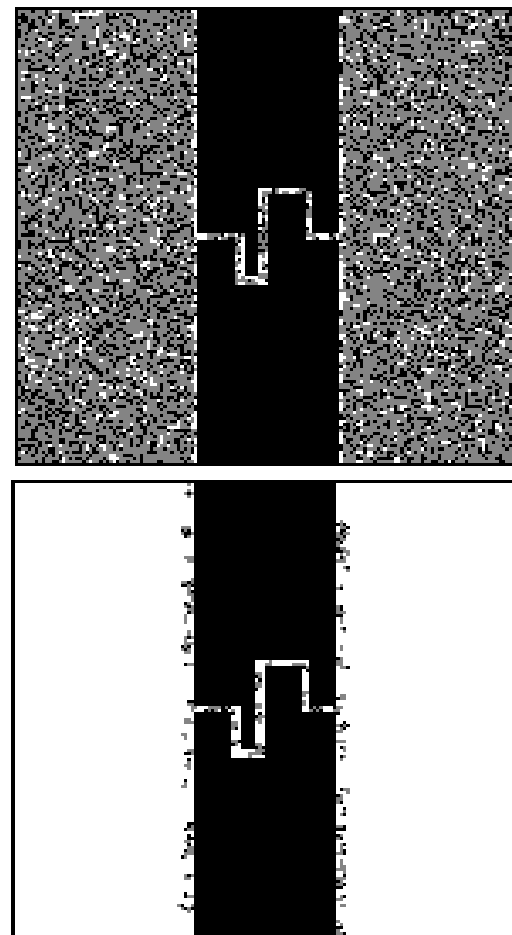


Sampling Strategies Near obstacles

- Gaussian sampling Examples

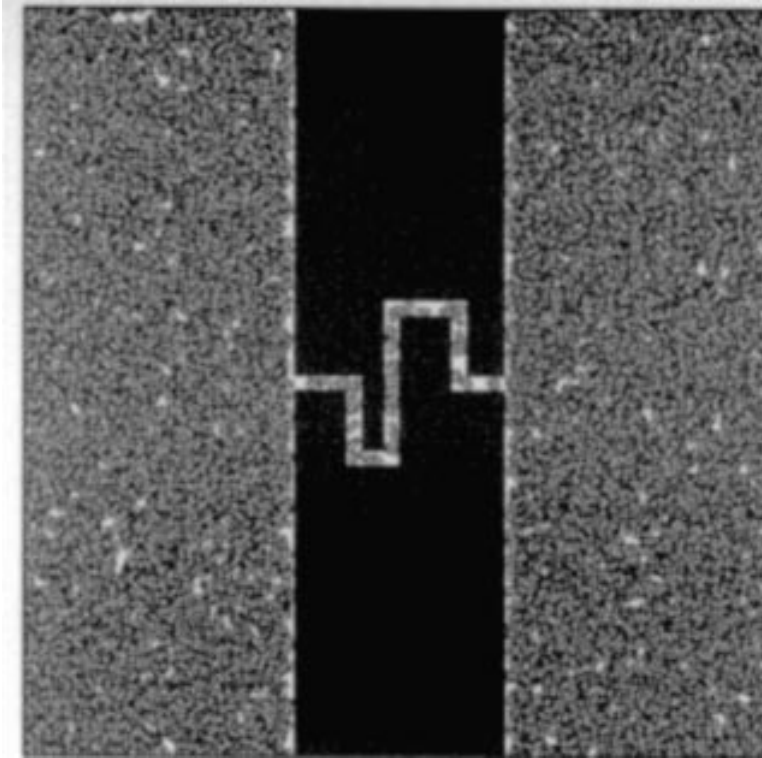


Sampling distribution for varying Gaussian width
(width decreasing from left to right)

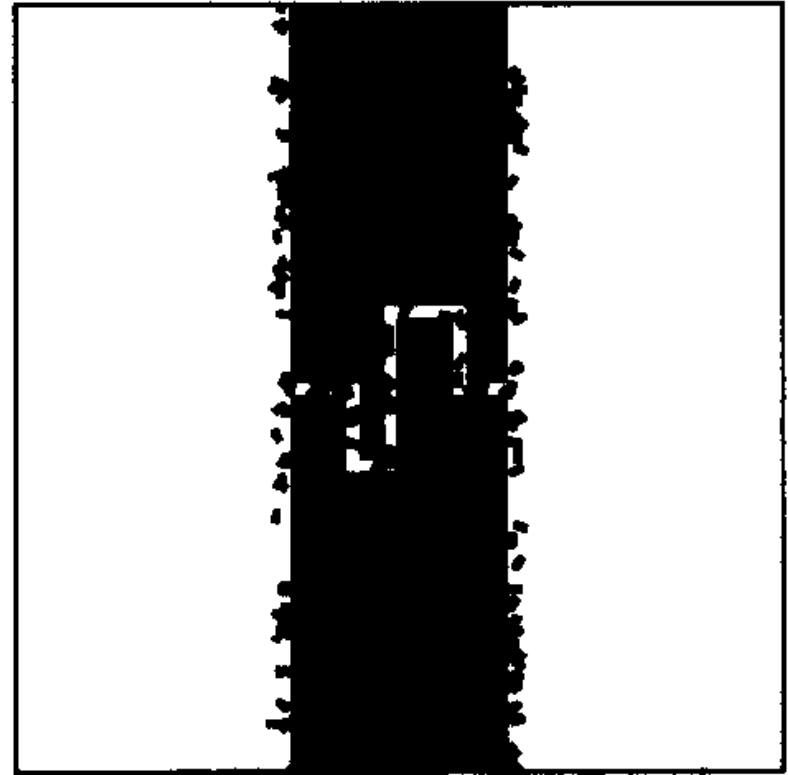


Uniform sampling vs.
Gaussian sampling

Uniform vs. Gaussian Sampling



Milestones (13,000) created by uniform sampling before the narrow passage was adequately sampled

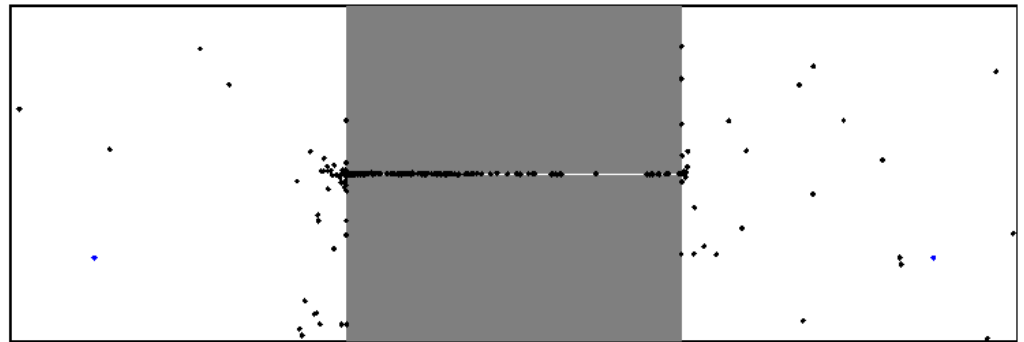
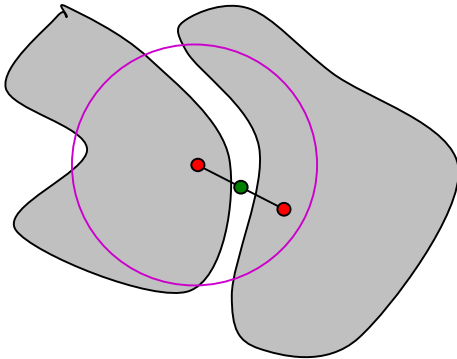


Milestones (150) created by Gaussian sampling

The gain is not in sampling fewer milestones, but in connecting fewer pairs of milestones

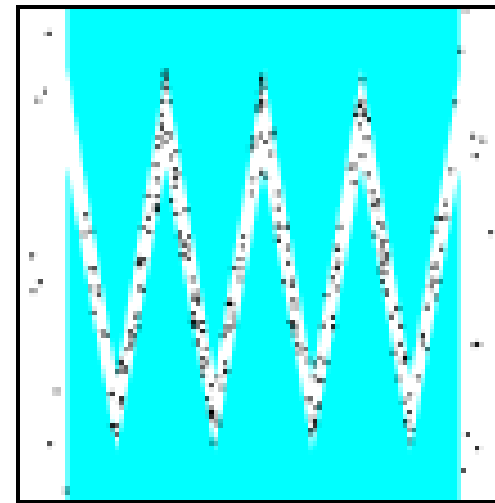
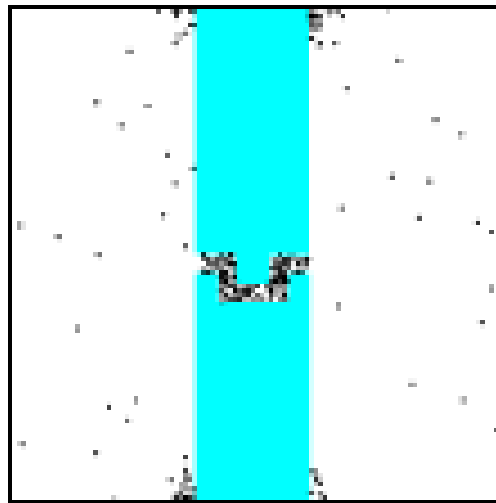
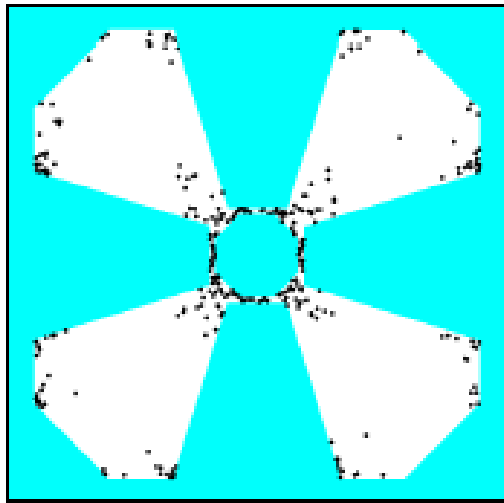
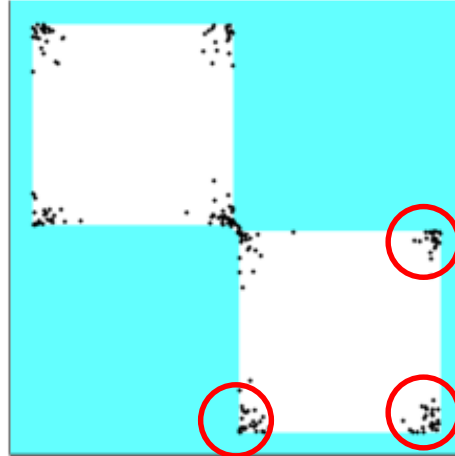
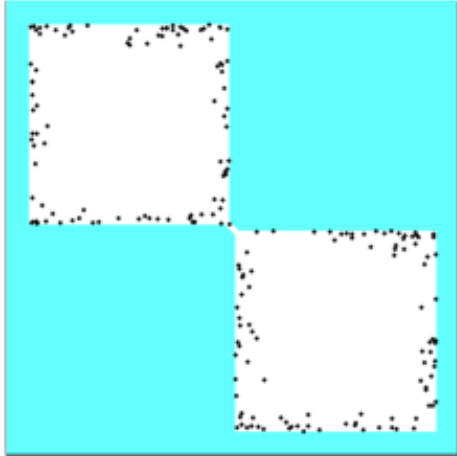
Sampling Strategies – Bridge Test

- Bridge Test:
 - Sample a q_1 that is in collision
 - Sample a q_2 in neighborhood of q_1 using some probability distribution (e.g. Gaussian)
 - If q_2 in collision, get the midpoint of (q_1, q_2)
 - Check if midpoint is in collision, if not, add it as a node



Sampling Strategies: Bridge Test

- Bridge Test Examples:

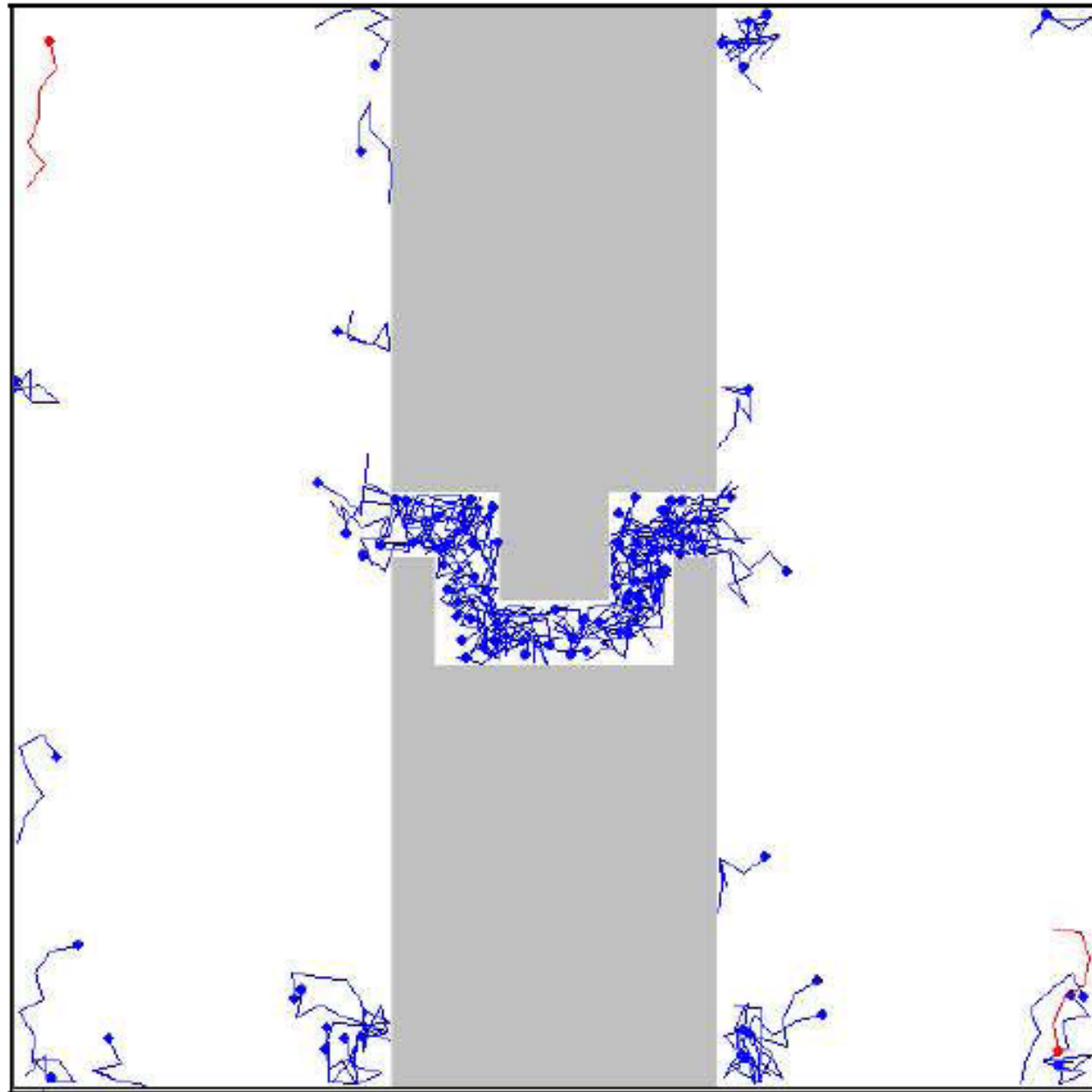


(a)

(b)

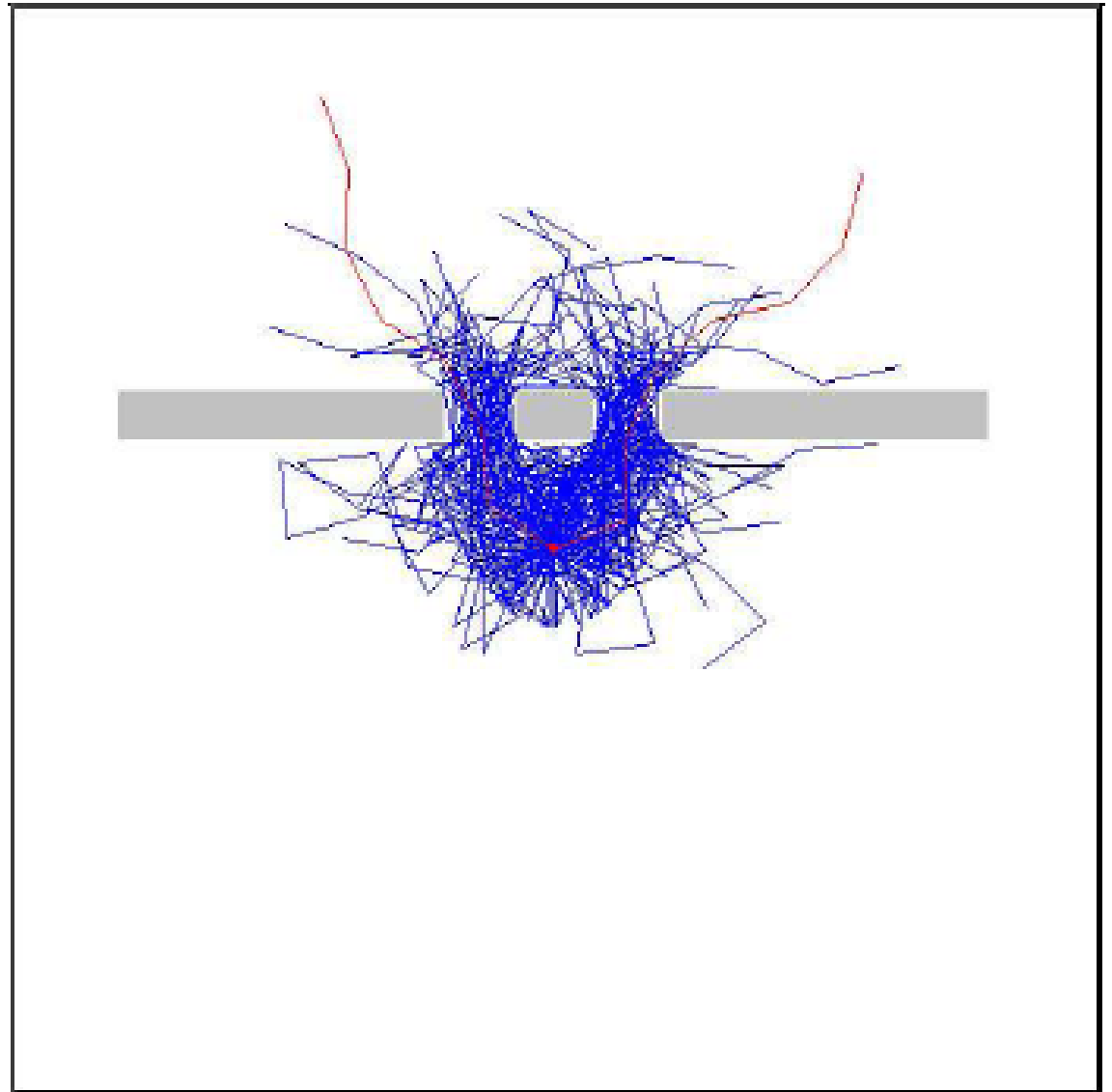
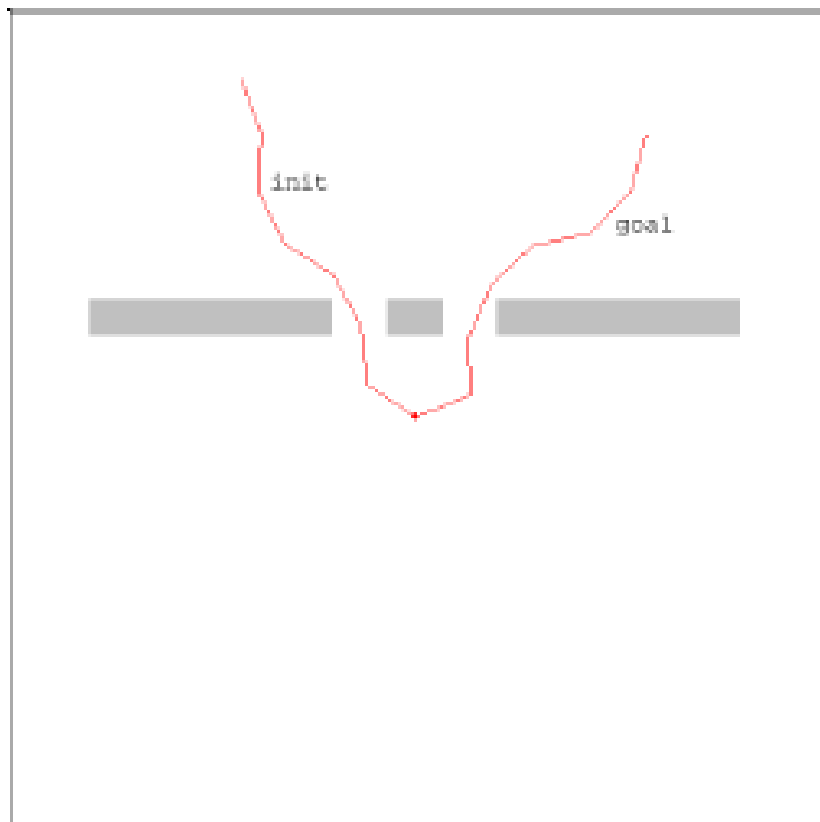
Example of Distribution

8-joint robot
with mobile base



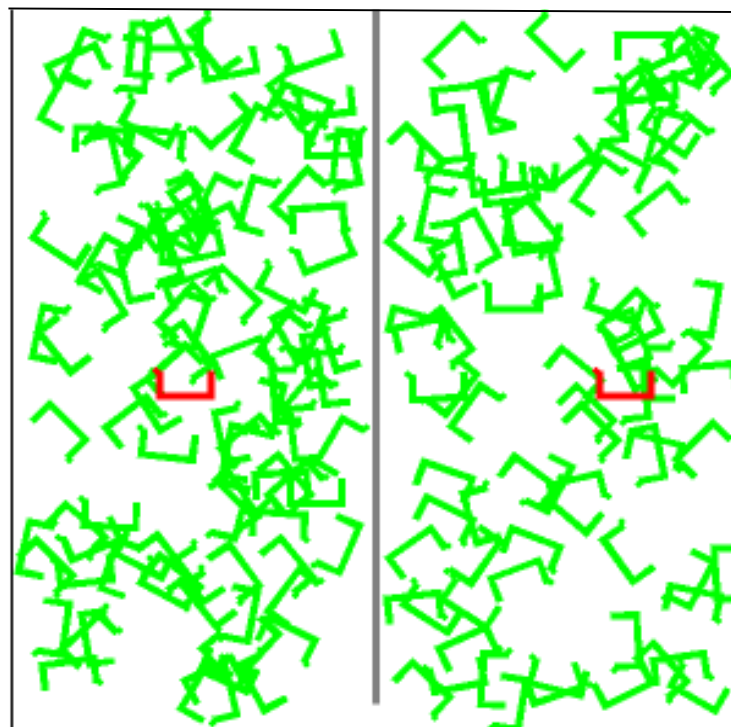
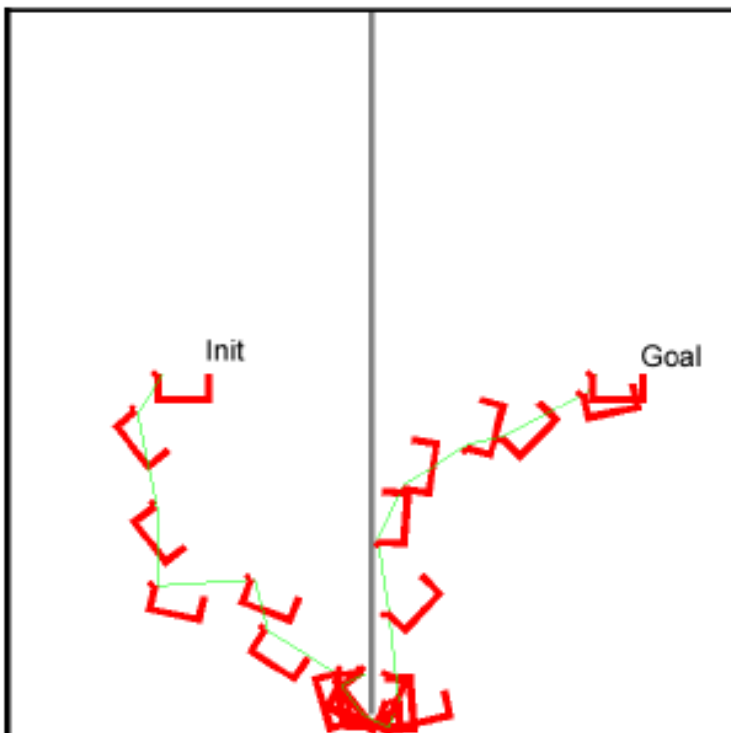
Example of Distribution

7-joint robot
with fixed base

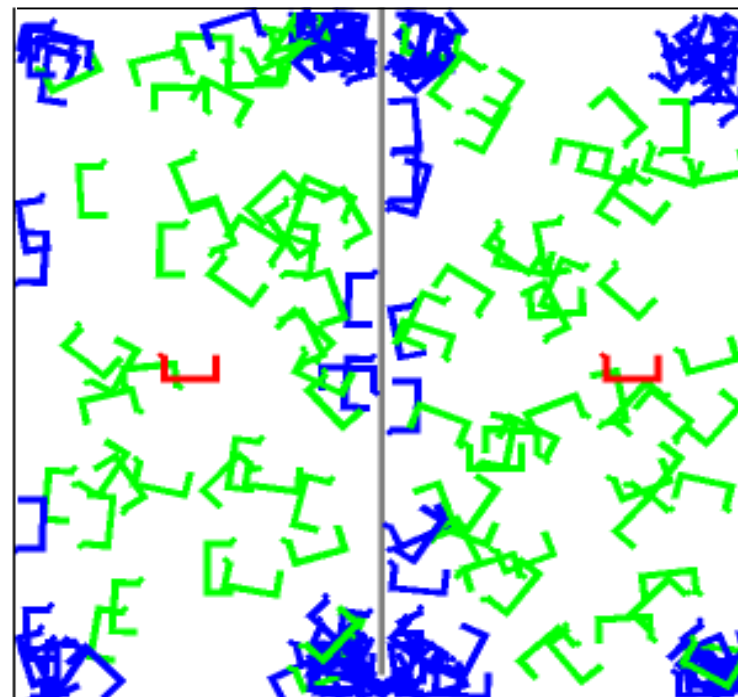
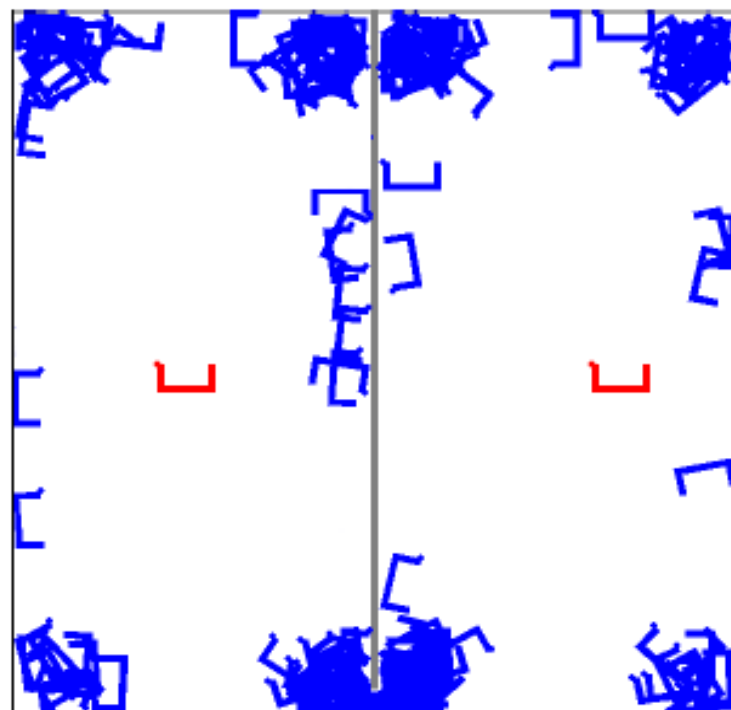


Hybrid Sampling

- 1) Sample two configurations q and q' using Gaussian sampling technique
- 2) If both are in free space, then retain one (any of the two) as a node with low probability (e.g., 0.1)
- 3) Else if only one is in free space, then retain it as a node with intermediate probability (e.g., 0.5)
- 4) Else if $q_m = (q+q')/2$ is in free space, then retain it as a node with probability 1



Uniform



Uniform
+
Bridge
test

Bridge
test

How to identify poor visibility regions?

What is the source of information? Robot and workspace geometry

How to exploit it?

- Workspace-guided strategies
 - Configuration guided strategies (OBPRM)
 - Filtering strategies
 - **Adaptive strategies**
 - Deformation strategies
-
- **Workspace-guided strategies**
Identify narrow passages in the workspace and map them into the configuration space
 - **Filtering strategies**
Sample many configurations, find interesting patterns, and retain only promising configurations
 - **Adaptive strategies**
Adjust the sampling distribution on the fly
 - **Deformation strategies**
Deform the free space, e.g., to widen narrow passages

Adaptive Strategies

Main idea:

Use intermediate sampling results to identify regions of the free space whose connectivity is more difficult to capture

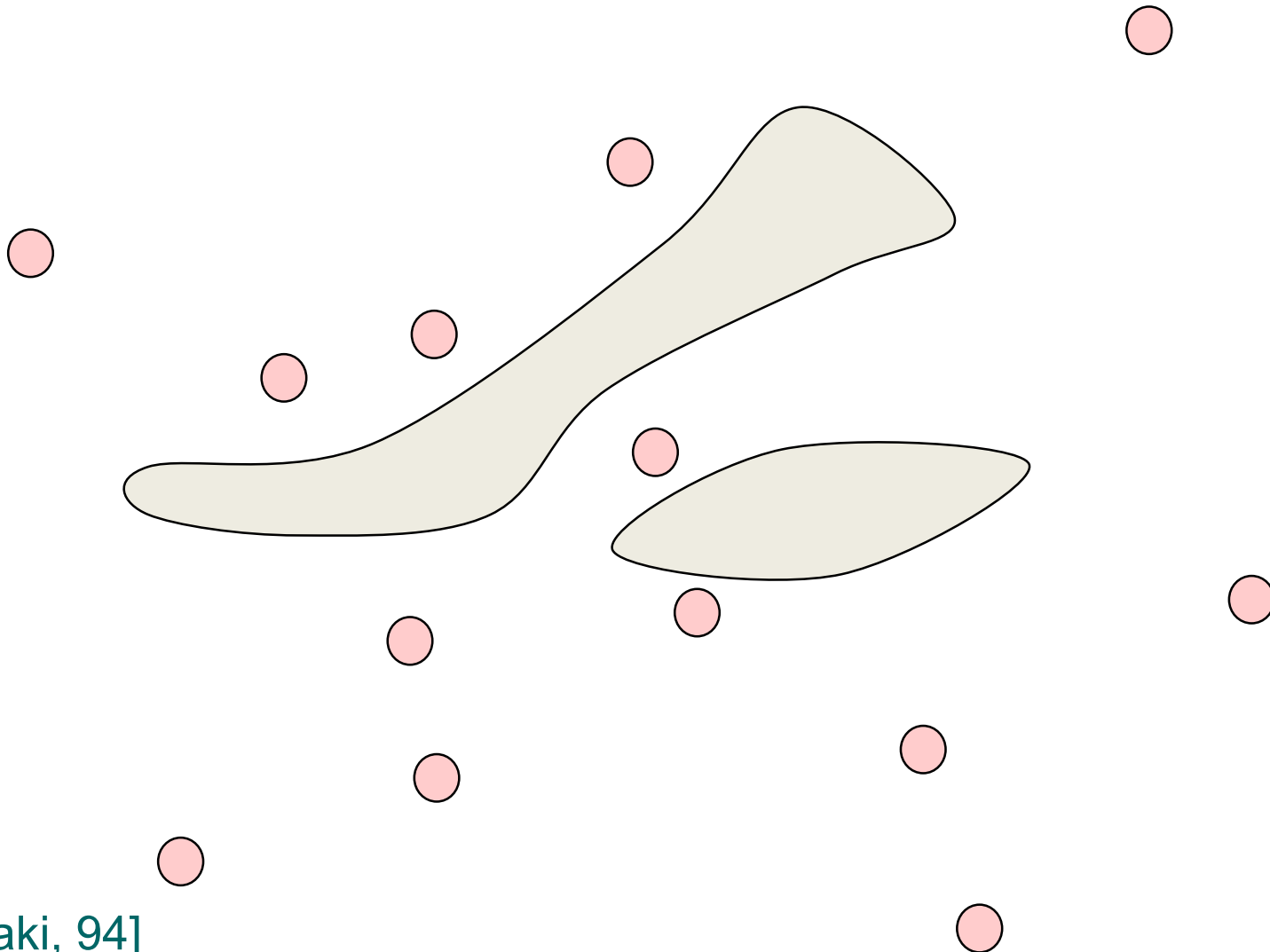
→ Time-varying sampling measure

Methods:

- Connectivity expansion
- Diffusion

Connectivity Expansion

Use work already done to detect poor-visibility regions

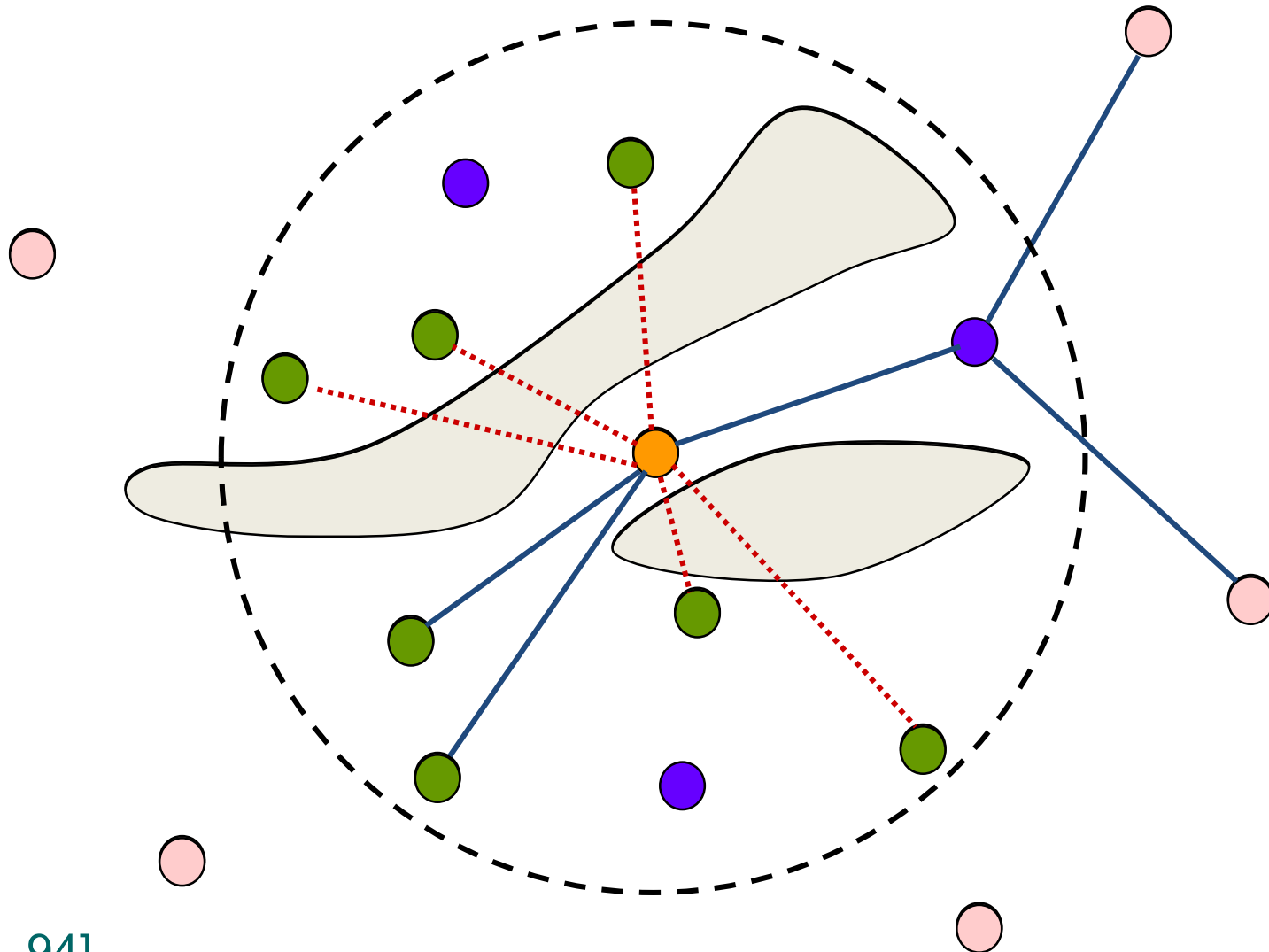


[Kavraki, 94]

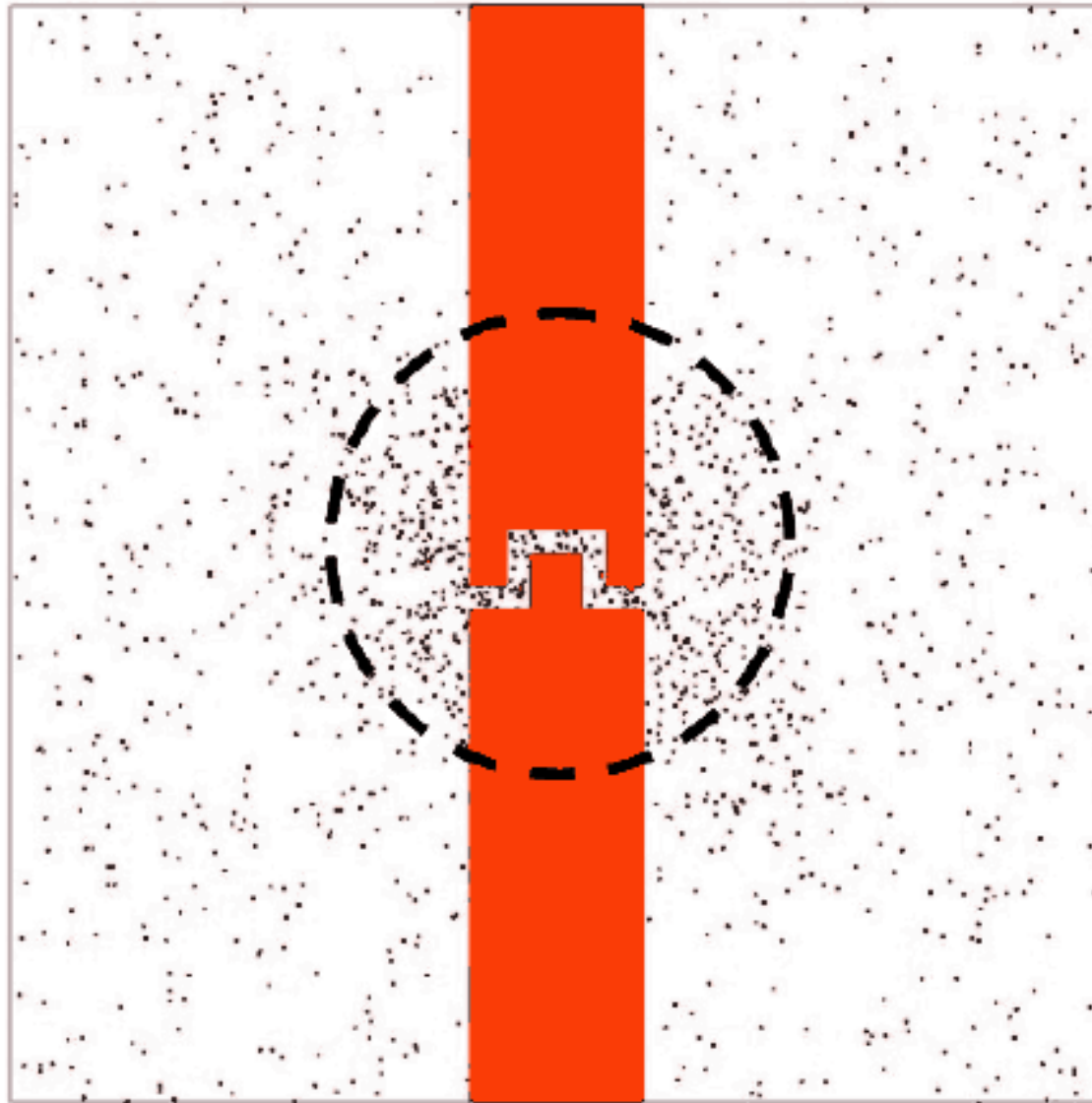
Connectivity Expansion

Use work already done to detect low-visibility regions

orange node: 2 of 7 nodes are “visible”



Example of Distribution



How to identify poor visibility regions?

What is the source of information? Robot and workspace geometry

How to exploit it?

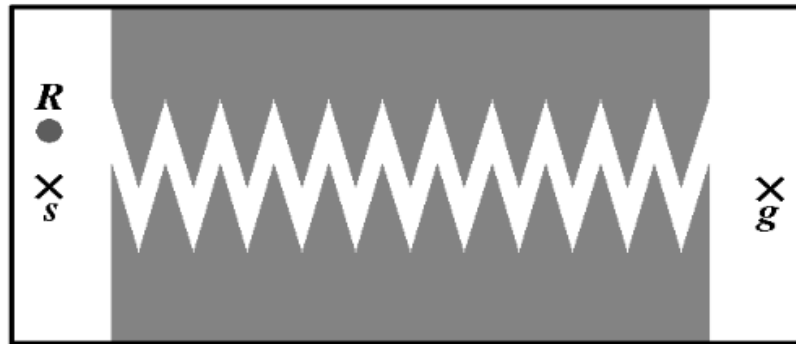
- Workspace-guided strategies
 - Configuration guided strategies (OBPRM)
 - Filtering strategies
 - Adaptive strategies
 - **Deformation strategies**
-
- **Workspace-guided strategies**
Identify narrow passages in the workspace and map them into the configuration space
 - **Filtering strategies**
Sample many configurations, find interesting patterns, and retain only promising configurations
 - **Adaptive strategies**
Adjust the sampling distribution on the fly
 - **Deformation strategies**
Deform the free space, e.g., to widen narrow passages

Deformation Strategies

Main idea: Deform the free space to make it more expansive

Method: Free space dilatation

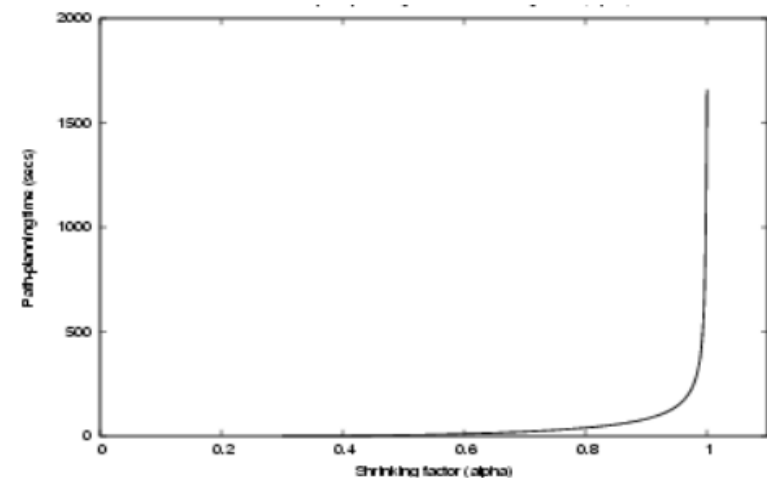
Motivating Experiment:



(a)

α	Path-planning time (secs)
1.000	1659.0
0.997	346.0
0.993	160.0
0.987	67.0
0.984	44.0
0.700	5.23
0.500	1.32
0.300	0.87

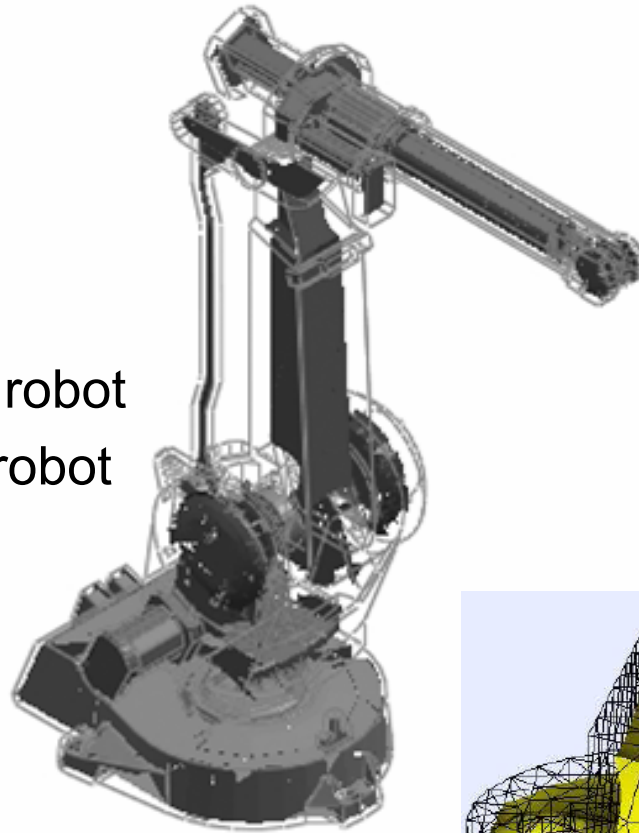
(b)



(c)

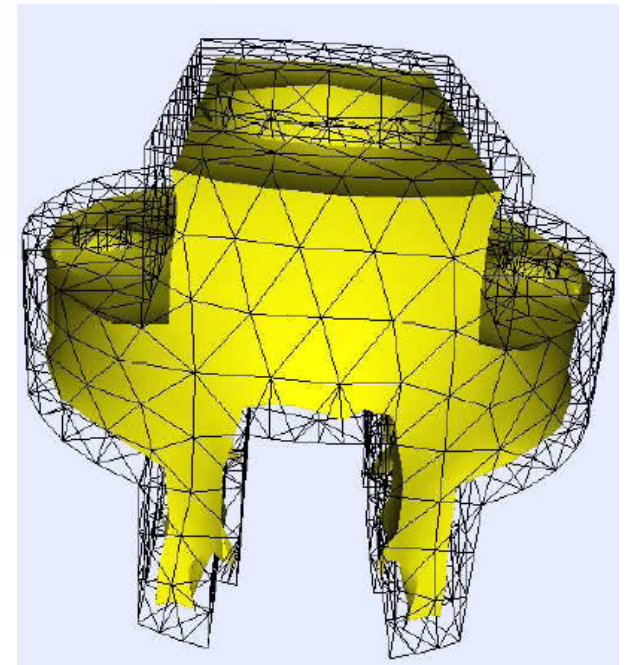
Free Space Dilatation

1. **Pre-computation:**
Slim the robot / obstacles
2. **Planning:**
 - Compute a path for slimmed robot
 - Deform this path for original robot

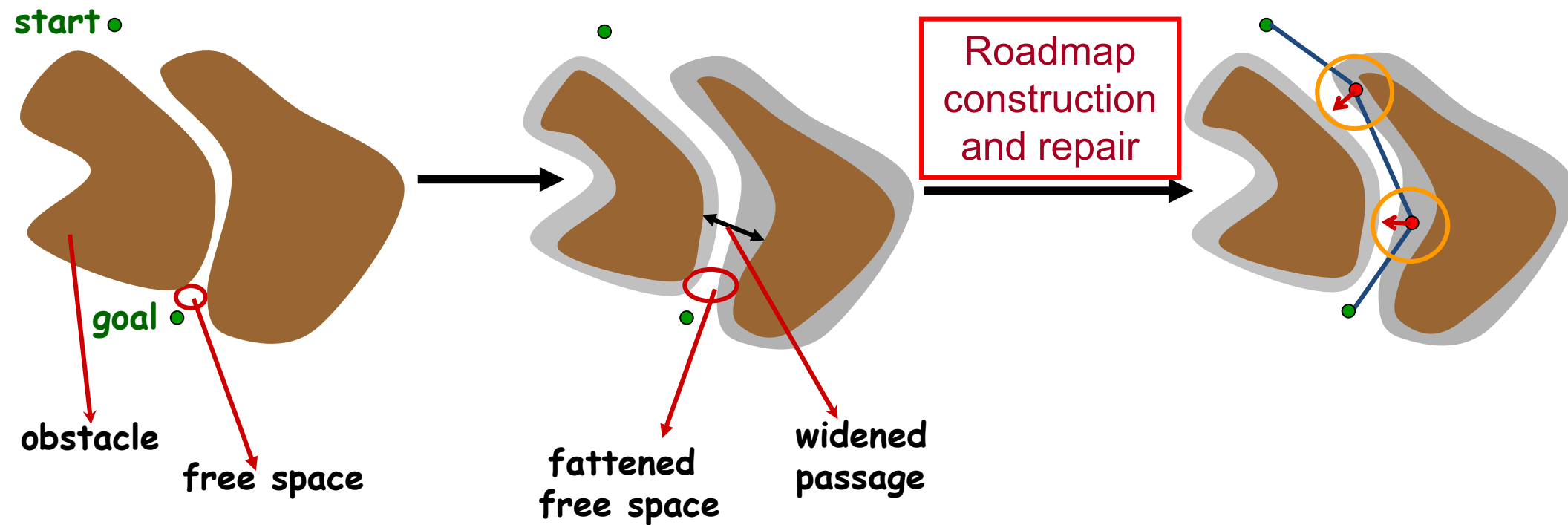


-M. Saha, J.C. Latombe, Y.-C. Chang, F. Prinz. Finding Narrow Passages with Probabilistic Roadmaps: The Small-Step Retraction Method. Autonomous Robots, 19(3):301-319, Dec. 2005.

-H.-L. Cheng, D. Hsu, J.-C. Latombe, and G. Sánchez-Ante . Multi-level free-space dilatation for sampling narrow passages in PRM planning. Proc. IEEE Int. Conf. on Robotics & Automation, 2006.



Free Space Dilatation

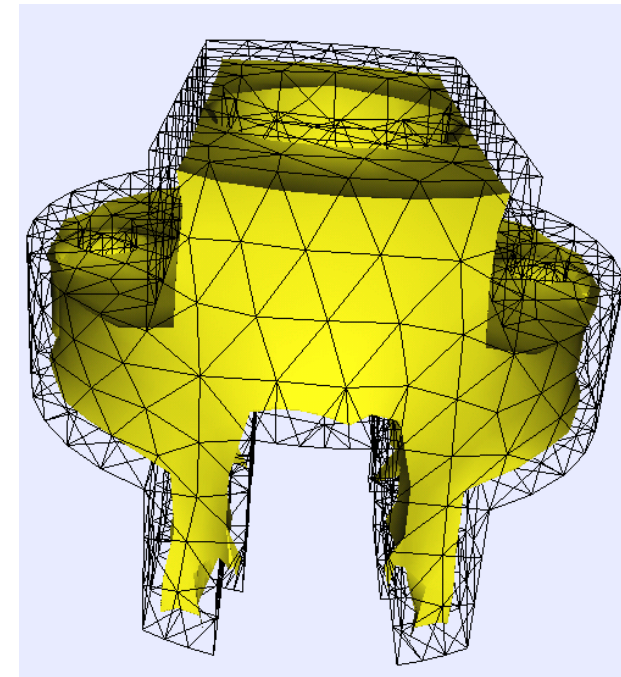


→ ~ up to 2 orders of magnitude speedup

Sampling inside the Narrow Passageways - Dilate

Dilation by shrinking geometric models of robots and/or obstacles

- guarantee that a shrunken model M is always contained inside the original model M so that F' is a dilation of F , i.e., $F \subseteq F'$.
- Advantages:
 - much simpler to implement.
 - This method can shrink M in a local region without affecting others and thus avoid unnecessarily changing the free space during dilation.
 - There is no need to re-compute the bounding volume hierarchy for collision checking when a new shrunken model is generated.
 - This method is efficient in both running time and memory usage.



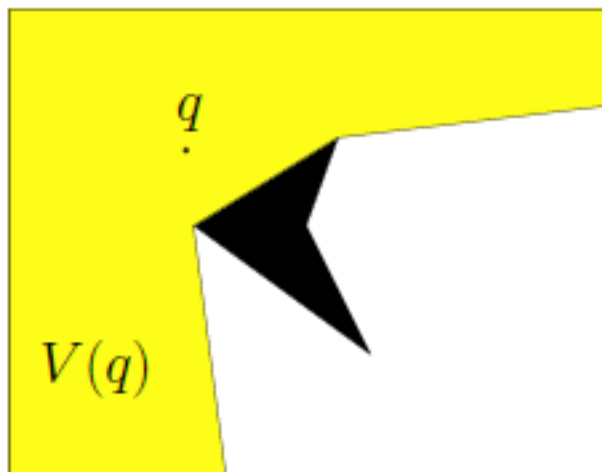
More on Sampling Strategies

-
- **Visibility-based**
- Manipulability-based
- Quasirandom
- Grid-based

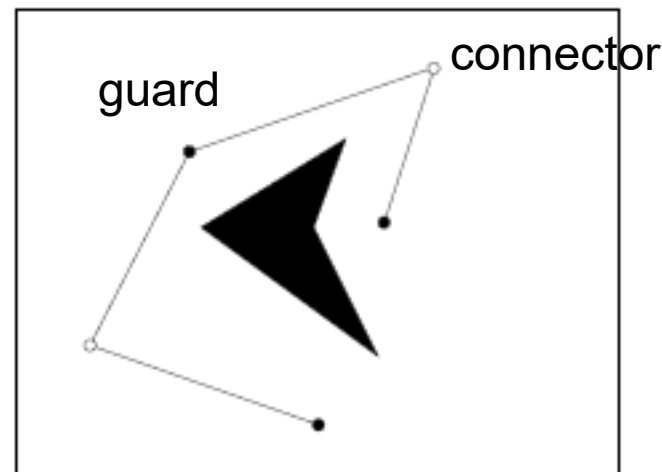
Sampling Strategies – Visibility Roadmap

Visibility Roadmap

- The roadmap representation is small yet covering C_{free} well.
- The running time is often greater than the basic PRM
- Two different kinds of vertices in G :
 - Guards: To become a guard, a vertex, q must not be able to see other guards. Thus, the visibility region, $V(q)$, must be empty of other guards.
 - Connectors: To become a connector, a vertex, q , must see at least two guards. Thus, there exist guards q_1 and q_2 , such that $q \in V(q_1) \cap V(q_2)$.



(a) Visibility definition



(b) Visibility roadmap

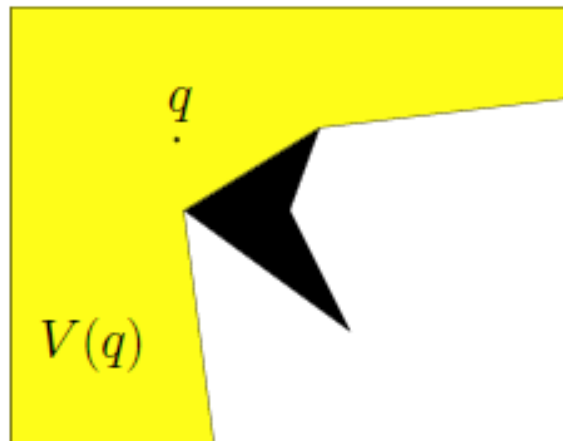
Guards: black dot , Connectors: white dot in the roadmap

Sampling Strategies – Visibility Roadmap

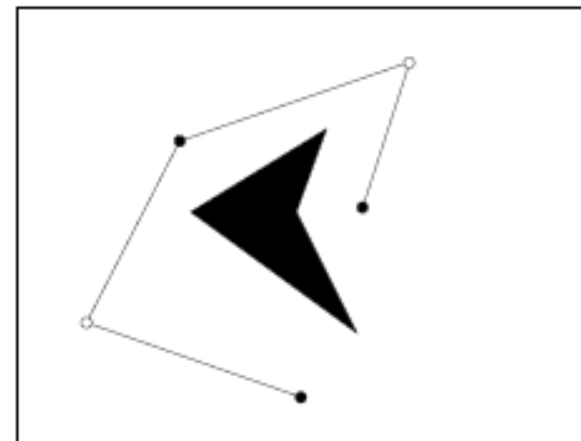
Strong criterion to determine whether to keep a sample and its associated edges in G .

Three possible cases for each sample:

1. The new sample is not able to connect to any guards. It will become a guard itself and is inserted into G .
2. The new sample can connect to guards from at least two **different connected components** of G . It will become a connector that is inserted into G along with its associated edges, which connect it to these guards from different components.
3. Neither of the previous two conditions were satisfied. This means that the sample could only connect to guards in the same connected component. It is discarded.



(a) Visibility definition

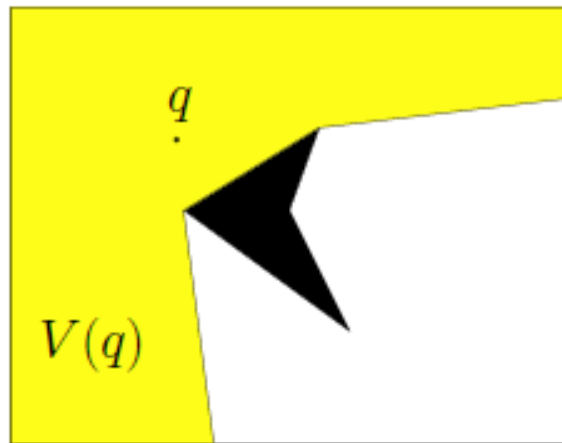


(b) Visibility roadmap

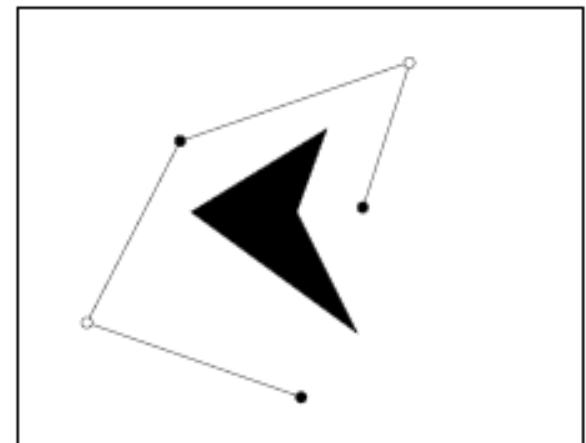
Sampling Strategies – Visibility Roadmap

Properties

- Case 3 reduces the number of roadmap vertices dramatically.
- The placement of guards depends strongly on the order in which samples appear in.
- Does not allow guards to be deleted in favor of better guards.
- The method may perform poorly if guards are not positioned well early in the sequence.
- It would be better to have an adaptive scheme in which guards could be reassigned in later iterations. Accomplishing this efficiently remains an open problem



(a) Visibility definition



(b) Visibility roadmap

Different Sampling Strategies

- Near obstacles
- Narrow passages
- Visibility-based
- **Manipulability-based**
- Quasirandom
- Grid-based

Manipulability Sampling

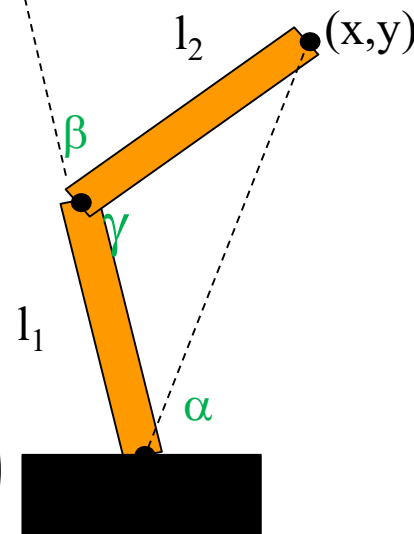
$$w(q) = \sqrt{\det(J(q)J^T(q))}$$

- Example: our favorite two-link arm:

$$J = \frac{\partial f}{\partial(\alpha, \beta)} = \begin{pmatrix} -l_1 s_\alpha - l_2 s_{\alpha+\beta} & -l_2 s_{\alpha+\beta} \\ l_1 c_\alpha + l_2 c_{\alpha+\beta} & l_2 c_{\alpha+\beta} \end{pmatrix}$$

- Two-link arm in a singularity: $\alpha = 90, \beta=0, l_1=l_2=1$:

$$J = \begin{pmatrix} -2 & -1 \\ 0 & 0 \end{pmatrix}, w(90,0) = 0$$



$$\begin{pmatrix} x \\ y \end{pmatrix} = f(\alpha, \beta) = \begin{pmatrix} l_1 c_\alpha \\ l_1 s_\alpha \end{pmatrix} + \begin{pmatrix} l_2 c_{\alpha+\beta} \\ l_2 s_{\alpha+\beta} \end{pmatrix}$$

More on Sampling Strategies

-
- Visibility-based
- Manipulatability-based
- **Quasirandom**
- **Grid-based**

Pseudorandom number generation

Computer-generated numbers are not random.

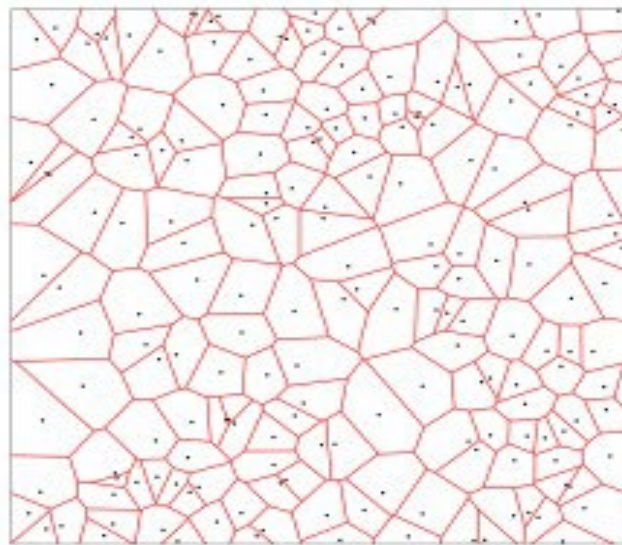
- A pseudorandom number generator is usually employed as a deterministic method that simulates the behavior of randomness.
- advantage of extending the samples over Cartesian products does not necessarily hold.
- Sometimes problems are caused by unforeseen deterministic dependencies.
- One of the best pseudorandom number generators for avoiding such troubles is the **Mersenne twister**
- Its period length is chosen to be a [Mersenne prime](#) $2^n - 1$.

Classical **linear congruential** pseudo-random number generator:

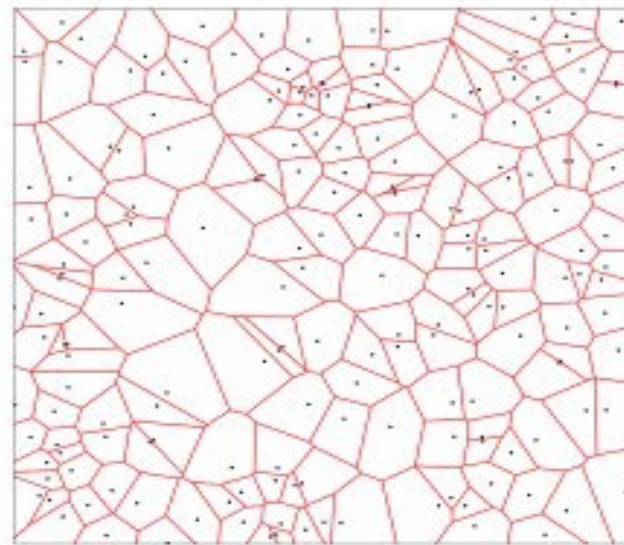
- uses three integer parameters, M , a , and c chosen by the user.
- M and a , must be relatively prime, meaning that $\gcd(M, a) = 1$.
- c must be chosen to satisfy $0 < c < M$.
- Using modular arithmetic: $y_{i+1} = (ay_i + c) \bmod M$
- starting with some arbitrary seed $1 < y_0 < M$.
- Pseudorandom numbers in $[0, 1]$ are generated by the sequence $x_i = y_i/M$.

Quasirandom Sampling (deterministic)

- Problem: Random sampling (biased or not) can be unpredictable and irregular
 - Each time you run your algorithm you get a different sequence of samples, so performance varies
 - In the limit, space will be sampled well, but in finite time result may be irregular



(a) 196 pseudorandom samples



(b) 196 pseudorandom samples

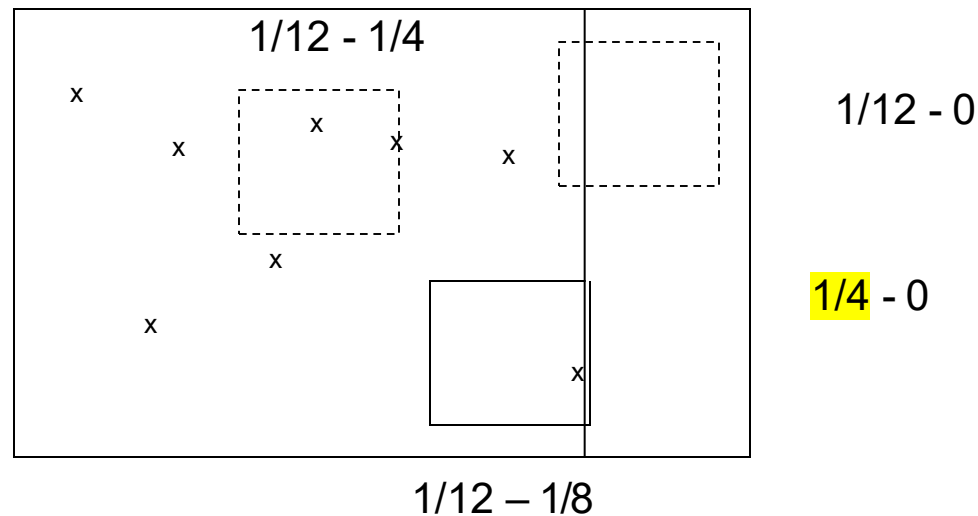
Irregularity in a collection of (pseudo)random samples can be nicely observed with Voronoi diagrams.

Quasirandom Sampling (deterministic)

- Discrepancy

$$D(P, \mathfrak{R}) = \sup_{R \in \mathfrak{R}} \left| \frac{\mu(R)}{\mu(X)} - \frac{|P \cap R|}{N} \right|$$

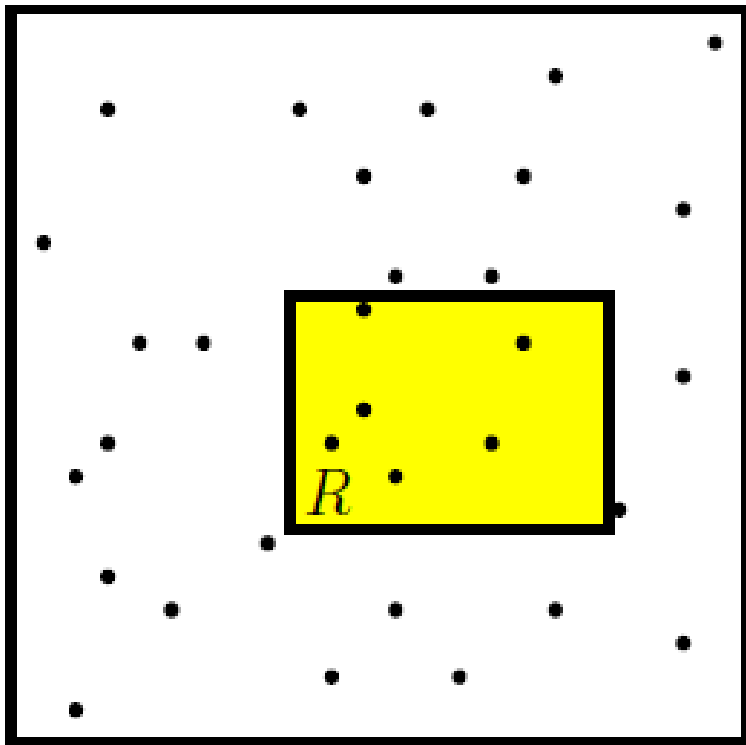
Samples
Set of partitions
Partition
Range Space
Number of points



intuitively, the “nonuniformity” of samples

Quasirandom Sampling (deterministic)

Discrepancy measures whether the right number of points fall into boxes.

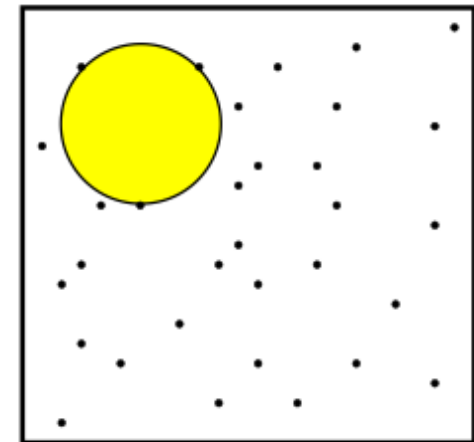
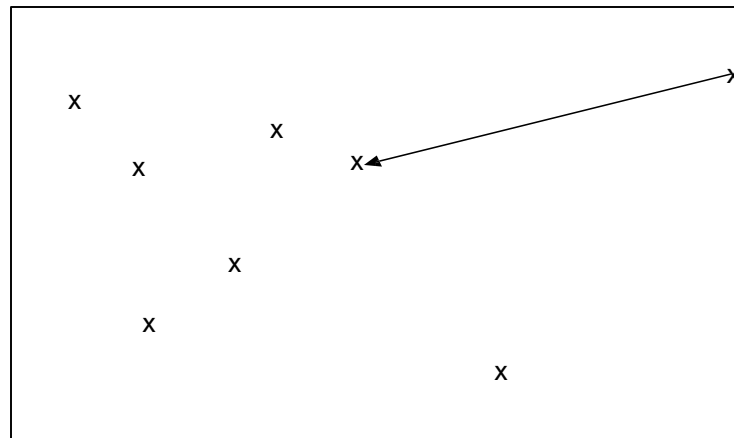


Quasi-Random Sampling

- Consider N points P in a space X and let $R \in \mathfrak{R}$ be some rectangular subset (i.e generalized interval) of X .

- Define **dispersion** $\delta(P) = \sup_{x \in X} \left\{ \min_{p \in P} \{ \rho(x, p) \} \right\}.$

- intuitively, the largest empty ball or “unoccupied” space

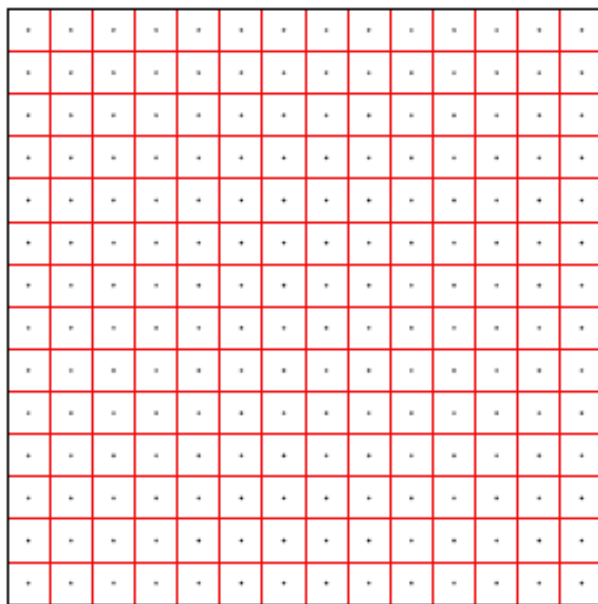


(a) L_2 dispersion

Grids: low dispersion sampling

Quasi-Random Sampling

- Example: Sukharev grid



(a) 196-point Sukharev grid
[0, 1]²

Sukharev sampling: ρ is L_∞ norm

$$\delta(P, \rho) \geq \frac{1}{2^{\lfloor N^{\frac{1}{d}} \rfloor}} = \frac{1}{2^8} = 0.0357$$

$$\delta^* \geq \frac{1}{2^{\lfloor N^{\frac{1}{d}} \rfloor}} \rightarrow N \geq \left(\frac{1}{2\delta^*} \right)^d$$

The number of cubes per axis

Quasi-Random Sampling

Relating dispersion and discrepancy

- for any $P \subset [0, 1]^d$,

$$\delta(P, L_\infty) \leq D(P, \mathcal{R})^{1/d};$$

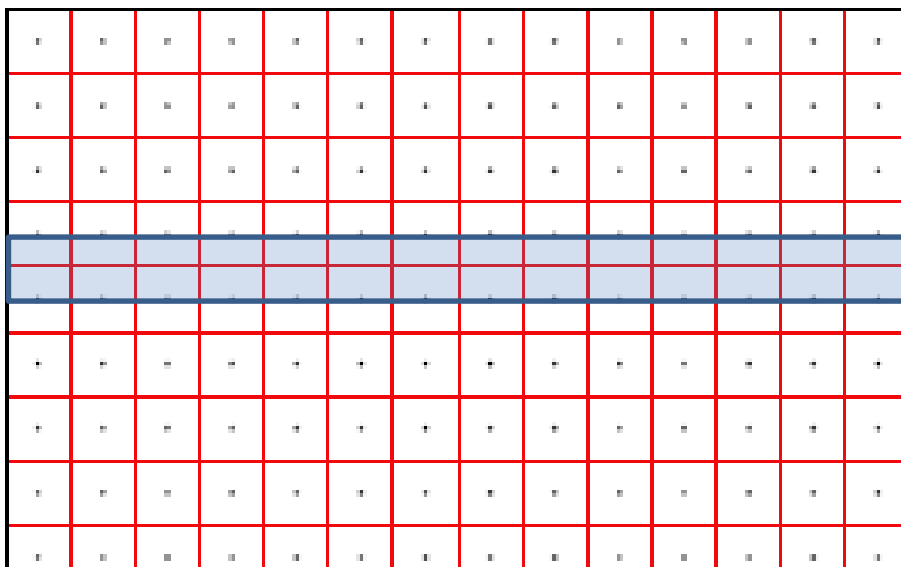
- low-discrepancy implies low-dispersion.
- The converse is not true:
- An axis-aligned grid yields high discrepancy because of alignments with the boundaries of sets in R , but the dispersion is very low.

Low discrepancy sampling

- In some applications, selecting points that align with the coordinate axis may be undesirable.
- Extensive sampling theory has been developed to determine methods that avoid alignments while distributing the points uniformly.
- In sampling-based motion planning, grids sometimes yield unexpected behavior because a row of points may align nicely with a corridor in C_{free} .

Quasi-Random Sampling

- Example: Sukharev grid



$$0.267 = \sqrt{\frac{1}{14}} = \sqrt{D(P, \mathcal{R})} \geq \delta(P, \rho) \geq \frac{1}{2^{\lfloor N^{\frac{1}{d}} \rfloor}} = \frac{1}{28} = 0.0357$$

Sampling Strategies

- **Van der Corput sequence** is **guaranteed to be dense** because it is deterministic.
- published in 1935 by van der Corput, a Dutch mathematician.
- it is only defined for the unit interval.
- Suppose that we want to place 16 samples in $C = [0,1]$.
 - $S = \{i/16 \mid 0 \leq i < 16\}$ evenly spaces the points at intervals of length $1/16$.
 - no point in C is further than $1/32$ from the nearest sample.

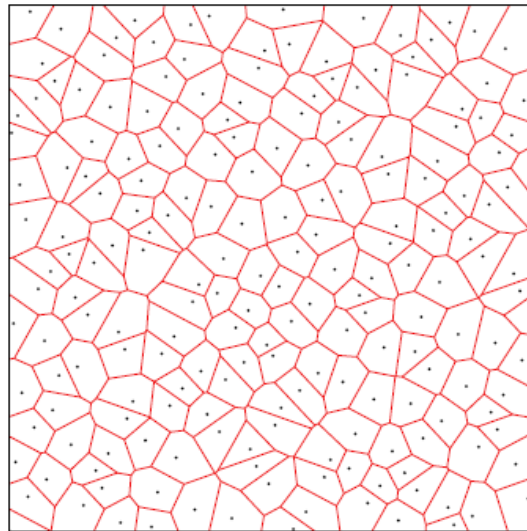
- Questions:

- What if we want to make S into a **sequence**?
- What is the best ordering?
- What if we are not even sure that 16 points are sufficient?
- Maybe 16 is too few or even too many

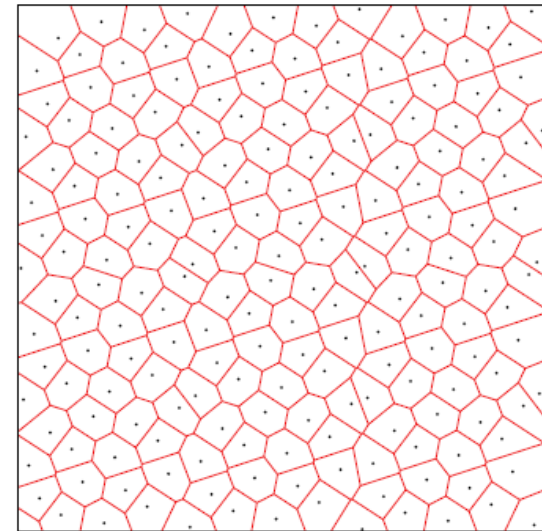
i	Naive Sequence	Binary	Reverse Binary	Van der Corput	Points in $[0, 1]/ \sim$
1	0	.0000	.0000	0	
2	1/16	.0001	.1000	1/2	
3	1/8	.0010	.0100	1/4	
4	3/16	.0011	.1100	3/4	
5	1/4	.0100	.0010	1/8	
6	5/16	.0101	.1010	5/8	
7	3/8	.0110	.0110	3/8	
8	7/16	.0111	.1110	7/8	
9	1/2	.1000	.0001	1/16	
10	9/16	.1001	.1001	9/16	
11	5/8	.1010	.0101	5/16	
12	11/16	.1011	.1101	13/16	
13	3/4	.1100	.0011	3/16	
14	13/16	.1101	.1011	11/16	
15	7/8	.1110	.0111	7/16	
16	15/16	.1111	.1111	15/16	

Quasi-Random Sampling

- Generalization of Van der Corput Sequence in higher dimensions
 - Halton sequence: n-dimensional generalization of van der Corput sequence
 - Hammersley Sequence: Adaptation of Halton sequence that yields a better distribution. BUT need to know number of samples in advance
 - Halton and Hammersley points are easy to construct (see book Lavallo)
 - Provide a nice alternative to random sampling that achieves more regularity.



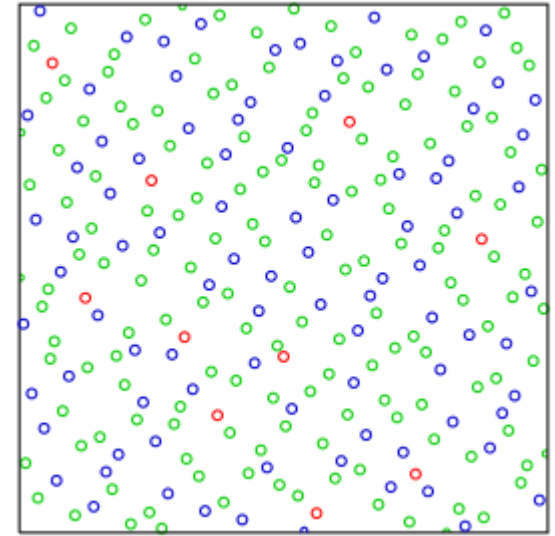
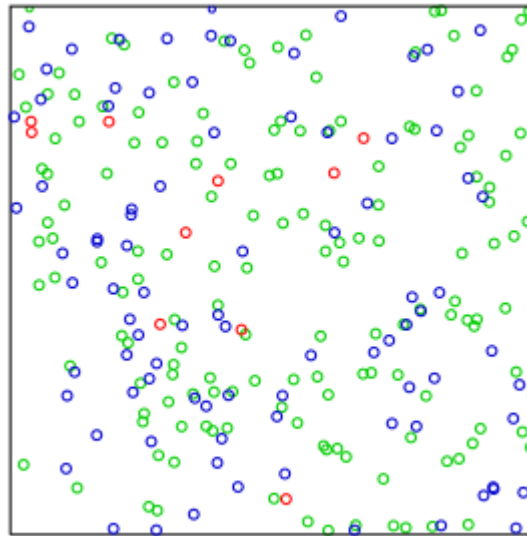
(a) 196 Halton points



(b) 196 Hammersley points

Halton Sequence (higher dims)

n	$\Phi_2(n)$	$\Phi_3(n)$
0	0	0
1	1/3	1/2
2	2/3	1/4
3	1/9	3/4
4	4/9	1/8
5	7/9	5/8
6	2/9	3/8
7	5/9	7/8
8	8/9	1/16
9	1/27	9/16
10	10/27	5/16
11	19/27	13/16
12	4/27	3/16
13	13/27	11/16
14	22/27	7/16
15	7/27	15/16



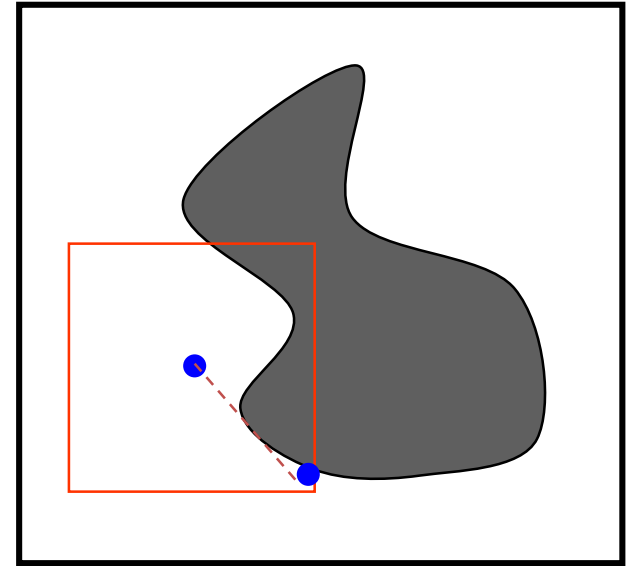
256 points from a pseudorandom number source (left) compared with the first 256 points from the 2,3 Halton sequence (right). The Halton sequence covers the space more evenly. (red=1,..,10, blue=11,..,100, green=101,..,256)

Connection Strategies

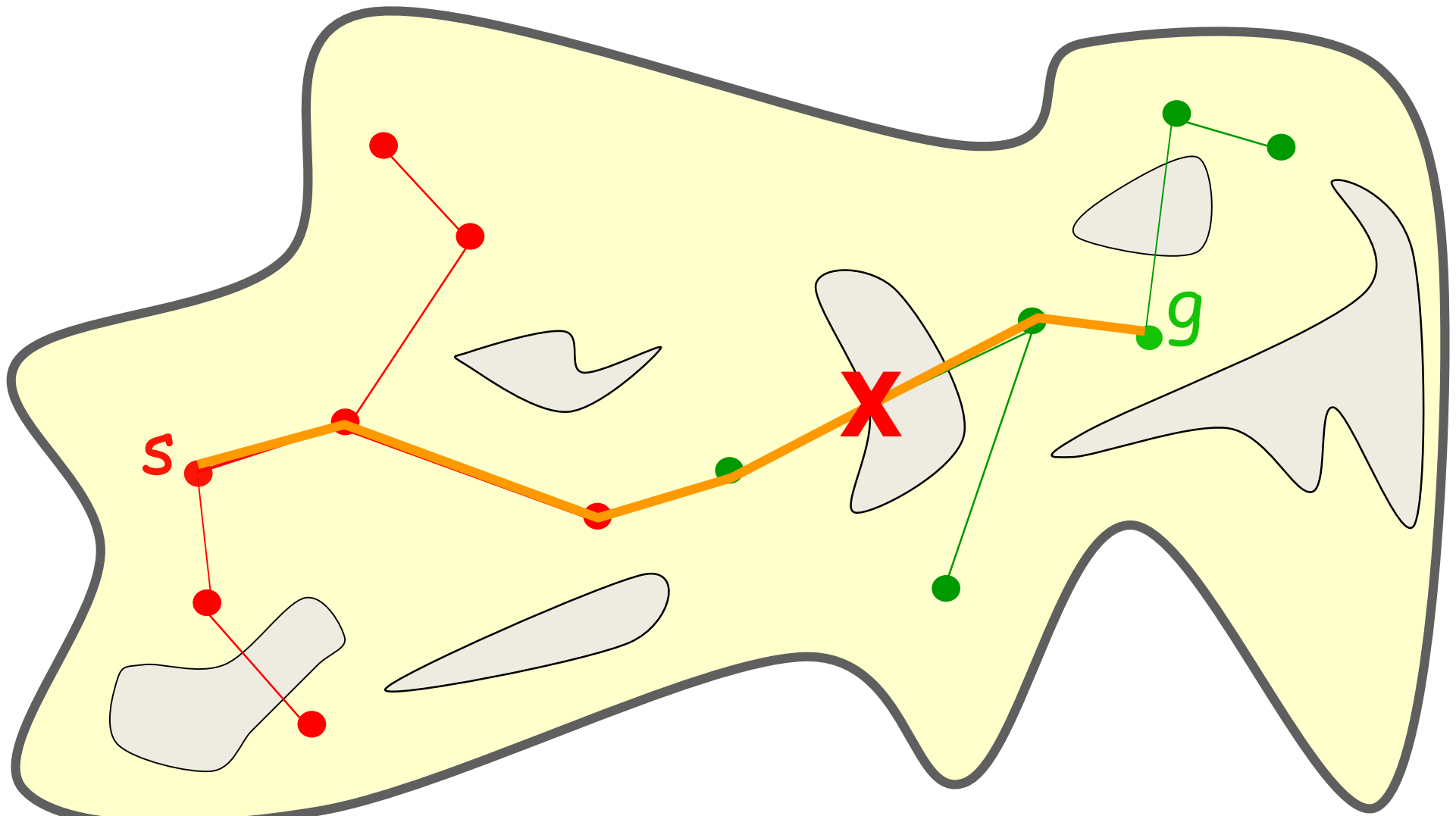
- Limit number of connections:
 - Nearest-neighbor strategy
 - Nearest K: The K closest points are considered (a typical value $K = 15$)
 - Component K: Try to obtain up to K nearest samples from each connected component of G. A reasonable value is $K = 1$; otherwise, too many tries
 - Radius: Take all points within a ball of radius r . An upper limit, K, may be set to prevent too many connections from being attempted. Typically, $K = 20$.
 - Visibility: try connecting to all vertices in G.
- Delay costly computation:
 - Lazy collision checking [Sanchez-Ante, 02]
- Increase expansiveness:
 - ...

Rationale of Lazy Collision Checking

- Connections between close milestones have high probability of being free of collision
- Most of the time spent in collision checking is done to test connections
- Most collision-free connections will not be part of the final path
- Testing connections is more expensive for collision-free connections
- Hence: Postpone the tests of connections until they are absolutely needed

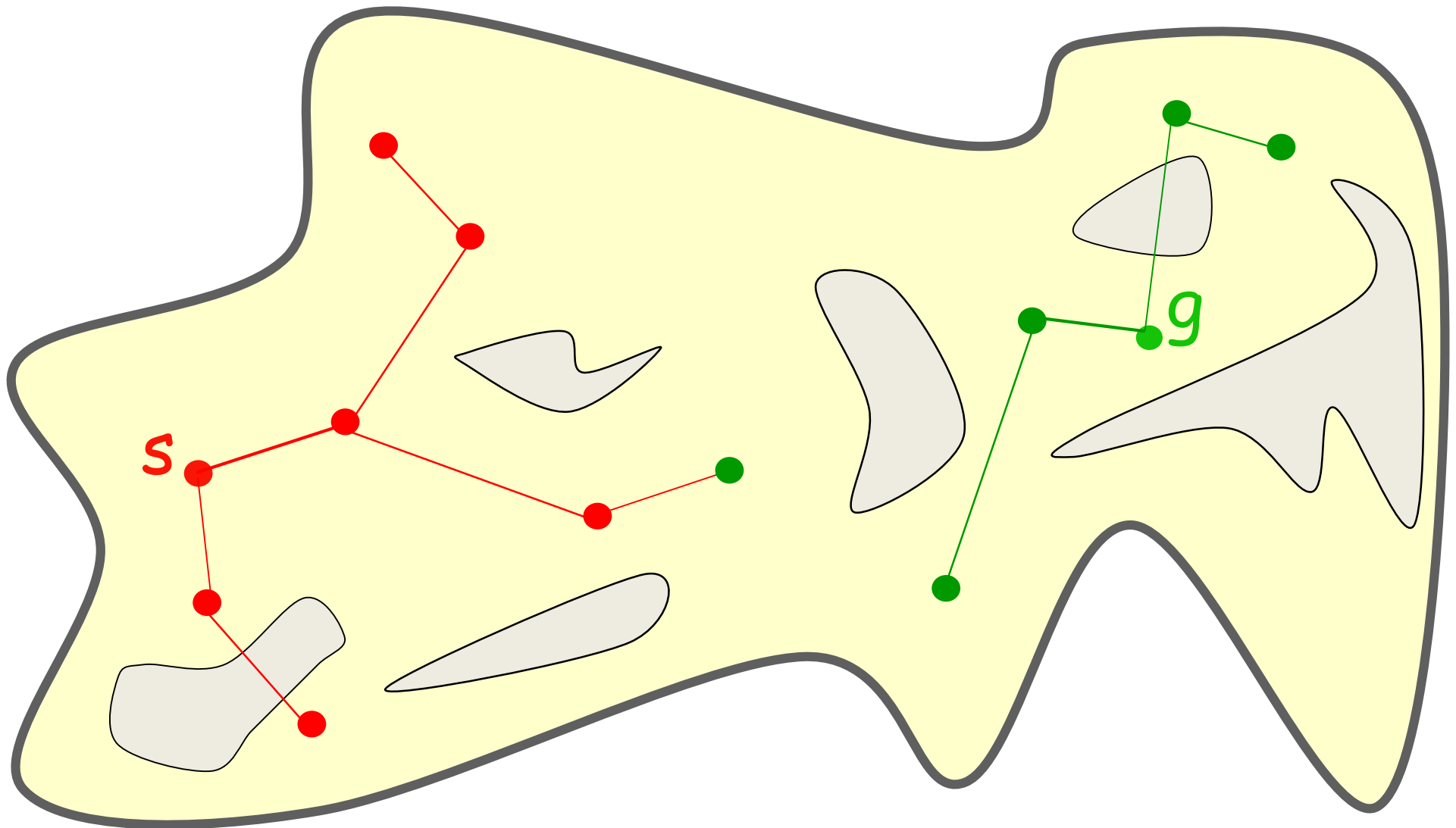


Lazy Collision Checking



[Sánchez-Ante, 2002]

Lazy Collision Checking

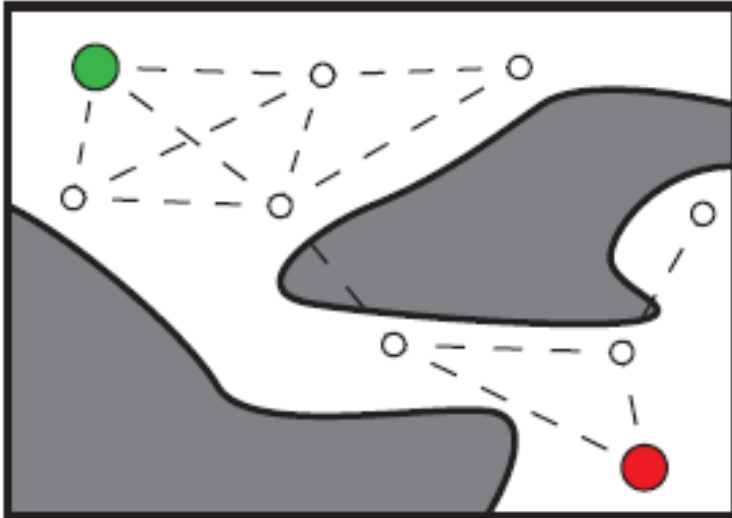


x10 speedup

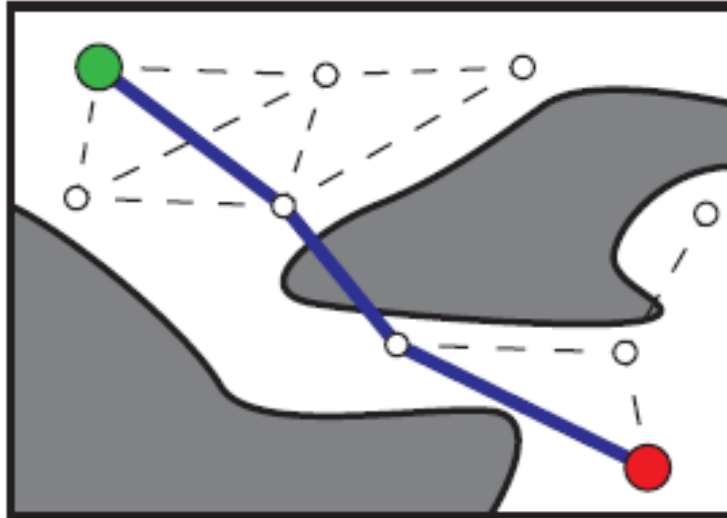
[Sánchez-Ante, 2002]

Lazy Collision Checking

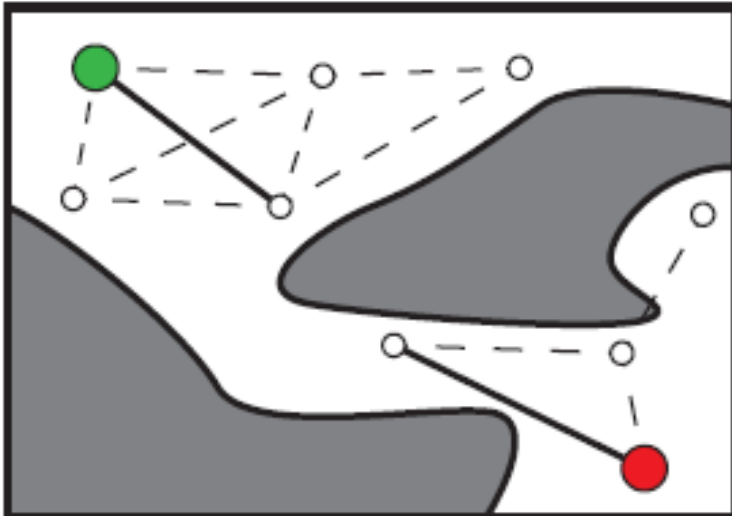
1



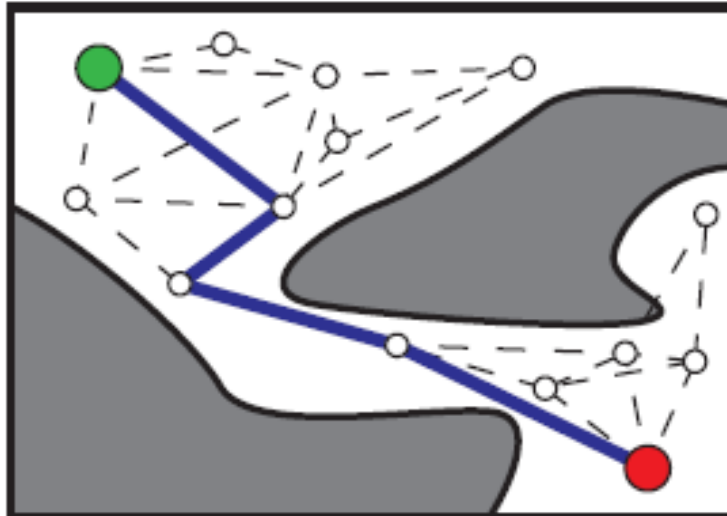
2



3



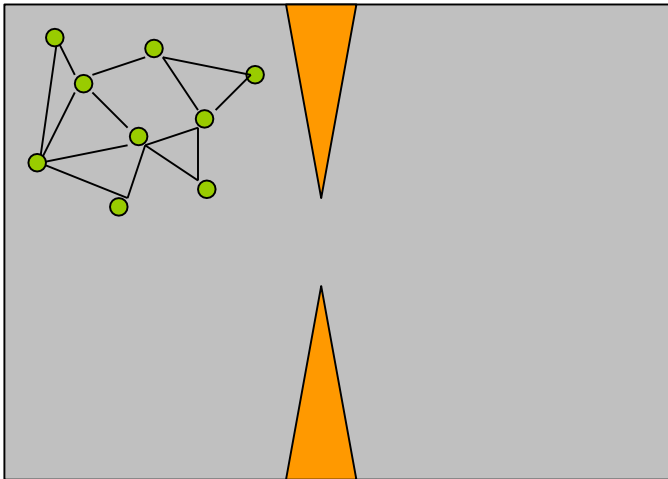
4



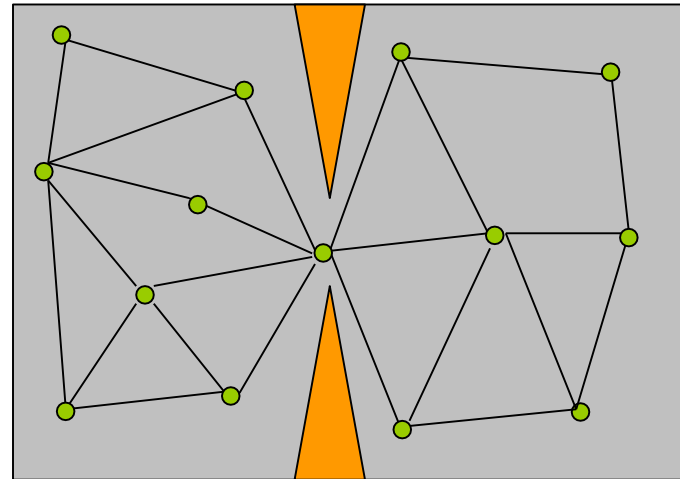
Is the coverage adequate?

- Milestones should be distributed so that almost **any** point of the configuration space can be connected by a straight line segment to one milestone.

Bad



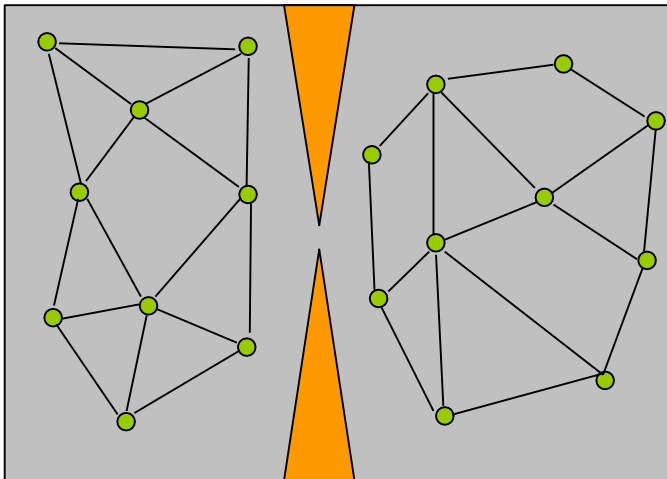
Good



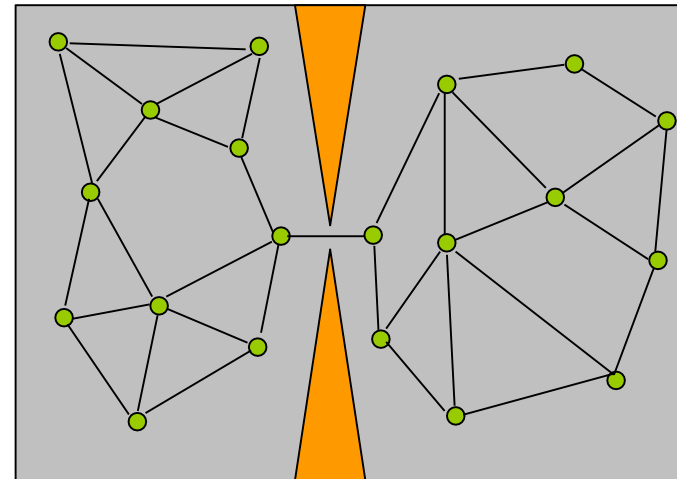
Connectivity

- There should be a one-to-one correspondence between the connected components of the roadmap and those of F .

Bad

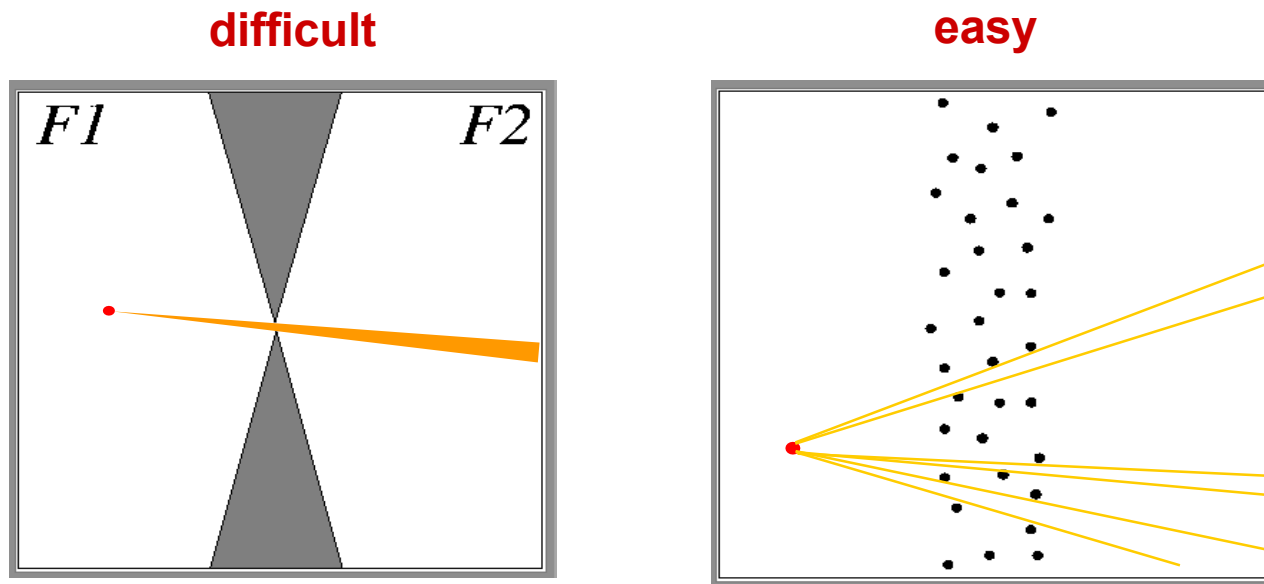


Good



Narrow passages

- Connectivity is difficult to capture when there are narrow passages.
- Narrow passages are difficult to define.

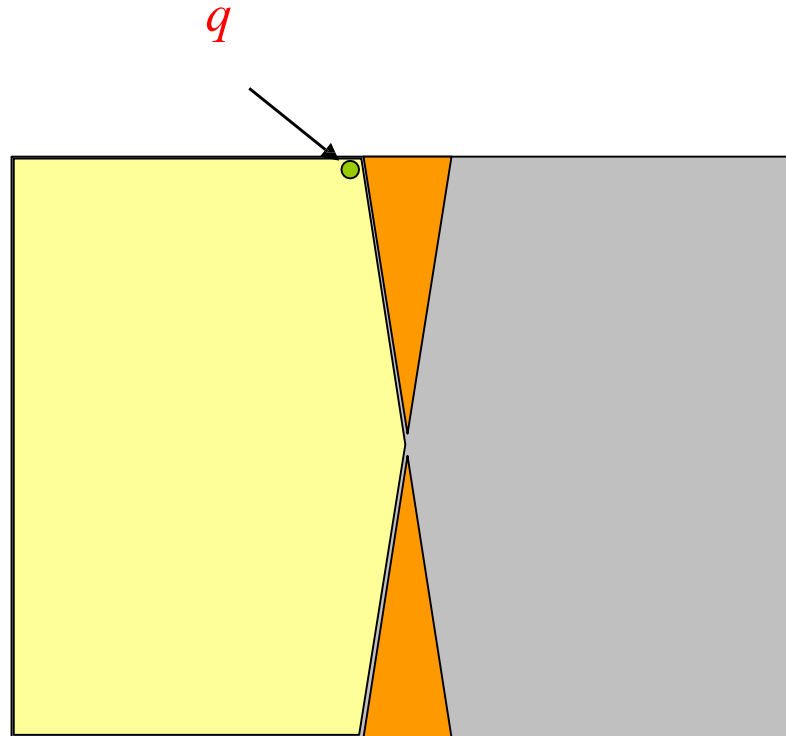


- How to characterize coverage & connectivity?

Expansiveness

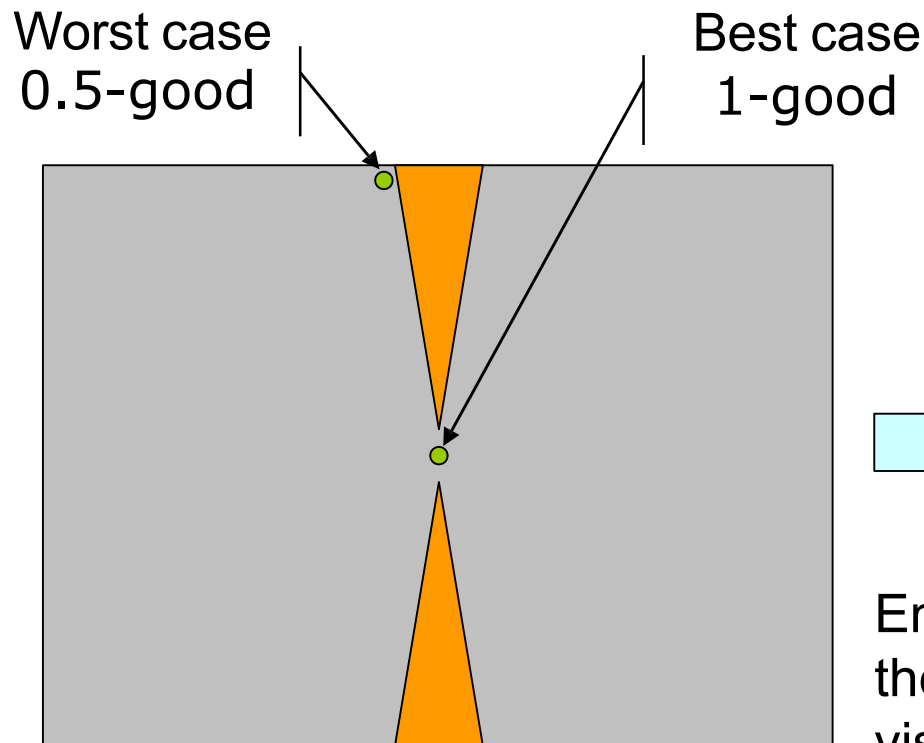
Definition: visibility set

- Visibility set of q
 - All configurations in F that can be connected to q by a straight-line path in F
 - All configurations seen by q

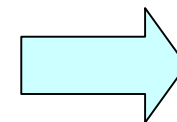


Definition: ϵ -good

- Every free configuration sees at least ϵ fraction of the free space, ϵ in $(0,1]$.



$$\epsilon(\mathcal{C}_{free}) = \min_{q \in \mathcal{C}_{free}} \left\{ \frac{\mu(V(q))}{\mu(\mathcal{C}_{free})} \right\}$$



F is 0.5-good

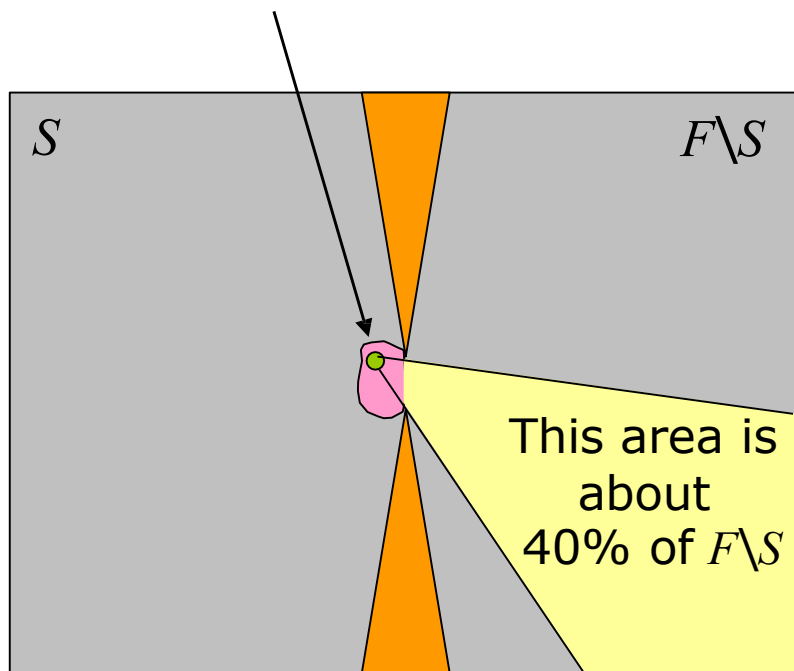
Ensures that a certain fraction of the free configuration space is visible from **any point** in the free configuration space

Definition: lookout of a subset S

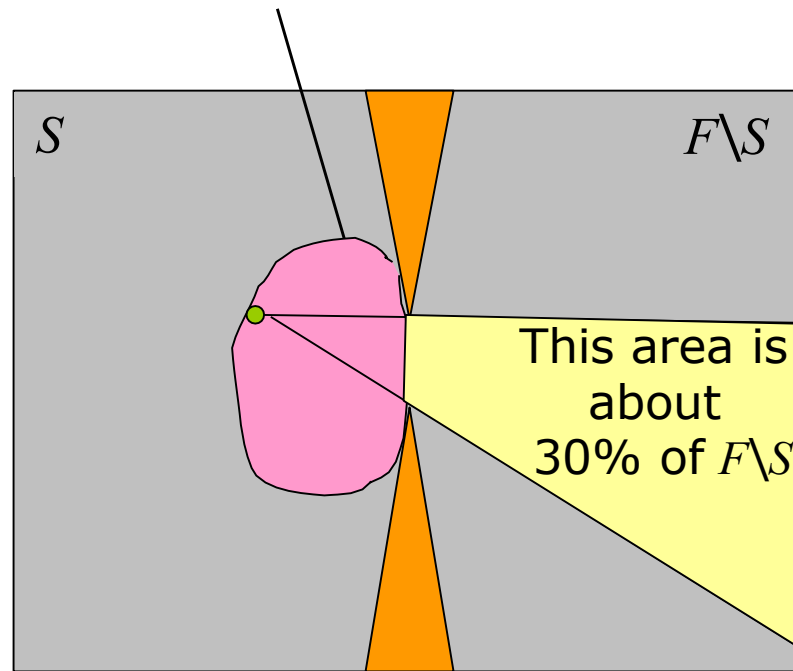
- Given a β is in $(0,1]$.
- Subset of points in S that can see at least β fraction of $F \setminus S$

Points in S that can see a β fraction of points not in S over all points not in S

0.4-lookout of S

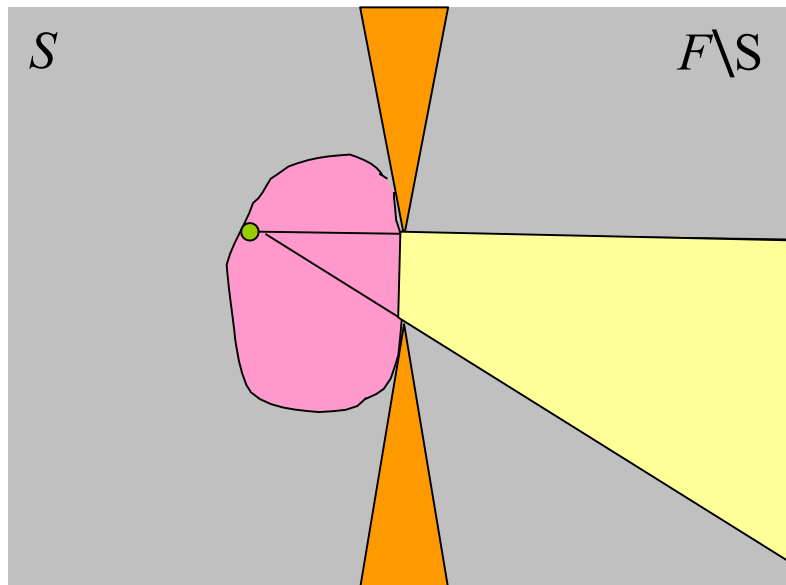


0.3-lookout of S



Definition: $(\epsilon, \alpha, \beta)$ -expansive

- The free space F is $(\epsilon, \alpha, \beta)$ -expansive if
 - Free space F is ϵ -good
 - For each subset S of F , its β -lookout is at least α fraction of S . ϵ, α, β are in $(0, 1]$



F is ϵ -good $\rightarrow \epsilon=0.5$

β -lookout $\rightarrow \beta=0.3$

$$\frac{\text{Volume}(\beta\text{-lookout})}{\text{Volume}(S)} \rightarrow \alpha=0.2$$

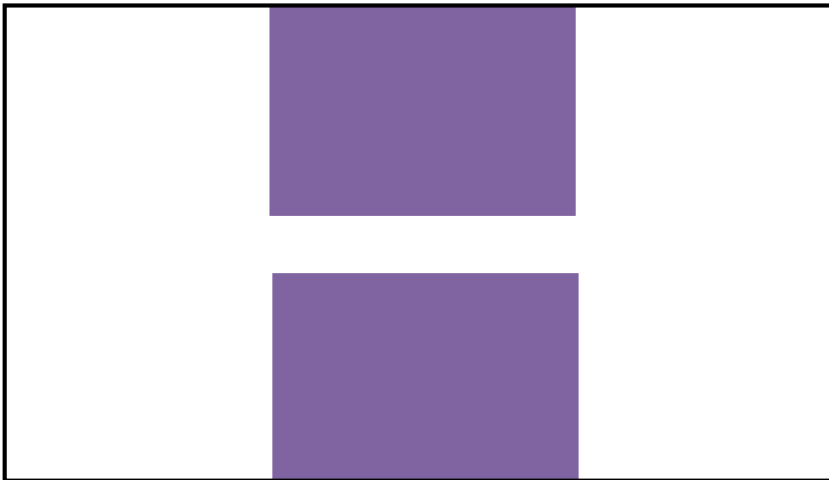
F is $(\epsilon, \alpha, \beta)$ -expansive,
where $\epsilon=0.5, \alpha=0.2, \beta=0.3$

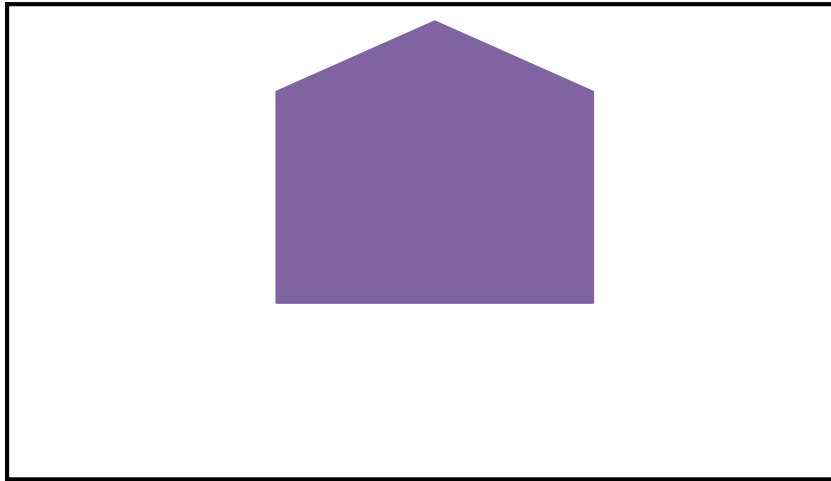
Comments

Expansiveness only depends on **volumetric ratios**

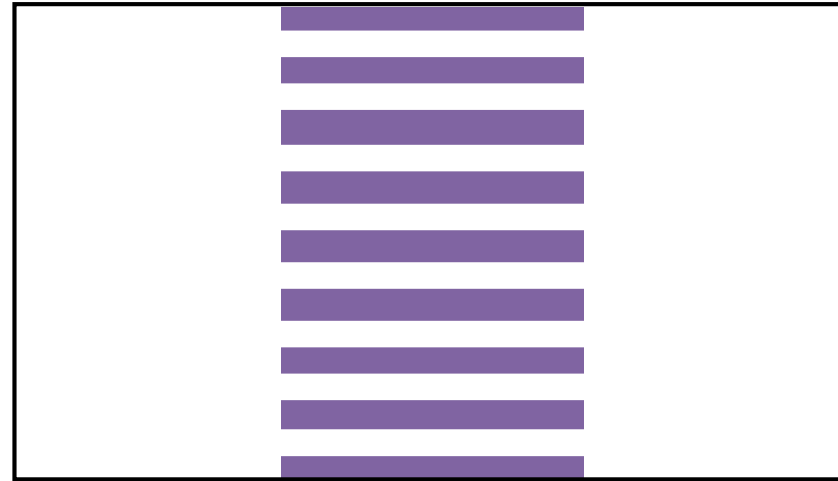
It is not directly related to the dimensionality of the configuration space

E.g., in 2-D the expansiveness of the free space can be made arbitrarily poor

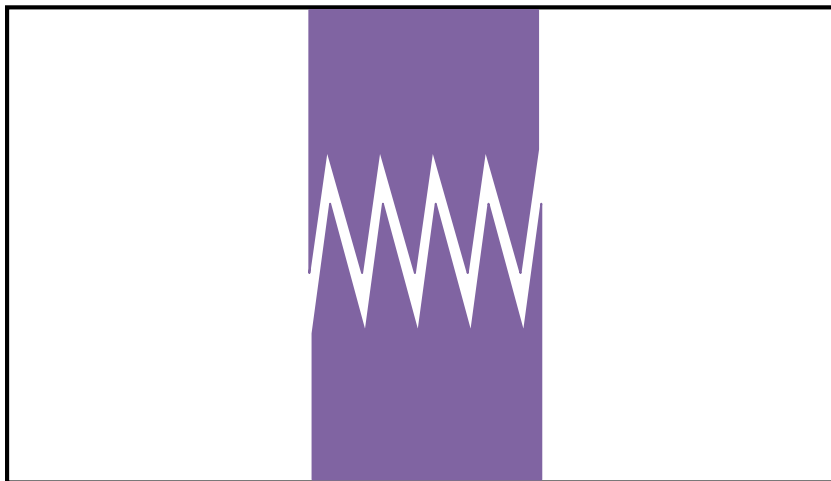




Thanks to the wide passage at the bottom this space is favorably expansive



Many narrow passages might be better than a single one



This space's expansiveness is worse than if the passage was straight



A convex set is maximally expansive, i.e., $\varepsilon = \alpha = \beta = 1$

Why expansiveness?

- ε , α , and β measure the expansiveness of a free space.
- Bigger ε , α , and β $\rightarrow$ easier to construct a roadmap with good connectivity and coverage.

Theorem 1 (Connectivity)

- Probability of achieving good **connectivity** **increases exponentially** with the number of milestones (in an expansive space).
- If ε , α , β decreases, then need to **increase the number of milestones** (to maintain good connectivity)

Path planning in expansive configuration spaces. Hsu, *et al.*, 1999.

Uniform sampling

Theorem 1: A roadmap of $2n+2$ uniformly-sampled milestones has the correct connectivity with probability at least $1 - \delta$, where

$$n = \left\lceil \frac{8 \ln(\frac{8}{\varepsilon \alpha \delta})}{\varepsilon \alpha} + \frac{3}{\beta} \right\rceil$$

Example:

$\varepsilon = 0.5; \beta = 0.4; \alpha = 0.2; \delta = 0.1$

$n = 543,$

1088 uniformly sampled configurations necessary

Theorem 2 (Coverage)

- Probability of achieving good **coverage**, **increases exponentially** with the number of milestones (in an expansive space).

Completeness

- Complete algorithms are slow.
 - A **complete** algorithm finds a path if one exists and reports no otherwise.
 - Example: Canny's roadmap method
- Heuristic algorithms are unreliable.
 - Example: potential field
- **Probabilistic completeness**
 - Intuition: If there is a solution path, the algorithm will find it with high probability.

Probabilistic completeness

In an expansive space, the probability that a PRM planner fails to find a path when one exists goes to 0 **exponentially** in the number of milestones ($\sim$ running time).

[Kavraki, Latombe, Motwani, Raghavan, 95]

[Hsu, Latombe, Motwani, 97]

Summary

□ Main result

- If a C-space is expansive, then a roadmap can be constructed efficiently with good connectivity and coverage.

□ Limitation in practice

- It does not tell you when to stop growing the roadmap.
- A planner stops when either a path is found or max steps are reached.
- $\varepsilon, \alpha, \beta$ are nearly impossible to compute in high-dimensional spaces